Imperial College London

Department of Electrical and Electronic Engineering

Project Title:	**Bilbo: A Domain-Specific Language for Graphs**
Student:	**Christopher Macca**
CID:	**01058868**
Course:	**Electronic & Information Engineering MEng**
Project Supervisor:	**Dr Thomas J.W. Clarke**
Second Marker:	**Dr James J. Davis**

Abstract

Graph structures are ubiquitous in science and engineering. Existing tools aimed at working with graphs either prioritise performance, providing an interface to common algorithms, or having formal semantics. These things impinge on the expressiveness and productivity of the tool by increasing the semantic gap between the graph problem and the software solution.

This work follows the design and reference implementation of Bilbo, a self-contained, domain-specific language for solving graph problems. Bilbo's design objective is to provide an expressive and productive way to work with graphs, and to do so at a level of abstraction that liberates programmers from unrelated language constructs and the handling of low-level data structures.

Bilbo achieves these objectives by offering a powerful graph transformation model, concise value-level syntax for constructing and manipulating graphs, and a novel graph concept with intuitive addition and subtraction. Furthermore, Bilbo embraces the inherent nondeterminism in graph transformation and provides a host of original constructs to both control and leverage this.

Acknowledgments

I would like to express my heartfelt gratitude to Dr Thomas Clarke for his support, vision, and invaluable insight throughout this project. I would like to thank him for his willingness to let me explore what I find interesting while challenging me at all times to take the project further.

I want to thank Thomas, Luke, Doug, Tom, and Mikkel for being exceptional friends. Thank you for making my time at Imperial an educational eden where we explored ideas together, thank you for the unabating comradery, and thank you for everything you have taught me, about engineering and otherwise.

Finally, I want to thank my parents, Rula and Jacques, and my brother, John-Jack. Thank you for your unwavering love and unending support. Without you, I would not be who I am, or where I am, today.

Contents

1 Introduction **9**

2 Background **13**

2.1 Technologies for Graph Programming . . . 13

2.1.1 Graph Databases . . . 13

2.1.2 Graph Libraries and Embedded DSLs . . . 14

2.1.3 Graph DSLs . . . 14

2.1.4 Graph Rewriting Tools . . . 14

2.2 Graph Rewriting As a Model of Computation . . . 15

2.2.1 Preliminaries . . . 16

2.2.2 The Double Pushout Approach . . . 17

2.2.3 The Single Pushout Approach . . . 19

2.2.4 Negative Application Conditions . . . 20

2.2.5 Nondeterminism . . . 20

2.2.6 Applying Production Rules In Sequence . . . 22

2.2.7 Computational Completeness of Graph Rewriting . . . 22

2.3 Existing Graph Rewriting Tools . . . 23

2.3.1 PROGRES . . . 23

2.3.2 AGG . . . 24

2.3.3 FUJABA . . . 25

2.3.4 GrGen . . . 25

2.3.5 GP 2 . . . 26

2.3.6 Program State Within Graph Rewriting Tools . . . 27

2.3.7 Other Tools That Provide Graph Rewriting . . . 27

2.4 Summary . . . 29

3 Requirements Capture **31**

3.1 Tool Format . . . 31

3.2 System-Level Requirements . . . 34

3.3 Type System and Program State Requirements . . . 35

3.4 Graph-Specific Requirements . . . 36

3.5 Reference Implementation Requirements . . . 37
3.6 Summary . . . 37

4 Analysis and Design **39**
4.1 Tool Mode . . . 39
4.2 Type System . . . 41
4.2.1 A Strong Type System . . . 41
4.2.2 A Dynamic Type System with Heterogenous Graphs . . . 41
4.3 The Bilbo Reference Implementation . . . 44
4.3.1 Reference Implementation Format . . . 44
4.3.2 Implementation Langauge . . . 45
4.4 Summary . . . 46

5 The Design of Bilbo **47**
5.1 Bilbo Basics . . . 48
5.1.1 Variables and Primitive Types . . . 48
5.1.2 Printing and Importing . . . 48
5.1.3 User-Defined Types and Objects . . . 50
5.2 Graph Construction . . . 51
5.2.1 Nodes, Edges and Paths . . . 51
5.2.2 Graph Addition and Subtraction . . . 53
5.2.3 Path Comprehensions . . . 57
5.2.4 Nested Paths . . . 58
5.2.5 Outputting Graphs . . . 59
5.3 Graph Manipulation . . . 61
5.3.1 Transforms . . . 61
5.3.2 Multiple Matches, Nondeterminism and Collections . . . 63
5.3.3 Transform Composition and Pipelines . . . 65
5.3.4 Generalised Application Conditions . . . 66
5.4 Pipeline Modifiers and Reductions . . . 70
5.4.1 Multiplicative Application . . . 70
5.4.2 As-Long-As-Possible Application . . . 70
5.4.3 One-Match Application . . . 72
5.4.4 Optional Application . . . 74
5.4.5 The Or Pipe . . . 74
5.4.6 Reductions . . . 75
5.5 Other Bilbo Features . . . 78
5.5.1 Functions . . . 78
5.5.2 Manipulating Nodes In-Place . . . 79
5.6 Summary . . . 80

6 The Bilbo Interpreter **81**
6.1 Implementation of the Interpreter . . . 81
6.1.1 Overview of The Interpreter . . . 82
6.1.2 Binding Pattern Graphs . . . 84
6.1.3 Combing Pattern Graphs . . . 85
6.2 Testing the Interpreter . . . 89
6.2.1 Testing in the Early Stages of Development . . . 89
6.2.2 Testing in the Later Stages of Development . . . 90
6.2.3 Randomised and Property-Based Testing . . . 91

7 Evaluation **93**
7.1 Solving Graph Problems In Bilbo . . . 93
7.1.1 Constructing the Transitive Closure of a Graph . . . 93
7.1.2 The Shortest Path Problem . . . 96
7.1.3 Where to Build a Bridge? . . . 100
7.1.4 The Vertex Colouring Problem . . . 103
7.1.5 Summary . . . 104
7.2 Usability Testing . . . 105
7.2.1 Tool for Comparison . . . 105
7.2.2 Participants . . . 105
7.2.3 Experimental Design . . . 106
7.2.4 The Questions . . . 107
7.2.5 Results and Statistical Analysis . . . 107
7.2.6 Threats to Validity . . . 110
7.2.7 Participant Feedback . . . 111
7.2.8 Summary . . . 112
7.3 Revisiting the Requirements . . . 112
7.3.1 Summary . . . 112

8 Conclusion **115**
8.1 Achievements . . . 115
8.2 Future Work . . . 116
8.2.1 The Open Source Development of Bilbo . . . 117
8.2.2 Future Language Features . . . 117
8.3 Limitations . . . 118
8.4 Summary . . . 119

A Worksheet for Usability Testing Study **121**

B Worksheet Results **135**

C Worksheet Question Template **137**

D Bilbo Tutorial for Usability Testing Study **139**

E ReGraph Tutorial for Usability Testing Study **145**

F Bibliography **151**

Chapter 1

Introduction

Graphs are everywhere. They provide a simple and expressive way to represent data. A node is used to represent an object or state and an edge is used to represent a relationship between those objects or states [1].

The idea of concentrating on the relationships between entities, as well as the entities themselves, is often citied as the reason that graphs lend themselves to representing highly interconnected data [2]. Relationships are materialised explicitly, rather than requiring ad hoc joining operations to create them, as is required when using tabular data. For example, compare the portion of the London Underground map presented in its normal form, as a graph, in Fig 1.1, with a tabular form of the same data in Table 1.1. While the graph makes use of colours and additional icons, it is easy to see that the tabular form breaks the important relationships between the stations, which the graph form faithfully preserves.

To wit, in bioinformatics graph databases have been effectively used to explore complex relationships between heterogenous data [4]. In geography, solutions to water scarcity are being

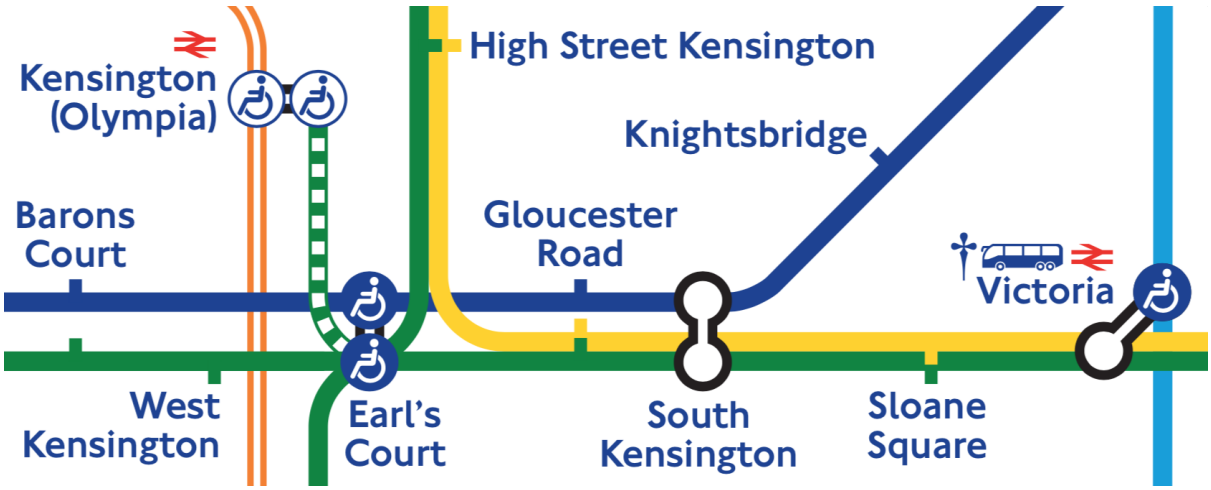


Fig 1.1: A portion of the London Underground map [3].

Source	Target	Line
West Kensington	Earl's Court	District
Earl's Court	Gloucester Road	District
Gloucester Road	South Kensington	District
South Kensington	Knightsbridge	Piccadilly
⋮	⋮	⋮

Table 1.1: A portion of the London Underground map presented in tabular form.

developed by representing water systems as graphs and looking for patterns of wasted water [5]. The large scale management of IoT (internet of things) networks is being made simpler using graphs [6]. Public transport systems have been modelled as graphs and optimised using concepts from graph theory [7], making commutes shorter and cheaper. Global money laundering has even been detected in real time using graph databases [8].

The generality of graphs as a way storing information also makes them common in the study and development of algorithms and data structures. Foundational data structures such as binary trees and linked lists are themselves specific kinds of graphs, while others such as hash tables and arrays can be readily recast as graphs.

Relationships modelled as graphs benefit from the umbrella of graph theory. Many obvious questions, when given a graph, have their theoretical analog. Asking for the quickest way to get from one node to another is the shortest path problem, asking whether it is possible to visit all the nodes exactly once is the Hamiltonian path problem, and asking for the quickest route that visits all nodes and returns to the original is the travelling salesman problem. While there exists algorithms for solving these problems, they remain an active area of research [9].

Graph programming can be used as a blanket term for working with graphs in a programmatic way. Many technologies for graph programming already exist, in a variety of formats.

- Graph databases allow querying highly interconnected data stored [10].
- Graph libraries, within general-purpose programming languages, provide ready-to-use interfaces to common graph algorithms [11].
- DSLs (domain-specific languages) for graphs concentrate on being highly performant by offering constructs for parallelism or control of memory allocation [12].
- Graph rewrite tools repeatedly transform graph towards a problem solution and have desirable theoretical properties [13].

However, each of these formats, as a consequence of their desired use case, have properties that impinge on their general productivity in graph programming.

Graph databases require the data to be stored within the database, and they concentrate on querying, rather than changing the graph. They also have added complications due concurrency and transactional concerns, derived from their use as information storage and retrieval systems. Graph libraries are subject to the syntax and semantics of the languages they reside in, which are unlikely to be designed for productivity in graph programming. DSLs that concentrate on efficiency require the programmer to manage low-level data structures, increasing the semantic gap[1] between the graph problem and the software solution. Finally, graph rewriting tools offer limited constructs outside of the theoretical frameworks they are based on. As a result, a number of constructs a programmer may expect, such as variables, do not exist in rewriting tools.

The ubiquity of graphs in science and engineering motivates the need for a tool that enables programmers to solve graph problems productively. Such a tool should be at a level of abstraction suited to graphs, allowing one to focus on the problem domain rather than the semantics of unrelated language constructs and the handling of low-level data structures.

This project follows the design of Bilbo, a completely self-contained language for programming with graphs. Bilbo's design objectives are expressivity and productivity in graph programming. Alongside the design of the language, this project also seeks to deliver a working reference implementation of the tool.

The rest of this report is structured as follows:

- **Background** reviews the main technologies for graph programming and their underlying computational models. Graph rewriting, the theoretical framework upon rewriting tools are based, is motivated and handled formally in a category-theoretic approach. Existing tools which use graph rewriting are also reviewed.
- **Requirements Capture** sets the main requirements for the rest of the project. Some early design choices, to allow for more detailed requirements, are also justified in this chapter.
- **Analysis and Design** outlines the overall design of Bilbo, and justifies the major design choices made regarding tool mode (visual or textual), the type system, and the implementation language.

[1] A *semantic gap* is the difference in meaning between constructs formed under different representations [14]. Here it used to describe the fact that, solving a problem in a given language requires adhering to the semantic idiosyncrasies of that language (things such as pointers, data structures, and other constructs) that are unrelated to the original graph problem. The two representations are therefore the original graph problem and its solution in software. The difference between these two is the semantic gap.

- **The Design of Bilbo** describes and motivates the semantics of the tool.
- **The Bilbo Interpreter** provides an outline of the Bilbo interpreter and explores two interesting and challenging parts of its implementation. This chapter also discusses the testing approaches used.
- **Evaluation** is split into three sections. Firstly, well-known graph problems and their solutions in Bilbo are presented and discussed. Secondly, the usability testing study, its results, and their validity are presented. Lastly, the requirements set at the start of the project are revisited.
- **Conclusion** summarises the project. The achievements and limitations of the work are reviewed. The future plans for the tool are also discussed.

Chapter 2

Background

Graph programming tools can come in a number of different formats. In order to produce an effective tool for graph programming this chapter first reviews the prominent technologies for graph programming. These are then examined with regards to their model of computation, allowing a more fundamental comparison rather than simply comparing the specific semantics of each technology. From this, graph rewriting, a model of computation specific to graphs, is motivated and presented in a category-theoretic approach. Lastly, the existing tools that use the graph rewriting model of computation are discussed.

2.1 Technologies for Graph Programming

The main technologies for graph programming are graph databases, graph libraries, graph DSLs, and rewriting tools. These each have intended use cases which motivate the constructs they provide. This section provides a brief overview of these, informing the choices made in later sections of the project.

2.1.1 Graph Databases

A graph database is a data storage and retrieval system designed to treat the relationships between data with equal importance as the data itself [15]. Graph databases aim to provide expressive querying capabilities to get information out of the graph. As a result, graph problems that require changing the graph may be difficult to solve using a graph database.

The graph queries are expressed in query languages that provide constructs such as traversals and pattern searches. Traversals offer the ability to start at a particular node in the graph and traverse a path based on constraints given in the query. Pattern searches allow searching for a particular layout of nodes and edges, as well as the data they hold, within a graph. Graph databases also offer a selection of ready-to-use common graph algorithms as queries, such as a shortest path query, as well as the ability to aggregate or summarise data from the graph. If a graph represented buildings as nodes, one could query for the shortest path between two

buildings or for the tallest building in the graph. Examples of graph query languages and their corresponding databases are Cypher for Neo4J [16], Gremlin for TinkerPop [17], and SPARQL for RDF graphs [18].

Database technologies are also designed with transactional concerns in mind such as the ACID (atomicity, consistency, isolation, durability) properties, as well as the ability to be queried concurrently. While these features are not inherently negative, they make the tool larger, more complicated, and do not increase productivity in graph programming.

2.1.2 Graph Libraries and Embedded DSLs

Libraries and embedded DSLs[1] (domain-specific languages) are collections of functionality available from within general-purpose programming languages. There exist a large number of libraries and embedded DSL aimed at graph programming. For example NetworkX [11] in Python and the Boost Graph Library [20] in C++. Such tools intend to provide an interface to implementations of commonly used graph algorithms. This makes them useful when an existing application written in a particular language (the *host language*) needs to solve a graph problem.

2.1.3 Graph DSLs

Graph DSLs such as GraphIt [12], Green-Marl [21], and Ligra [22] are standalone languages (rather than existing within a host language like libraries and embedded DSLs) with their own syntax and semantics, focused on graph programming. In recent times, there has been a significant interest in parallelising graph algorithms through DSLs [22]. As a result, such tools (including the three previously mentioned) tend to be aimed at high-performance graph programming rather than language expressivity and productivity.

2.1.4 Graph Rewriting Tools

Graph rewriting systems such as PROGRES [23] and AGG [24] are used to solve graph problems by repeatedly finding particular subgraphs in the input graph, called a *host graph*, and performing a transformation on those subgraphs, yielding a new graph.

This computational model, of repeatedly transforming the graph, is called *graph rewriting* and is covered in detail in Section 2.2. Graph rewriting is popular in theoretical computer science communities [25] and therefore rewriting tools, such as those mentioned above, provide constructs based on the theoretical framework. As a result of this, these tools are often used within a very specific problem domain or for formal reasoning, rather than for solving general graph problems. A number of graph rewriting tools are compared in Section 2.3.

[1] While the difference between a library and an embedded DSL is not well-defined, embedded DSLs can be seen as libraries where particular attention has been paid to how the library features compose [19].

2.2 Graph Rewriting As a Model of Computation

Graph libraries and embedded DSLs are generally used within host general-purpose programming languages and are therefore subject to the host language's paradigm. This is likely to be imperative, object-oriented, functional or logical [26]. These paradigms have generic language constructs due to the general-purpose nature of the languages, and therefore no language constructs specifically for graph programming. Graph DSLs are usually designed for high-performance computing (as mentioned in Section 2.1.3) and so have low-level language constructs that allow performant graph operations to be written.

Computation in graph databases and rewriting tools is based on repeatedly finding subgraphs in the host graph. This is what a pattern-based query within graph databases is [27]. In graph rewriting tools, this is taken further, and the subgraph is transformed within the host graph according to a rule. This model of computation[2] is called *graph rewriting*.

The computation abstractions used within graph DSLs, libraries, and embedded DSLs, are not specifically designed for graphs. They are designed to be domain-agnostic. On the other hand, graph rewriting is model of computation that *is* designed specifically for graphs. This motivates reviewing graph rewriting in more detail.

There are multiple approaches to formalising the process of finding a subgraph in a host graph and transforming it. The two most prominent are the double and single pushout approaches based on category theory, a field of mathematics that concerns formalising structures [29].

This section starts by reviewing the double and single pushout, and then reviews common extensions to the theoretical framework used within programming tools, such as those covered in Section 2.3. The extensions address two concerns that arise from the double and single pushout, these are *negative application conditions* and *nondeterminism*. Negative application conditions describe how the absence of a subgraph can be searched for (for example, a node that is *not* connected to any other node). Nondeterminism deals with the case where multiple valid subgraphs are found in the host graph.

[2] A tool's *model of computation* is the computational abstractions provided by that tool, which allow a programmer to express operations [28]. For example, assembly programming allows reading the bits stored at a particular memory address, while Python allows iterating over objects of distinct types stored in a list. The difference can be regarded as coming from the model of computation provided by each tool.

Both approaches specify a production rule p in terms of a

- left-hand side graph L that is to be matched to a subgraph within the host graph G, and a
- right-hand side graph R to which the matched subgraph is to be transformed.

This process yields a new graph H to which further production rules can be applied. Intuitively this transformation can be seen as deleting, from the matched subgraph, items[3] that appear in L but not in R and adding to the matched subgraph items that appear in R but not in L. This is called a *direct derivation* and is shown notationally as $G \overset{p}{\Rightarrow} H$. The double and single pushouts specify the precise semantics of the matching and the transformation. These are now covered.

2.2.1 Preliminaries

This section contains preliminary definitions for labelled graphs, sub graphs, and graph morphisms which are required in constructing the double and single pushout.

Definition 2.2.1. Let $\Omega = (\Omega_V, \Omega_E)$ be sets that represent the alphabets for *node labels* and *edge labels* respectively. A <u>labelled graph</u>

$$G = (V, E, s, t, l, w)$$

is a directed graph consisting of

1. the *node set* V and *edge set* $E \subseteq V \times V$,
2. the functions $s : E \to V$ and $t : E \to V$ which are, respectively, the *source function* and *target function* for an edge, and
3. the functions $l : V \to \Omega_V$ and $w : E \to \Omega_E$ which are, respectively, the *node labelling function* and *edge labelling function.*

Definition 2.2.2. A graph $G' = (V', E', s', t', l,' w')$ is a <u>subgraph</u> of $G = (V, E, s, t, l, w)$ if $V' \subseteq V$, $E' \subseteq E$ and the mappings s', t', l', w' are restrictions (functions with the same mappings but from subsets of the original function domains) of s, t, l, w respectively.

[3] The *items* of a graph G refers to the nodes and edges of G and is used for brevity. This is standard terminology and is used in textbooks such as [30].

Definition 2.2.3. Given two graphs $G = (V, E, s, t, l, w)$ and $G' = (V', E', s', t', l,' w')$ a graph morphism $f : G \to G'$ is a pair of maps

$$f = (f_V : V \to V',\ f_E : E \to E')$$

that preserve sources, targets and labels, that is

$$\begin{aligned} f_V \circ s &= s' \circ f_E \\ f_V \circ t &= t' \circ f_E \\ l' \circ f_V &= l \\ w' \circ f_E &= w. \end{aligned}$$

This can be shown diagrammatically. If the commutative diagram in Fig 2.1 commutes[4] for source and target mappings separately then $f = (f_V, f_E)$ is a graph morphism from G to G'.

$$\begin{tikzcd} & E \arrow[r, shift left, "s"] \arrow[r, shift right, "t"'] \arrow[dl, "w"'] \arrow[d, "f_E"'] & V \arrow[dr, "l"] \arrow[d, "f_V"] & \\ \Omega_E & E' \arrow[r, shift left, "s'"] \arrow[r, shift right, "t'"'] & V' & \Omega_V \end{tikzcd}$$

Fig 2.1: The commutative diagram shows the morphism $f = (f_V, f_E)$ of the labeled graphs $G = (V, E, s, t, l, w)$ and $G' = (V', E', s', t', l', w')$. The arrows represent function mappings from the domain (the source of the arrow) to the codomain (the target of the arrow).

A graph morphism $f = (f_V, f_E)$ is called injective (surjective) if f_V and f_E are injective (surjective) mappings. If a graph morphism is both injective and surjective then it is bijective. Bijective graph morphisms are called isomorphisms.

Lastly, the composition $f' \circ f : G \to H'$ of two graph morphisms $f = (f_V, f_E) : G \to G'$ and $f' = (f'_V, f'_E) : H \to H'$ is defined by $f' \circ f = (f'_V \circ f_V, f'_E \circ f_E)$.

Multiple approaches exist to motivating the construction of the double and single pushout. This following review is based on that of [31] and [32] with some simplifications.

2.2.2 The Double Pushout Approach

The double pushout approach requires that the subgraph matched in G be isomorphic to the left-hand side graph L. More specifically, there must exist an isomorphic matching morphism

[4] A commutative diagram is said to *commute* if all directed paths in the diagram with the same start and end point give the same result.

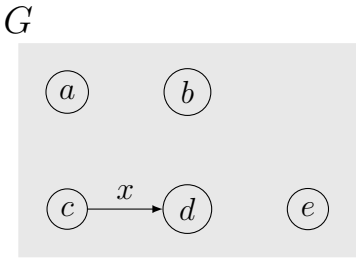


FIG 2.2: The graph G.

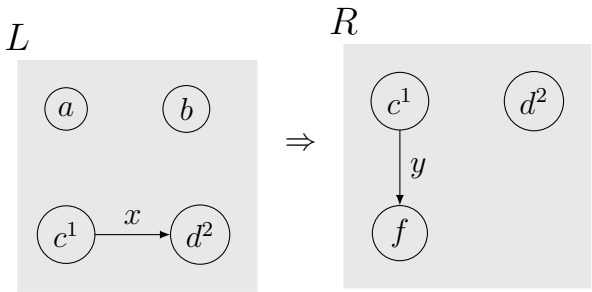


FIG 2.3: The graph production rule p_1.

$m' : L \to G'$ between the left-hand side graph L and a subgraph G' of the host graph G.

Furthermore, a production rule under the double pushout approach can only be applied if the *dangling condition* is met. Informally, the dangling condition states that edges in the host graph that do not appear in the left-hand side graph L cannot be deleted as result of applying the production rule. Such edges are called *dangling edges*. Dangling edges can arise when a node is present in the left-hand side graph L but not in the right-hand side graph R, which specifies that this node is to be deleted. However, if this node has an incident edge that is not in the left-hand side graph L then this edge is a dangling edge and, according to the rule, cannot be deleted. This is because at least one of the edge's endpoints, but not the edge itself, would be deleted if the production rule is applied. Therefore the edge would have one or zero endpoints (hence the name *dangling*) which is not valid. Therefore the production rule cannot be applied.

The labelled graph shown in FIG 2.2 contains five nodes and an edge. We mark nodes in host graphs with their label. Furthermore, using the notation for production rules that a node marked as x^y is a node with label x and y shows the injective mapping between the left- and right-hand side graphs, we can define graphically a production rule p_1 as shown in FIG 2.3.

There exists an isomorphic matching morphism between L and a subgraph of G. In fact, since labels do not have to match, there exists multiple subgraph isomorphisms between L and G. The node labelled a in L could map to any one of the nodes labelled a, b or e in G, and similarly

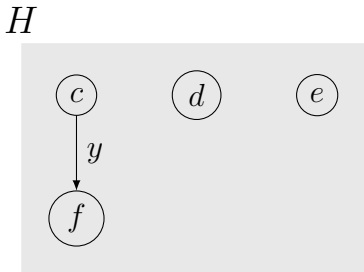


FIG 2.4: The graph H from the direct derivation $G \overset{p_1}{\Rightarrow} H$.

for the node labelled b in L. In this case, for convenience we concentrate on the trivial mapping where a maps to a and b maps to b. The fact that multiple matches may exist is the cause of the nondeterminism in graph rewriting, which is reviewed in SECTION 2.2.5. Note that the dangling condition is satisfied because there are no dangling edges. Applying p_1 to G is the direct derivation $G \overset{p_1}{\Rightarrow} H$ and H is shown in FIG 2.4.

The production rule p_1, however, under the double pushout approach, cannot be applied to either of the graphs G' and G'' shown in FIG 2.5. In G', the edge labelled s is a dangling edge because it does not appear in L but one of its endpoints (the node labelled b) would be deleted under p_1. For G'', there does exist a matching morphism between L and a subgraph of G'' but it is not isomorphic because the nodes labelled a and b in L both map to the node labeled i in G''. This restriction motivates the single pushout approach.

2.2.3 The Single Pushout Approach

The single pushout can be seen as a relaxation of the double pushout. In particular, if G is the host graph and a production rule has a left-hand side graph L, then the single pushout approach requires

- only a homomorphism, rather than an isomorphism, between the left-hand side graph L and a subgraph of the host graph G. A homomorphism allows multiple nodes in the left-hand side graph L to be mapped to the same node in the host graph G. Furthermore,
- the dangling condition need not hold. Conflicts between preservation and deletion are decided in favour of deletion.

As a result of these relaxations, the production rule p_1 in FIG 2.3 *can* be applied, under the single pushout, to the to the graphs G' and G'' in FIG 2.5. The resulting graph, in both cases, is H' in shown FIG 2.6

(a) The graph G' which has a dangling edge labelled s if the production rule p_1 is applied to it.

(b) The graph G'' which has a non-isomorphic matching morphism to the left-hand side graph of the production rule p_1.

FIG 2.5: The production rule p_1 cannot be applied to G' or G''.

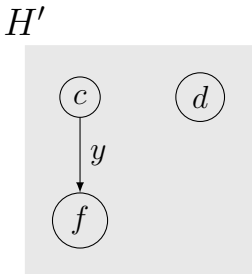


FIG 2.6: The graph H' from $G' \overset{p_1}{\Rightarrow} H'$ and $G'' \overset{p_1}{\Rightarrow} H'$ under the single pushout approach.

2.2.4 Negative Application Conditions

Within graph rewriting systems, a common extension to the rewrite semantics, whether that is the double or single pushout approach, is the inclusion of negative application conditions.

Negative application conditions allow matching on the absence of a subgraph within a host graph. Production rules with negative applications can be seen as having two left-hand side graphs. A positive graph L^+ that must match within the host graph and a negative graph L^- that must *not* match. This makes graph rewriting, as model of computation, more expressive.

The production rule p_2 shown in FIG 2.7 uses negative application conditions to construct an edge from one node to another only if the target node (labelled b) of that edge has no outgoing edges. The negative graph L^- is used to ensure that b has no outgoing edges by containing an edge to another node c. This means that if L^- *does* match then b *does* have an outgoing node and therefore the production rule should not be applied.

2.2.5 Nondeterminism

Graph rewriting, as presented in this section, can be viewed as a nondeterministic operation since the graph resulting from the application of a production rule is, in general, not unique.

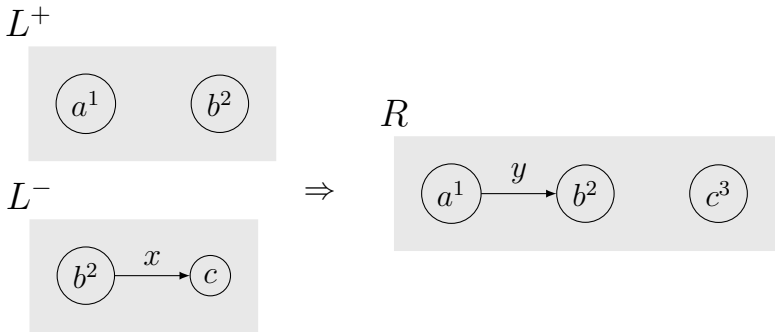


FIG 2.7: The graph production rule p_2 with negative application conditions. p_2 creates an edge from one node to another only if the target of that edge has no outgoing edges.

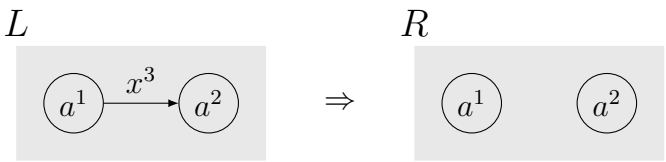


FIG 2.8: The graph production rule p_3 which matches on an edge and removes it.

This is because the subgraph matched within the host graph is, in general, not unique. This was noted in the example from SECTION 2.2.2

This is overcome in a theoretical setting by specifying the matching morphism $m' : L \to G'$ between the left-hand side L of the production rule and the subgraph G' of the host graph G. For a given production rule and host graph, the resulting graph may not be unique, however, by also specifying the matching morphism, the operation yields a unique result.

The production rule p_3 in FIG 2.8 can be viewed as matching on an edge and deleting it. If this was to be applied to the host graph G in FIG 2.9, there would be two distinct matches. One that maps the node a in L to i in G, and another that maps a in L to j in G. More precisely, there exists two unique matching morphisms $m'_1, m'_2 : L \to G$ and therefore there exists two direct derivations $G \xRightarrow{p_3, m'_1} H_1$ and $G \xRightarrow{p_3, m'_2} H_2$. Since the direct derivations specify the matching morphism, each direct derivation becomes a deterministic operation. Furthermore, the dangling condition is satisfied so these direct derivations are valid under the double

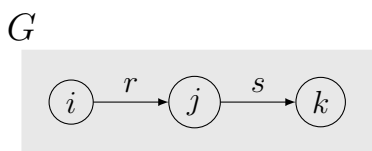


FIG 2.9: The graph G.

(a) The graph H_1 from $G \xRightarrow{p_3, m_1'} H_1$.

(b) The graph H_2 from $G \xRightarrow{p_3, m_2'} H_2$.

FIG 2.10: There exists two matching morphisms m_1' and m_2' when the production rule p_3 is applied to the graph G. This yields the two graphs H_1 and H_2.

pushout approach and are therefore valid under both the double pushout and single pushout approaches. For the sake of distinction, let m_1' be the morphisms that maps a in L to i in G and and m_2' the morphism that maps a to j. The resulting graphs H_1 and H_2 are shown in FIG 2.10.

Graph rewriting tools (which are reviewed in SECTION 2.3) generally deal with nondeterminism either by making a random choice whenever more than one valid choice is available (such as the existence of multiple matching morphisms) or by backtracking. Backtracking is a technique employed in many algorithms where, when a chosen candidate solution is determined to not actually be a solution, the algorithm *backtracks* and chooses another candidate solution [33]. This process is then repeated. This is encountered in graph rewriting when a particular production rule cannot be applied because of previous rule applications (a production rule may remove something from the host, that stops a future production rule being applied). At this point, the tool may backtrack to the previous rule applications and attempt different matching morphisms. Such tools are said to make choices *nondeterministically*.

2.2.6 Applying Production Rules In Sequence

Under the graph rewriting computational model, the application of a production rule is a single computational step. Production rules are therefore applied in sequence to solve a problem. For example, the production rule p_3 in FIG 2.8 can again be applied to the graphs H_1 and H_2 in FIG 2.10. In this case, there is a single match in each graph (since only one edge remains in each graph) and they yield the same result, that is

$$G \xRightarrow{p_3, m_1'} H_1 \xRightarrow{p_3, m_2'} H_3 = G \xRightarrow{p_3, m_2'} H_2 \xRightarrow{p_3, m_1'} H_3$$

and H_3 is shown in FIG 2.11. Furthermore, the matching morphism in the second direct derivation, in each case, is the first matching morphism in the other case.

2.2.7 Computational Completeness of Graph Rewriting

The Church-Turing thesis states that for any computable function on the natural numbers there exists a Turing machine that can compute that function [34]. This motivates the definition of Turing completeness. A programming language is said to be *Turing complete* if it can simulate a

FIG 2.11: The graph H_3 from $G \overset{p_3,m_1'}{\Longrightarrow} H_1 \overset{p_3,m_2'}{\Longrightarrow} H_3 = G \overset{p_3,m_2'}{\Longrightarrow} H_2 \overset{p_3,m_1'}{\Longrightarrow} H_3$.

Turing machine[5]. There exists a corresponding result for computation based on graph rewriting, for example using the double or single pushout approaches, which states that a graph rewriting system with constructs that provide

1. nondeterministic application of graph production rules,
2. sequential composition of graph production rules, and
3. the ability to repeatedly apply a given production-rule until there no longer exists a matching morphisms

is computationally complete [37].

In POINT 1, *nondeterministic application* means the ability to apply a production rule to a host graph without specifying the matching morphism, as explained in SECTION 2.2.5. Furthermore, the property in POINT 3 is called *as-long-as-possible iteration* and implies that at each stage the production rule is applied to the resulting graph of the previous application.

Computational completeness, here, is the property of being able to compute every possible function on labelled graphs as defined in SECTION 2.2.1.

2.3 Existing Graph Rewriting Tools

The previous section presented the double and single pushout approaches to graph rewriting and their common extensions for dealing with negative application conditions and nondeterminism. Reviewing graph rewriting in detail was motivated by the fact that it is a model of computation specifically for graphs. In light of this, this section covers existing graph rewriting tools and is summarised in TABLE 2.1.

2.3.1 PROGRES

PROGRES (for *PROgrammed GRaph REwriting System*) [23] is a hybrid visual/textual language that mixes graphical and textual elements. PROGRES was designed as a CASE (computer-

[5] A practical set of requirements for Turing completeness is presented by the Böhm-Jacopini theorem [35]. These requirements are: (1) sequences of code, (2) conditional execution (through an `if` {...} `else` {...} construct, for example), and (3) the ability to perform potentially unbounded iteration (through a `while (True)` {...} loop, for example) [36].

aided software engineering) tool [38] to model and prototype complex software projects while in their infancy.

PROGRES provides both graph rewriting semantics based on the *programmed graph replacement* approach [39], which is similar to the single pushout approach, with the addition of negative application conditions, but is defined algorithmically rather than algebraically.

Programs in PROGRES are production rules sequenced together using imperative language constructs for control flow. Graph schemas are used to specify what constitutes a valid graph and such a check is made automatically by the system after a production rule has been applied. Separate languages are used for graph manipulation and graph definition, graphs however can be imported and exported as UML.

The type system is similar to that of the Java programming language [40] in that it has an object model and allows class-like user defined types. Objects are placed on nodes and edges to embed data in the graph. The placing of objects on nodes or edges in a graph is called *attributing* and is an extension of labelling in labeled graphs (defined in Section 2.2.1). Such graphs are called *attributed graphs* and the objects can be accessed and changed in a production rule.

Backtracking is used to exploit nondeterminism, however, after a production rule has been applied, only one graph is kept. If a further production rule is to be applied and no match is found, the system backtracks. Backtracking is implicitly managed by the system and cannot be controlled programmatically.

2.3.2 AGG

The *AGG* (for *Attributed Graph Grammars*) system [24] is also a hybrid visual/textual CASE tool aimed at modelling software. AGG uses textual code to create objects and embed these on the graph, and visual elements to draw graphs and visualise production rules.

The single pushout approach is used for rewrite semantics with an extension that allows production rules to have multiple negative application conditions.

An AGG program is a *rule application strategy*. These come in two varieties. In *debug* mode, one production rule is applied exactly once on the host graph. If there are multiple matching morphisms, one is chosen at random. In *interpretation* mode, a universe of production rules is defined and the rules are then applied in sequence. The rule to be applied at each stage is chosen at random and this process is continued until no matches exist for all production rules

in the universe. Rules cannot be applied in a specified order. In either mode, nondeterminism is not handled using backtracking but instead a random choice is made whenever more than once choice is valid. This means that, in general, the resulting graph is not unique since the application of one rule may stop another rule being applied [41].

AGG, like PROGRES, uses attributed graphs but inherits its type system for objects directly from Java. As a result, Java objects are embedded in the graph. Any valid Java computation can be applied to these objects as part of the application of a production rule. Graphs can be imported or exported as XML.

2.3.3 FUJABA

FUJABA (for *From UML to Java And Back Again*) [42] is a visual programming CASE tool aimed at round-trip engineering [43]. Round-trip engineering is concerned with techniques to synchronise two aspects of a software project. In the case of FUJABA, the aim is maintain synchronisation between a UML model of a project with its Java source code (hence the name). It does this by modelling the project as a graph of interactions (as edges) between functions, classes, and objects (as nodes). The type system is therefore inherited directly from Java. FUJABA allows either the UML model or the Java source code to be changed. These changes are then highlighted and the project can be resynchronised by generating Java function bodies or UML based on their visual specification. Functionality can be added to the project by modelling this as production rule that transforms the graph. Once these have been added, the corresponding Java and UML is regenerated.

FUJABA uses a custom algorithmic rewrite semantics called the story-diagram language [44] designed to integrate graph rewriting with object-oriented languages. The story-diagram languages makes choices nondeterministically but backtracking is not used.

2.3.4 GrGen

GrGen (for *Graph Rewrite Generator*) [45] is visual programming tool for compiler construction.

GrGen's graph concept is *attributed typed multigraphs* which extends attributed graphs by allowing

- multiple inheritance for the objects embedded on nodes and edges, and
- multiple edges in the same direction and of the same type between two nodes.

The rewrite semantics are based on the single pushout approach but, by default, requiring an isomorphic (rather than homomorphic) matching. Negative application conditions are also sup-

ported.

GrGen is made up of four distinct languages [46]:

1. The *graph model language* is used to define object types to be used on nodes and edges. The type system and object model is similar to common object-oriented languages. The graph model language is also used to define *connection assertions* which are like the graph schemas in PROGRES.

2. The *rule and computations language* is used to construct graph production rules.

3. The *rule application control language* is used to sequence the application of production rules. This language also has features to loop over applying production rules, revert the graph having applied a production rule, or define *embedded sequences* which are a sequence of production rules encapsulated as a single rule.

4. The *shell language* is a GrGen REPL (read, evaluate, print loop) with extra functionality for importing, exporting, and visualising graphs.

To deal with nondeterminism, GrGen materialises only one graph at a time and backtracks if necessary. This cannot be controlled programatically.

2.3.5 GP 2

GP 2 (for *Graph Programs*) [47] is a graph programming language with the objective of being a practical graph transformation language while having formal semantics. It is intended to be used both for solving graph problems and formal reasoning. GP 2 is a visual tool.

Graphs in GP 2 are *partially labeled graphs*. This is a relaxation of labeled graphs that allows unlabelled nodes but still requires all edges to be labelled. The rewrite semantics are based on the double pushout approach with the ability to relabel nodes and edges. GP 2 doesn't have negative application conditions but does allow applying a rule conditionally based on

- whether the left-hand side of another rule can match the current graph,
- the value of a node or edge label,
- the existence of an edge, or
- the indegree or outdegree (the number of incoming and outgoing edges, respectively, at a node) of a node.

GP 2 programs are an ordered sequence of production rules. Further to this, it is possible to apply a production rule according to as-long-as-possible iteration (explained in Section 2.2.7).

A label in GP 2 is an atom, which is type union of integer and string, or lists of atoms. This is a relatively simple type system in comparison to the other tools reviewed in this section. In particular, GP 2 does not have an object model.

GP 2 can be used to generate all possible resulting graphs up to isomorphism (if multiple resulting graphs are isomorphic, only one is kept), fully exploiting the inherent nondeterminism of graph rewriting. This feature can also be switched off and thus only one result graph is generated. This feature, however, is not programatic. That is to say, the option to generate all possible graphs or one graph is not a construct within the GP 2 language, but rather part of the system, as a command line option.

2.3.6 Program State Within Graph Rewriting Tools

Program state in all the rewriting tools reviewed is maintained as a single graph, in so far as there are no variable bindings. Data can only be stored on the labels of nodes and edges. The graph is therefore the implicit input, state and implicit output of all programs. Furthermore, with the exception of the handling of nondeterminism, programs only maintain one graph. There are no constructs for binding graphs to variables and, as a result, no constructs for handling multiple graphs. This is a result of the these tools being inline with the theoretical framework of graph rewriting.

2.3.7 Other Tools That Provide Graph Rewriting

There exist a number of libraries that provide graph rewriting capabilities, through an API (application programming interface), rather than as an underlying model of computation. Two such libraries are:

- *ReGraph* [48], a Python library for generic graph rewriting that implements the sesqui pushout approach[6], and
- *GGL* (for the *Graph Grammar Library*) [50], a C++ library for modelling chemical reactions as graph transformations, using the double pushout approach.

[6]The sesqui pushout approach is similar to the double and single pushout approaches but is more conservative on deletion than the single pushout and doesn't require the dangling condition to hold [49].

Tool	Mode	Intended Domain	Graph Concept	Rewrite Semantics	Type System	Nondeterminism
PROGRES	hybrid	software modelling	attributed graphs	programmed + graph schema	object-oriented	implicit backtracking
AGG	hybrid	software modelling	attributed graphs	SPO + multiple NACs	Java	random choice
FUJABA	hybrid	round-trip engineering	UML-like	story-diagram	Java	nondeterministic choice
GrGren	hybrid	compiler construction	attributed typed multigraphs	SPO + NAC + graph schema	object-oriented with multiple inheritance	implicit backtracking
GP 2	visual	graph problems, formal reasoning	partially labelled graphs	DPO + conditional application	lists of integers and strings	implicit backtracking
ReGraph	Python	general purpose	attributed graphs using Python dictionaries	SqPO	Python	all graphs generated
GGL	C++	chemical reactions	undirected labelled graphs with string labels	DPO + NAC	C++ but string labels	all graphs generated

Table 2.1: An overview of the existing graph rewriting tools. The abbreviations SPO (single pushout), DPO (double pushout), NAC (negative application conditions), and SqPO (sesqui pushout) are used.

2.4 Summary

The review of graph rewriting showed that it is a model of computation aimed specifically at performing operations on graphs. On the other hand, the review of existing graph rewriting tools highlighted the lack of general-purpose graph programming tools that utilise graph rewriting. Only ReGraph and GP 2 are intended for solving domain-agnostic graph problems. The other tools are intended for use in a particular domains, such as synchronising software projects as a graph (FUJABA's intended use).

ReGraph is library that must be used within Python and so the syntax and semantics of Python, unrelated to graphs, distract from the problem.

GP 2 has no variable bindings and so program state is maintained as a single graph. Furthermore, it has a very primitive type system that allows only lists of integers and string.

In summary, while there exist many graph programming tools, they are either

- focussed mainly on performance (graph DSLs),
- focussed mainly on querying graphs (graph databases),
- for solving domain-specific graph problems (most of the rewriting tools),
- constrained by the syntax and semantics of their host languages (graph libraries), or
- have primitive type systems not suited to modelling data (GP 2).

Chapter 3

Requirements Capture

The overall objective for this project is to design an *expressive* and *productive* tool for graph programming. Expressivity, in this case, is seen as the ease with which important operations can be expressed syntactically and semantically. Productivity, with respect to graph programming, can be seen as how readily the language allows a graph problem to be solved.

This chapter aims to make this objective more precise and measurable in order to effectively evaluate the success of the project. The requirements have been split up into four distinct groups, system-level, type system and program state, graph-specific, and reference implementation requirements. While there is of course an interplay, for example, between the type system of a tool and the expressivity of its constructs, separating these is beneficial both for the design of the tool and its evaluation. All requirements have been motivated by an example usage in graph programming and a principled argument.

To allow for more precise requirements to be set, the decision on the format of the tool (that is, whether it will be a graph database, library, embedded DSL, rewriting tool, or a standalone DSL) is discussed and justified in this chapter, before the requirements are set. This reflects the true development of the project, in which this decision was made early on. In retrospect, making this decision sooner rather than later was advantageous to the project because having the precise requirements aided in making later design decision in a principled way.

3.1 Tool Format

In order to decide on a tool format, a comparison is drawn between the formats introduced in Section 2.1. The comparison is guided by the five desirable properties shown in Table 3.1.

Property	Description	Reason
Querying	Querying the graph for information.	It should be possible to get data out of the graph.
Transformation	Transforming the graph in a programmatic way.	It should be possible to change the data in the graph.
Non-graph program state	Maintaining state through more than just a single graph.	Having only a single graph as the program state is restrictive.
Graph-specific syntax	Syntax specifically designed for graph programming.	Expressing graph programming concepts should be syntactically straightforward.
Graph-specific semantics	Semantics specifically designed for graph programming.	Expressing graph programming concepts should be semantically straightforward.

Table 3.1: Desirable properties of graph programming tools.

Graph databases provide querying capabilities through OLAP (online analytical processing) and transformation through OLTP (online transaction processing). Graph databases store only the graph and thus have no variable bindings. Program state is therefore maintained as a single graph. The syntax and semantics of database graph query, graph definition and graph transaction languages are however designed specifically for graph programming.

Graph libraries and embedded DSLs typically provide, through an API, access to the underlying graph structure. This means querying and transformation can be done either directly (if the API offers it), or can be implemented in the host language using other API calls. Host languages tend to be general-purpose programming languages and so use variable bindings for program state. On the other hand, libraries and embedded DSLs cannot provide graph specific syntax and semantics, since they must use the syntax and semantics of the host language.

Graph DSLs can provide querying and transformation and can do so through syntax and semantics specifically designed for graph programming. Furthermore, the program state can be maintained either as a single graph or variable bindings. DSLs are very flexible since they have no a priori restrictions.

Graph rewriting tools provide transformation through rewriting. Querying, on the other hand, is generally not supported because this is not part of the graph rewriting formalism (presented in Section 2.2). Rewriting tools benefit from having graph-specific syntax and semantics, which can also be visual or textual.

The summary of this comparison can be seen in Table 3.2.

Tool Format	Querying	Transformation	Non-Graph Program State	Graph-Specific Syntax	Graph-Specific Semantics
Graph database	✓	✓		✓	✓
Graph library or EDSL	✓	✓	✓		
Graph DSL	✓	✓	✓	✓	✓
Graph rewriting tool		✓		✓	✓

Table 3.2: A comparison of the desirable properties of graph programming tool formats. The abbreviation EDSL (embedded domain-specific language is used).

While graph databases provide almost all of the desirable features, it should be noted that querying and transformation is usually provided through two distinct OLAP and OLTP languages, and a particular graph database is usually aimed at either OLAP or OLTP [51, 52]. Furthermore, as discussed in Section 2.1, databases are designed to meet other requirements regarding concurrency and ACID properties which do not, with regards to graph programming, contribute to the productivity or expressivity of the tool.

Since graph libraries and embedded DSLs are used within host languages, this burdens the user with having to concisely handle other language constructs, such as low-level data structures, unrelated to graph programming.

As seen in Table 3.2, graph DSLs can provide all the desirable properties. Existing graph DSLs, however, focus on graph programming from a high-performance computing perspective rather than expressivity and productivity. The three DSLs mentioned thus far (GraphIt, Green-Marl, and Ligra) all prioritise performance.

There are no (to the knowledge of the author) graph DSLs that prioritise productivity and

expressivity over performance. Furthermore, there are no graph DSLs (to the knowledge of the author) that utilise a graph rewriting model of computation while offering the familiarity of a variable bindings approach to maintaining program state. As such, designing such a tool is a potentially uncharted area but, based on this discussion and the background review in Chapter 2, should lead to an expressive and productive tool for graph programming.

As a result of this comparison, the decision to focus on designing a graph DSL was made. The remaining parts of this chapter set requirements in light of this decision.

3.2 System-Level Requirements

These requirements concern the tool seen as a system. In particular these address how the tool can handle input and output, and how the tool is delivered. The requirements are discussed in the text and summarised in Table 3.3.

Firstly, it should be possible to input and output graphs to and from the file system. This would mean graphs can be defined once, saved in a file, and be used within multiple programs without having to be redefined each time. This would also allow graphs to be exported from one program and imported into another. This allows the separation of code and data within a project as well as the reuse of data. These are both good practices in software engineering. Furthermore, this means the tool can interact with other systems or be part of a pipeline of tools composed together, which is a notable DSL design pattern called *pipelining* [53].

Similarly, it should be possible to import functionality from other files. This means common graph operations can be implemented once and imported into all programs that need to use them, creating a library-like system. This increase code reusability and modularity [54, 55].

Finally, the tool should be a single language, rather than distinct languages for graph definition and graph manipulation. GrGen, presented in Section 2.3.4, is four distinct languages. This makes the system more complicated because the semantics of interactions between constructs (potentially across the multiple languages) is less clear. Furthermore, defining a graph as a combination of two other graphs can be seen, *conceptually*, as both graph definition and graph manipulation. Therefore, construction and manipulation should not be separated *semantically* as two distinct languages. The semantics of the language should reflect the key concepts within the problem domain. Moreover, having one unified language allows more, and clearer, composition between language constructs. In this case, constructs related to graph definition and graph manipulation.

Requirement	Usage Motivation	Principled Reason
Ability to input and output graphs	Reusing common graphs	Separation of code and data and allows tool pipelining
Ability to import functionality	Library-like modules of common operations	Increase code reusability and functionality
One language	Defining a graph as a combination of two other graphs	More composition between language constructs.

Table 3.3: System-level requirements.

Requirement	Usage Motivation	Principled Reason
Mutable variable bindings	Binding data and graphs to identifiers allowing multiple graphs to exist	Familiarity and productivity
User-defined object types	Embedding data on nodes and edges	Encapsulation

Table 3.4: The type system and program state requirements.

3.3 Type System and Program State Requirements

Almost all of the rewriting and graph database tools seen in Section 2.3 maintain program state as a single graph. This is uncommon outside of graph databases and rewriting tools. This is also restrictive since variables cannot be used and programs involve only a single graph. The tool should therefore maintain program state using variable bindings. That is, maintain a mapping of variable identifiers (as well as other constructs, such as functions) to their value. The bindings should also be mutable to allow constructing graphs incrementally. Maintaining program state as variable bindings is done in the major programming paradigms (imperative, object-oriented, functional, and logic [26]) and the familiarity of this will aid productivity [54].

Graphs provide an advantage in the way they represent data. This motivates having user-defined object types, allowing the user to decide how best to embed the data in the graph. This also allows appropriate encapsulation of the data represented on the graph.

The requirements for this section are summarised in Table 3.4.

3.4 Graph-Specific Requirements

This section concerns requirements directly related to graph programming. The requirements for this section are summarised in Table 3.5.

Firstly, graphs will be defined frequently and so the appropriate constructs to do this quickly and easily increase the productivity of the tool. The syntax for defining graphs should therefore be declarative [56]. Declarative constructs raise the level of abstraction to that of the problem domain, graphs. Moreover, a DSL should provide syntactic constructs for the major concepts in the intended domain [57].

While graph construction is important, to solve problems graphs need to be manipulated. The graph rewriting model of computation (discussed in Section 2.2) is specifically designed for graphs and so also brings the level of abstraction closer to the problem domain. Furthermore, many well known graph problems have straight forward solutions under the graph rewriting model, shown in [58].

None of the rewriting languages discussed in Section 2.3 allow the user to programatically control how nondeterminism (covered in Section 2.2.5) is handled[1]. Whether a match is picked at random (as done by AGG) or whether all matches are used (as done by GP 2) should be the choice of user. In particular, sometimes only a random choice is needed [59], avoiding the overhead of evaluating all matches or backtracking. Since backtracking is an part of the model of computation, there should be constructs for controlling it.

As a result of maintaining problem state using variable bindings, multiple graphs can simultaneously exist at the same time. There should therefore be constructs that allow combing these. This makes constructing larger graphs easier, as it can be done incrementally.

The notion of computational completeness for graph rewriting, covered in Section 2.2.7, formalises the ability to perform any operation on a graph. Since the aim of the tool is productivity when solving graph problems, it should be computationally complete in this sense. More precisely, the language should contain constructs for as-long-as-possible iteration and applying production rules in sequence. Completeness also requires nondeterministic application of production rules, which has been covered by a previous requirement.

[1] To the knowledge of the author, no rewriting tools have this feature.

Requirement	Usage Motivation	Principled Reason
Declarative syntax for graphs	Graphs will be defined frequently	Raises the level of abstraction
Graph rewriting model of computation	Can be used to solve graph problems	Aligns level of abstraction with problem domain
Programmatic control of nondeterminism	Backtracking or evaluating all matches is not always needed	Part of problem domain, so should be captured by the tool
Constructs for combining multiple graphs	Build up graphs incrementally	Combing graphs can is used in solving graph problems [60]
Computational completeness	Required to be able to solve all graph problems	As-long-as-possible iteration and sequencing are part of the problem domain, so should be captured by tool

Table 3.5: Graph-specific requirements

3.5 Reference Implementation Requirements

Alongside the design of the language, this project seeks to deliver a reference implementation, that is, a language processor for the tool. As mentioned in [61] and used as motivation in [58],

The most interesting question about a program is "What does it do?".

Further to this, having an implementation is beneficial in gaining a practical working understanding of the language [61]. It can also find nuanced ambiguities in the specification or provoke reflection.

The requirements for the implementation are that it should provide a good portion of the functionality designed for the language and should also be well-tested. Aside from this, the code should be well organised and in a good state to go forward as an open source project.

3.6 Summary

This chapter first discussed tool formats, and justified the choice of a graph DSL. This was done to allow more precise requirements in the sections that followed. These requirements will be revisited in the evaluation to assess the success of the project.

Chapter 4

Analysis and Design

The chapter introduces Bilbo, a self-contained, domain-specific language for graph programming. Bilbo is a fully textual language that utilises graph rewriting as its underlying computational model for graph operations. As well as this, it offers a strong dynamic type system, with user-defined record-like object types and heterogeneously-typed graphs. Bilbo has a host of operators for sequencing transforms (based on production rules) and novel constructs for controlling nondeterminism.

Bilbo has reference interpreter and REPL implemented in F#.

In particular, this chapter justifies four major design decisions in the development of Bilbo. Firstly, the textual nature of the language, despite graphs being very naturally represented visually. Secondly, the type system and the tradeoffs made as a result. Next, the choice to implement an interpreter rather a compiler is discussed. Finally, the choice of implementation language, F#, is justified.

The important design choice to make Bilbo a graph DSL has already been discussed in Section 3.1. As mentioned previously, this decision was made early on, to allow for precise requirements to be set. The following choices were made in light of that choice.

4.1 Tool Mode

The *mode* of a tool is the way in which it is experienced by users, that is, visually or textually. Visual programming tools allow users to *create* programs visually while textual tools require that the programs be written.

Graphs are naturally represented in a visual manner. In graph programming, this provides obvious benefits for visualising input and output graphs, or graph rewriting as seen in the diagrams from Section 2.2. In particular, it helps in visualising the relationships between nodes

by showing the edges. Textual tools can, however, capitalise on this benefit. It is common for textual languages to have software that allows visualising the components within a program. For example, it is possible to generate an RTL netlist diagram from textual Verilog HDL code. Beyond visualising input and output graphs, the benefit from a visual tool in graph programming is limited. Programatically describing imperative operations (rather than data-flow or transformations) is difficult to do visually. Consider the construction of a record-like object, this is done easily in textual languages but there is no obvious way to do this visually. Such an operation is common in graph programming because structured data is embedded on graphs.

While visual tools offer an advantage in visualising graphs, programs will be made up of operations rather than simply graphs to be visualised. There are a number of benefits in having a textual tool.

Textual tools can facilitate modularity through constructs such as functions, classes, and namespaces. This increases productivity since related functionality can be abstracted away or encapsulated together. These constructs are harder to transpose to a visual programming environment.

Furthermore, a number of language constructs (such as those mentioned above) are found in many general-purpose programming languages (sometimes under different names, for example C++ *namespaces* are analogous to Python *modules*) and, while the precise semantics may differ from between languages, the behaviour of these constructs is conceptually similar. This provides a *conceptual familiarity*, allowing programmers to be productive independently of the language being used, as long it provides these similar constructs. This familiarity, and so the productivity gained from it, would be lost in a visual programming tool.

Textual programs also provide *structural familiarity*. This is the motivation for style guides within code bases: if the code looks similar, it is easier to understand and extend. While such guides are language-specific, there are common structural components found in many languages that relate to the same semantics. For example, indentation is often used to resemble scope. To wit, some languages (such as Python) enforce the use of indentation when a new scope is entered, and in languages that don't (such as C++), indentation will be idiomatic and a de facto practice. This structural familiarity, like conceptual familiarity, provides language-independent productivity for programmers. Due to minimal adoption of visual programming languages, they do not benefit from such a familiarity.

As program size increases, the scalability of visual programs also comes into question. The *Deutsch limit* [62] is a criticism of visual programming tools, it suggests one cannot productively develop with more than fifty visual primitives on the screen at the same time. Furthermore, as the amount of coupling between program elements increases, it gets harder to represent these

relationships effectively in a visual way, due to the complexity of the interactions between components. In textual languages this is handled using modularity. As such, any benefits of being able to visualise the program comes with diminishing returns.

In summary, a visual programming tool would allow input and output graphs to be readily visualised, but on the other hand

- the difficulty in representing operations visually,
- the productivity gained from modularity,
- the productivity gained from conceptual and structural familiarity, as well as
- the option of having a separate graph visualisation tool

suggest that Bilbo would be more productive as a textual tool.

4.2 Type System

Graphs are effective at representing relationships between data points. In order to fully leverage this, a graph programming language needs a type system that allows embedding data on graphs. This section reviews the type systems considered for Bilbo and justifies the final choice.

4.2.1 A Strong Type System

A *strong* type system can be regarded as a system in which types will not *implicitly*, or unexpectedly, change as a result of an operation [63]. The implicit change in type is called *type coercion* and strongly typed languages, in general, do not allow it. For example, a *weak* type system may coerce, in the expression `"10" + 5`, either the string to an integer, or vice versa, to allow the addition to occur. Due to the ambiguity of weak type systems, even in this simple example, it was decided that Bilbo would have a strong type system. With regards to productivity, this frees users from having to worry about implicit changes in type or remember type coercion rules.

4.2.2 A Dynamic Type System with Heterogenous Graphs

A language has a *static* type system if type checking takes place before run time. Languages with *dynamic* type systems will either type check an operation just before executing it or not check at all, attempt the operation, and report an error if the operation fails.

Static typing offers a number of benefits. Firstly, the safety that comes from type checking before run time. This means that type errors are detected earlier, in comparison to using a dynamically typed language, where a type error will only be detected upon execution of the erroneous code. Other benefits include the possibility of better code completion in an IDE

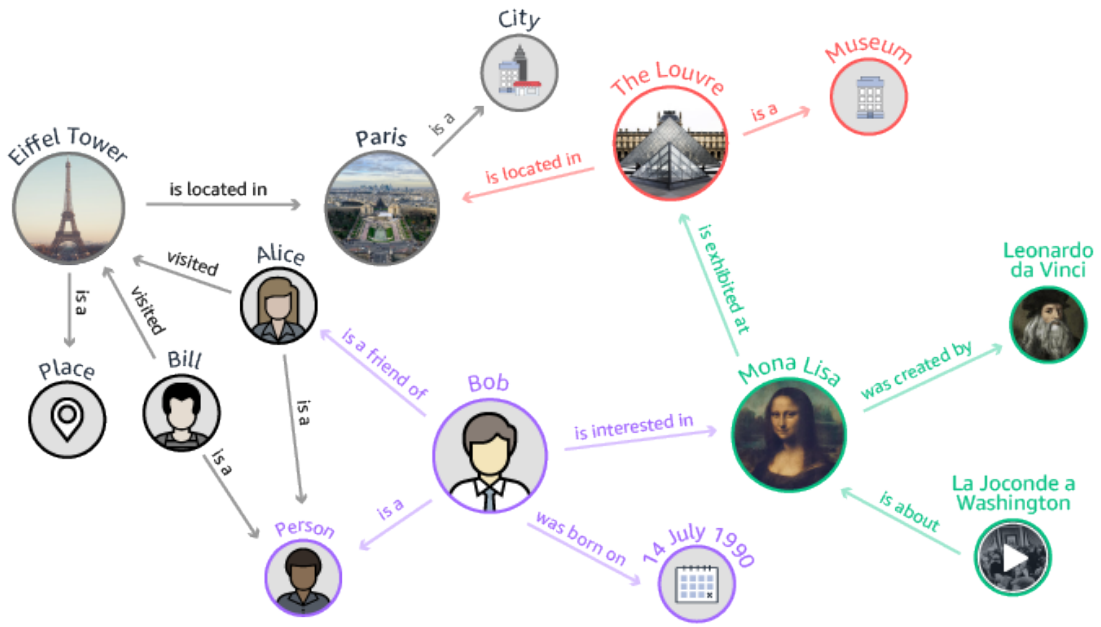


FIG 4.1: A graph containing nodes that represent people, locations, buildings, paintings, and types. Edges represent relationships between these nodes such as `is a friend of`, `is located in`, and `is a` [65].

(integrated development environment) because the IDE can infer which types are valid in the current context, and opportunities for compiler optimisations [64].

The benefits of dynamic typing include the decreased development time of the language processor since a static type checker[1] need not be implemented, and easier rapid prototyping (while the code base is very small) since less considerations have to be made with regards to type.

Static typing can also produce self-documenting code if type annotations are required, however these are not necessary if the language has type inference (F# and Haskell do this by default, C++ can do it in some cases using `auto`). On the other hand, dynamically typed languages can be seen as less verbose than statically typed languages if type annotations are required (in the statically typed language) and a type inference system requires development time.

Bilbo would benefit from the safety of being statically typed because of the added type safety. A key consideration, in the case of graph programming, is the type of a graph. In the case where all the nodes are of type `'TN` and all the edges are of type `'TE` then, using a template-like syntax, such a graph could be statically typed as

[1] A type check may still take place just before a particular expression or statement is executed. However, such a type check is straightforward to implement since it need not resolve types more than one step in advance.

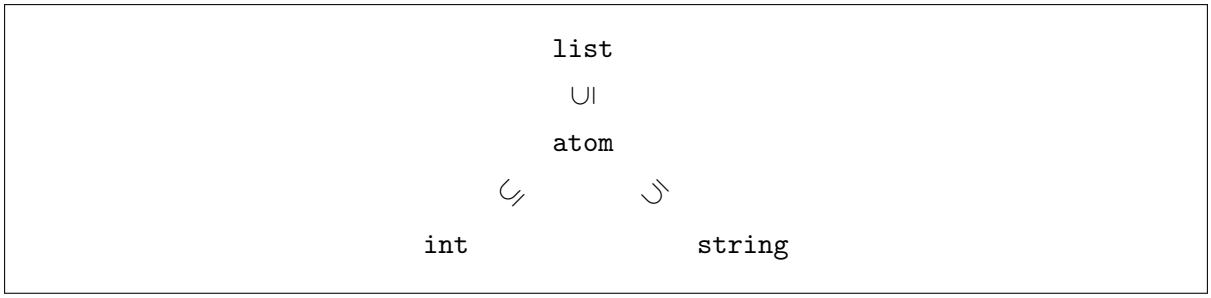


FIG 4.2: The type hierarchy for GP 2 [47]. `atom` is the union of `int` and `string`, and a `list` is zero or more `atom`s.

```
Graph<'TN,'TE>
```

where the type `Graph<'T1,'T2>` represents a graph that contains nodes of `'T1` and edges of type `'T2`. However, imposing this restriction (called *homogeneity*) would seriously harm the productivity of the tool. Graphs are effective at representing information because they show the relationships between heterogenous data, that is, data of different types. Consider the graph in FIG 4.1. This graph contains nodes that represent people (for example `Alice`), locations (`Paris`), buildings (`Eiffel Tower`), paintings (`Mona Lisa`) and even types (`Person`). The edges also present relationships of multiple types. This is more difficult to statically type. This can be done by unifying the types polymorphically, for example, using an F#-like syntax where `|` represents type union, as

```
type NodeType = Person | Location | Building | Painting | TypeAsNode
```

and edges similarly. This solution, however, is not scalable. Consider if a further node of type `Animal` was to be added. Either the `NodeType` hierarchy would have to be rewritten, or it could be unified further with `Animal` as

```
type NodeType2 = NodeType | Animal
```

which would require replacing all instances of the type annotation. This shows that static typing is not suitable for heterogenous graphs unless all types are known in advance. If all possible node and edge types were known in advance, then type hierarchies could be created once and used by all graphs. GP 2, for example, does this using the type system shown in FIG 4.2. All data on nodes and edges is of a type `list`. While this allows graphs to be statically typed, it only allows data on nodes and edges to be lists of integers and string. This would be a harmful restriction on expressivity and productivity since it doesn't allow data of a user-defined type to be in the graph and therefore can lead to badly encapsulated data [66].

The problem is overcome in Bilbo by having a dynamic type system and not restricting a graph to have a particular type for nodes and a particular type for edges. Such a data structure is said to be *heterogeneously typed* [67] (for example, Python lists are heterogeneously typed).

In [64] the authors suggest: *static typing where possible, dynamic typing when needed.* In this case static typing would impinge on other important language features, affecting productivity and expressivity, in a number of ways. Having a static type system would mean, either:

- *Homogeneously typed graphs.* All data on nodes and edges, separately, would have to be of the same type. This would mean a node not of the given node type cannot be added to the graph. Similarly with edges.
- *Single type hierarchy.* All data types would be in a single hierarchy. This means data of a user-defined type cannot be placed on the graph.

In conclusion, static typing is suited to cases where the types of all the data to be stored within the graph are known a priori. To maintain productivity and expressivity, this restriction cannot imposed and so Bilbo has a dynamic type system with heterogeneously typed graphs.

4.3 The Bilbo Reference Implementation

This section introduces the Bilbo reference implementation, which was implemented alongside the design of the language. The two major considerations in developing the reference implementation were whether it should be a compiler or an interpreter, and what language should be used. To allow for productive development and a useful end product, both decisions must be considered carefully.

4.3.1 Reference Implementation Format

A language processor is a piece of software that operates on code written in a defined source language, in this case Bilbo. The two prominent types of language processors, from [68], are

1. *compilers*, which translate the code from the source language to the target language, often at a lower level of abstraction, and
2. *interpreters*, which directly evaluate the code in the source language.

Compilers, in general, offer faster[2] execution times than interpreters. On the other hand, interpreters often take less time to develop because the developer need not consider the semantics of a target language.

Understanding a tool's computational model, especially if it is novel or unconventional (as is the case for Bilbo, with graph rewriting), can be done effectively through an interactive programming environment, such as a REPL [69]. REPLs are readily created from interpreters because the interpreter need only be modified to accept and evaluate lines one at a time. Creating a REPL from a compiler is more work since it either requires

[2] While the compiler does not execute the source code itself, once the source code has been compiled to the target language it will generally run faster than the interpreter. This doesn't take into account the compilation time.

- the implementation of virtual machine upon which the target language can be executed one line at time, or
- accumulating a program one line at a time and recompiling[3] and running that program every time a line is added.

Based on the decreased development time, as well as the option of easily creating a REPL, which is motivated in the case of Bilbo due to its unconventional computational model, Bilbo's reference implementation is an interpreter. While the decrease in execution time offered by a compiler is important, it was deemed less beneficial to the project than being able to deliver a tested interpreter and REPL on time.

4.3.2 Implementation Langauge

The choice of which language to use was based on a number of things. In particular, the implementation language should be cross-platform, have a type system that allows handling of exceptional behaviour (such as erroneous user input) and have libraries for parsing and testing to allow productive development.

In light of these considerations, F# was a good choice.

F# has static polymorphic typing with discriminated unions. This allows type hierarchies to be easily created, safely manipulated and, using pattern matching, be easily unpacked. The Bilbo AST (abstract syntax tree) uses recursive discriminated unions to represent a program. This makes traversing the AST simple.

The interpreter uses recursion extensively to process the AST. For example, given the expression `1+2*(3+4)`, then the expression evaluator will recurse when this is broken down into `1` and `2*(3+4)` and again for `2` and `3+4`. Recursion requires a call stack of a size that varies linearly with the recursion depth, which risks a stack overflow. F# has optimised tail recursion [70, 71] meaning that the call stack size is not increased whenever the recursive call is part of a returned value.

A language reference should have accurate and useful error messages. Especially in the case of Bilbo, where the language is domain-specific and so common program bugs may be unfamiliar. F# allows monad types to be created and used easily. This provides a typed way of handling exceptional behaviour. Monads are used extensively in the Bilbo interpreter to seamlessly handle valid results as well error messages.

The .NET Core platform provides complete interoperability between Windows, MacOS, and Linux. This means Bilbo's interpreter and REPL can be readily used on any of these operating

[3] Such a REPL is called *replay-based* [69].

systems.

With regards to testing, *Expecto* [72] and *FsCheck* [73] are two actively maintained F# testing libraries that can be used together to randomly generate tests based on the required type. Both of these libraries were used to test Bilbo's interpreter.

Finally, *FParsec* [74] is A well-tested parser combinator library in which parsers are first-class values (meaning they can be assigned to identifiers as well as passed to and returned from a function). This means individual parsers can be created independently and composed to produce more complex parsers. Bilbo uses the same syntax for constructing graphs as well finding subgraphs in the host. In this case, using FParsec meant that one parser could be used in both instances.

These properties of F# show that it was justified choice for the interpreter.

4.4 Summary

This chapter introduced Bilbo from a high-level perspective, concentrating on the design choices regarding the mode of the tool (textual), its type system and the reference implementation. The choice of a textual language was made on the basis of the productivity gained from a familiar setting as well the difficulty of representing operations graphically. The dynamic type system was chosen to allow embedding heterogeneous data on a graph. It was shown that this cannot be done easily with a static type system. Finally, the choice of developing an interpreter was justified based on the decreased development time as well as the option of easily extending it to a REPL (which was indeed done) and F# was justified based on its type system, interoperability and exiting libraries.

The following chapter introduces Bilbo in terms of its syntax and semantics. The chapter after that will concentrate on the implementation of the interpreter and the testing.

Chapter 5

The Design of Bilbo

This chapter describes and motivates the design of Bilbo's syntax and semantics.

The first section provides an introduction to the basic features of the language, such as variables, types and objects. The following two sections, after that, focus on graph construction and graph manipulations.

The section on graph construction introduces *paths*, Bilbo's syntax for constructing graphs. The notions of graph addition and subtraction also show how graphs can be built up incrementally, or precise parts of graphs can be easily removed. Lastly in graph construction, the notions of *path comprehensions* and *nested paths* show certain regularities in graphs can be exploited to make graph construction more concise.

The section on graph manipulation introduces *transforms*, which allow graph rewriting based on a variant of the single pushout approach, which was discussed in Section 2.2.3. Transforms naturally lead to discussing nondeterminism, which are handled using *collections*. The semantics for sequencing transforms into *pipelines* and how this affects the nondeterminism is also discussed. Finally in the graph manipulation section, Bilbo's generalisation of negative application conditions (discussed in Section 2.2.4) are presented, which allow constraining matches using well-known logical operators.

The penultimate section in the chapter covers *pipeline modifiers* which allow controlling nondeterminism when it is not required, applying a transform as-long-as-possible, and how to apply a transform not knowing in advance whether it will match. This section also covers *reductions* which are a completely novel idea that allow interaction with the collection of graphs maintained due to the nondeterminism.

The final brief section in this chapter introduces two features of Bilbo that are noteworthy, but did not fit under the other sections.

5.1 Bilbo Basics

This section introduces the very basics of Bilbo. Some of the ideas presented will be familiar to the reader, and so this section is concise.

5.1.1 Variables and Primitive Types

Bilbo maintains program state using variable bindings, similar to other general-purpose programming languages. Variables are assigned using `=` and no type annotation is needed. Variables are mutable. Mutability was chosen as a language feature because it allows values to be built up incrementally without having to introduce a new identifier for each change. This is particularly useful for constructing graphs incrementally and the choice of mutability is discussed again in Section 5.2.2.

The primitive value types in Bilbo are `int`, `float`, `bool` and `str`. Boolean values are capitalised, `True` and `False`, and strings use double quotes, for example `"Hello, world!"`. The standard operators for addition (`+`), subtraction (`-`), multiplication (`*`), division (`/`) and modulo (`%`) can be used between the numerical types, as can the comparison operators. Exponentiation is done with `^`. Strings can be added using `+` and booleans can be manipulated using `and`, `or` and `xor`. Values can be cast between the primitive types by enclosing the literal or variable in brackets, with the desired type before the brackets. Casting to a `bool` will yield `True` unless the value cast is `0`, `0.0`, `""` or `False`. Assignments and type casts can be seen in Listing 5.1.

The smallest program unit in Bilbo is a statement. That is to say, expressions are not valid alone, they must be bound to a variable. There are no downsides to this feature because expressions outside of assignments would be wasted computation. The feature also makes Bilbo less verbose as it means no end-of-line character, such as a semi-colon, is needed. This means a parser can delimit new statements with a one-character look-ahead, because expressions alone are not valid.

Whitespace has no meaning in Bilbo. This was chosen so graphs in existing structured formats, such as JSON, could be easily converted, programatically, into Bilbo's graph format (discussed in Section 5.2.1). Python has a similar reasoning for allowing trailing commas in lists, it makes programatic code generation easier [75]

5.1.2 Printing and Importing

Almost all values in Bilbo are immediately serialisable. Values can be printed to the console using `print` or to a file by adding a `to` followed by a filename. If the file doesn't exist then it is created, if it does, it is appended to. Printing is shown in Listing 5.2.

```
a = 10
b = 3.14159
c = "Hello, World"
d = False
e = int(b)       // 3
f = str(b)       // "3.14159"
```

Listing 5.1: Assignments and type casts.

```
outputFile = "out.txt"
hello = "Hello, World"
print hello
print hello to outputFile
```

Listing 5.2: This program prints `"Hello, World!"` to the console and writes it to the file `out.txt`.

```
// alphabet.bb
a = "a"
b = "b"
```

(a) The file `alphabet.bb`.

```
import "alphabet.bb"
print a // prints "a"
```

(b) Imports `alphabet.bb` to the current namespace and prints `"a"`.

```
import "alphabet.bb" as alpha
print alpha.b // prints "b"
```

(c) Imports `alphabet.bb` to the namespace `alpha` and prints `"b"`.

Listing 5.3: Importing other Bilbo files.

```
type City = name, capital, population
ldn  = City("London", True, 100)
manc = City(population=20, name="Manchester", capital=False)
print ldn
// prints City(capital=True, name="London", population=100)
```

Listing 5.4: User-defined type and object instantiation.

Files can also be read using `import` followed by the filename as a string literal. The file is read and executed as a Bilbo program, so it must be valid Bilbo code. Anything defined in the imported file becomes available in the current namespace. To allow for better modularity and to reduce name clashes, files can also be imported into a particular namespace using `as` followed by the namespace name. The namespace is accessed using `.`, as shown in Listing 5.3. Namespaces provide modularity as well conceptual and structural familiarity (discussed in Section 4.1) which can increase productivity.

The `print ... to ...` statement can be passed a filename after the `to` as a variable, however `import` requires a literal. This allows all parsing to be done before starting evaluation and therefore syntax errors in imported files will be caught before any evaluation is done.

5.1.3 User-Defined Types and Objects

User-defined object types are created with the `type` keyword, followed by the type name, an equals sign, and then comma-separated field names. Objects can be instantiated using the type name followed by comma-separated field values in the same order that they were defined, enclosed in brackets. Alternatively, the field values can be given in any order if the field name is also specified using `=`. This allows objects to be completely serialisable. Fields can be accessed and assigned using the `.` operator. An example of a user-defined type and object instantiation can be seen in Listing 5.4.

```
x = ldn is City          // True
y = ldn has population   // True
z = ldn has country      // False
```

Listing 5.5: Type introspection. This program continues from Listing 5.4.

Type introspection is possible using the `is` and `has` operators. `is` will check if the value on the left is of the type on the right and `has` will check if the object on the left has the field on the right, as shown in Listing 5.5.

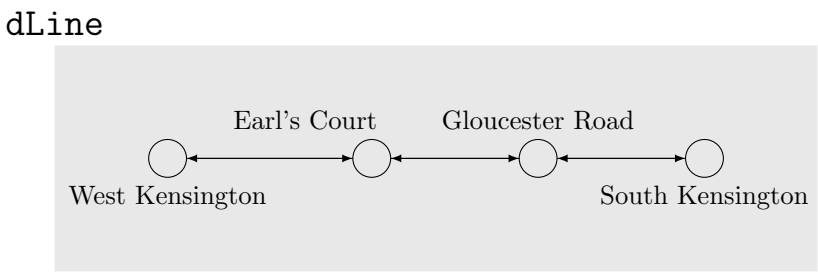


FIG 5.1: A portion of the District line from the London Underground.

```
type Station = interchange

inter = Station(True)
notInter = Station(False)

ec = "Earl's Court"::inter
sk = "South Kensington"::inter

gr = "Gloucester Road"::notInter
wk = "West Kensington"::notInter

dLine = [wk,<>,ec,<>,gr,<>,sk]
```

LISTING 5.6: Construction of the graph `dLine` shown in FIG 5.1.

5.2 Graph Construction

This section covers the syntax and semantics for constructs related to graph construction. The syntax and semantics for graphs, however, are used in multiple places in the language. For example they are used to specify the left- and right-hand side graphs in transformations. For this reason, graph construction is designed to be readable and concise.

5.2.1 Nodes, Edges and Paths

Consider the graph `dLine` (for *District line*) in FIG 5.1 that shows a portion of the London Underground. The bidirectional arrows represent two distinct edges (one in each direction) but in order to keep the diagram clear these are shown as a single arrow with two heads. We note that Earl's Court and South Kensington are interchange stations (this property is not shown on the graph). The program in LISTING 5.6 shows the construction of this graph.

Nodes in Bilbo have an *identifier* and a *load*. A load holds the data relevant to the node, while

```
dLine = [ec,>,wk,>,ec,>,gr,>,ec,gr,>,sk,>,gr]
```

Listing 5.7: The path from Listing 5.6 constructed again without using `<>`.

an identifier uniquely identifies it within a graph. The node construction operator `::` joins the identifier on the left to the load on the right, to form a node. So the node `ec` (on line 6) is a node with identifier `"Earls's Court"` and load `Station(interchange=True)`. Unless stated otherwise, for clarity graph diagrams in this section will only show node identifiers and not the loads.

Edges are constructed with the edge operators `>`, `<` that require a node either side. `>` will create an edge from the node on left to the node on the right, similarly for `<` but the other way round. Edges are unidirectional so the operator `<>` constructs two edges, one in each direction. Furthermore, loops (an edge with the same source and target node) are allowed as are multiple edges in the same direction.

Graphs are constructed using *path expressions*, or just *paths*. Paths are defined using square brackets enclosing comma-separated nodes and edges, as seen on line 12. The order of nodes within a path, as long as they are not part of an edge, has no importance. For example, `[x,y]` and `[y,x]` are the same. To be precise, a *path* refers to the syntax of constructing graphs. When evaluated, a path becomes a graph.

The `<>` operator is a quicker way to construct edges in both directions. If `x` and `y` are nodes then `[x,<>,y]` is a syntactic sugar for `[x,>,y,>,x]`. This is an appropriate use of syntactic sugar because it makes paths much more readable [54]. For example, the path in Listing 5.7 is the same path as that on line 12 in Listing 5.6, but is much more difficult to read, motivating `<>`. Nodes can appear multiple times within a path. In general, nodes that have the same identifier become the same node which is why `[x,>,y,>,x]` is the same as `[x,<>,y]`. This allows an intuitive concept of graph addition which is discussed in Section 5.2.2. Moreover, `[x,x,x,x]` is the same as `[x]`. If two nodes have the same identifier but different loads then the load appearing later in the path is always chosen.

Edges can also be weighted. If it takes 3 minutes to get between Gloucester Road and South Kensington, in both directions, this can be shown as `[gr, <3>, sk]` using the nodes defined in Listing 5.6. Similarly, the edge operators `>` and `<` are weighted as `w>` and `<w` for a weight `w`.

The edge operators were chosen to allow concise but readable definitions of graphs. The graph syntax structurally resembles the graph. Edge operators, however, cannot be used outside paths. For example,

```
e = 3>
```

is not valid. This means that readability of edges is not lost. If `e= 3>` was valid, then for the path

```
[gr, e, sk]
```

it is no longer obvious whether this is three nodes, or two nodes connected by an edge. The restriction ensures that paths remain readable and only requires a single character more for `>` and `<`, or two for `<>`. This is a worthwhile tradeoff since not much is gained in allowing the binding of edges (with no nodes) to a variable.

The identifier and load of a node can be accessed and assigned directly using the prefix operators `&`, which gets the identifier, and `#`, which gets the load. So `&ec` is `"Earl's Court` and `#ec` is `Station(interchange=True)`. To access load fields directly, analogous to accessing the field of pointed to object in C++ with `->`, one can use `..`. So `(#ec).interchange` and `ec..interchange` are the same. The operator `..` was used rather than `->` because `->` may cause confusion as it looks like an edge operator.

Two nodes are equal if they the same identifier and load. Moreover, two graphs are equal if they have exactly the same nodes and exactly the same edges (including the weights).

To allow for Bilbo to be used for problems where the data is already in a structured format, such as JSON or XML, the graph construction syntax was designed to be very readable for humans, and easily parseable and *writable* by a program. This is discussed more in Section 5.2.5.

5.2.2 Graph Addition and Subtraction

In graph theory there are no standardised definitions of graph addition and subtraction. However, Bilbo defines graph addition such that larger graphs can be built up incrementally. Graph addition was designed to reflect an intuitive notion of combing graphs in an additive way. For example, consider the graph `dLine2` in Fig 5.2 showing a larger portion of the District line. `dLine2` build on `dLine`. Intuitively this is a superposition of the original graph `dLine` and another graph that contains West Brompton, Earl's Court and High Street Kensington (and the edges between them). Graph addition captures this superposition. The construction of `dLine2` is shown in Listing 5.8. Since the node `ec` is used, and in particular, because the identifier of `ec` is already present in `dLine`, the edges are joined to the graph in the expected way (recall that nodes with the same identifier are the same node).

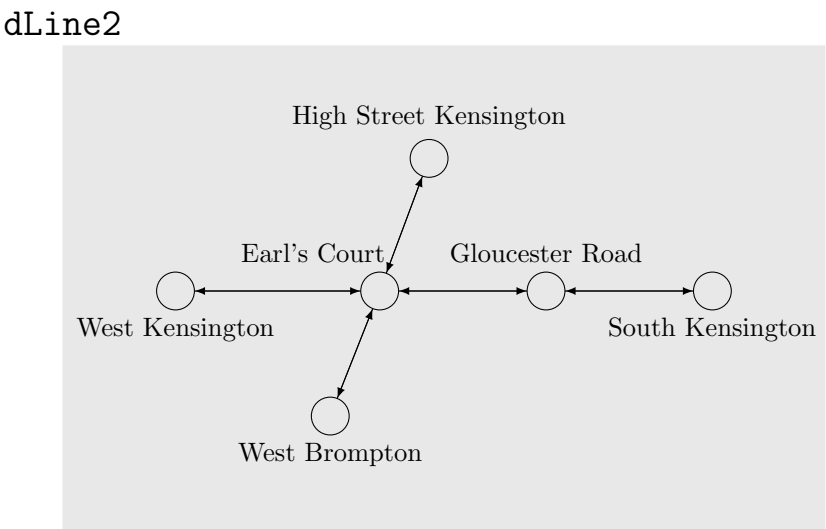


FIG 5.2: The District line portion shown in FIG 5.1 with the addition of High Street Kensington and West Brompton (which is an interchange station).

```
wb = "West Brompton"::inter
hk = "High Street Kensington"::notInter

dLine2 = dLine + [wb, <>, ec, <>, hk]
```

LISTING 5.8: Construction of the graph `dLine2` shown in FIG 5.2. This program carries on from LISTING 5.6.

Graph subtraction was also defined to capture intuitive subtractive operations. In particular, the definition of subtraction is motivated by two potential subtractive operations relevant to graphs:

1. Remove a node and all its incident edges.
2. Remove an edge but not the endpoints of the edge.

The definition of subtraction that follows, at first, may seem contrived or peculiar, however it allows these two operations to be expressed easily and intuitively.

The subtraction `g - h`, where `g` and `h` are both graphs, will

1. remove from `g` any edges in `h`, and
2. remove from `g` any node in `h` that is not an endpoint of an edge in `h`. Edges must have two endpoints and so any edges incident to the removed node are also removed.

Edges are only removed if the weight (or lack of weight) and direction match up.

Recall the two subtractive operations mentioned above.

The first is expressed as follows. Let `g` be a graph and `n` be the desired node to remove from `g`. This is simply expressed as

`g - [n]`.

Since the graph `[n]` has no edges, `n` is not an endpoint, and therefore it, and any edges incident to it, are removed from `g`, which was the desired operation.

The second operation is also easily expressed. Let the desired edge, to remove from the graph `g`, be from a node `n` to a node `m`. The operation is expressed as

`g - [n,>,m]`.

The graph `[n,>,m]` has no nodes that are not endpoints (both `n` and `m` are endpoints), and so, per the definition above, only the edge is removed and not the endpoints, which was the desired operation.

We note that this definition of graph subtraction allows operations that are precise, in that a user can specify exactly what they want to be removed, but they do not expose the underlying representation of the graph. This allows for expressive operations while keeping the level of abstraction concentrated on the problem domain. Furthermore, this definition of graph subtraction syntactically resembles the semantic operation. That is to say, `g - [n]` syntactically denotes the removal of a node and `g - [n,>,m]` syntactically denotes the removal of an edge. These properties of graph subtraction justify its seemingly peculiar definition.

Two examples of graph subtraction are shown in Listing 5.9 with the corresponding graphs in Fig 5.3. `dLine3` removes the node for Earl's Court, while `dLine4` removes only the edge between Earl's Court and Gloucester Road, but neither of the nodes.

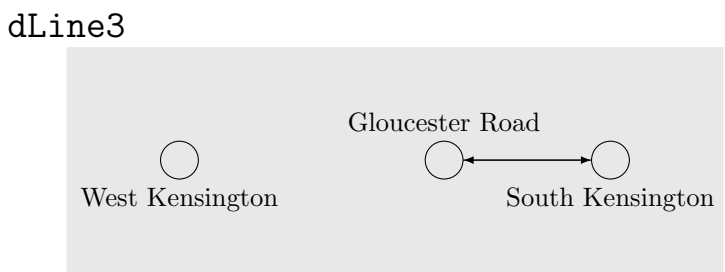
A third subtractive operation one may wish to do is, given graphs `g` and `h`, remove from `g` anything (nodes or edges) that appears in `h`. As a result, any edges in `g` (but not in `h`) that are connected to nodes in `h` will also be removed since an edge must have two endpoints. This operation can be expressed in Bilbo as

`(g - h) - (h - h)`.

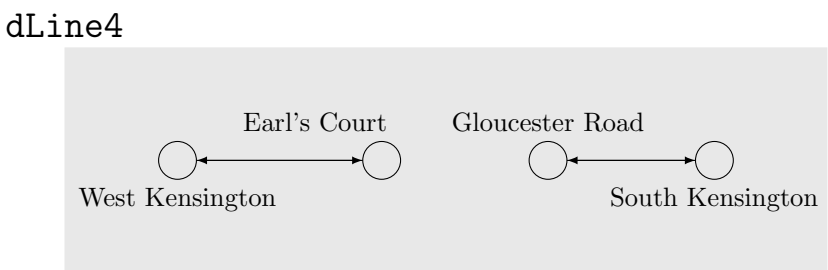
Firstly, the graph `(h - h)` will contain only the endpoint nodes of `h`. Secondly, `(g - h)` will remove from `g` any edges in `h` and any non-endpoint nodes in `h`. As a result, the only thing remaining in `(g - h)` from the original graph `h` is the endpoint nodes of `h`, which is exactly what the graph `(h - h)`. Since `(h - h)` has no edges, all nodes are non-endpoint nodes and therefore will be removed from `(g - h)`, leaving the desired graph.

```
dLine3 = dLine - [ec]
dLine4 = dLine - [ec,<>,gr]
```

Listing 5.9: Graph subtraction being used to remove nodes (`dLine3`) and remove edges (`dLine4`). The resulting graphs are shown in Fig 5.3. This listing carries on from Listing 5.6.



(a) The result of `dLine3 = dLine - [ec]`.



(b) The result of `dLine3 = dLine - [ec,<>,gr]`.

Fig 5.3: Graph subtraction can be used to remove nodes (as in Fig 5.3a) or edges (as in Fig 5.3b).

```
dLine = [<> : wk, ec, gr, sk]
```

Listing 5.10: Constructing the graph `dLine` from Fig 5.1 using a path comprehension.

5.2.3 Path Comprehensions

Path comprehensions and nested paths (which are covered in the next section) are used to exploit regularity in graphs, making them easier to construct. These features are inspired by list comprehensions from F#, Python and Haskell as well as set-builder notation from mathematics. Consider again the graph `dLine` in Fig 5.1. There is some regularity in the graph: all nodes appear in a line and any node has a weightless edges in both directions, to the node before and after it (except the nodes at the ends of the line). This can be constructed using a path comprehension as shown in Listing 5.10. This path comprehension applies the edge operator given on the left of the `:` in between consecutive nodes on the right, correctly constructing `dLine`.

There are three regularities that path comprehensions can be used for, as shown in Table 5.1. Multiple path comprehensions can be used consecutively, by comma-separating them before the `:`. They are applied in order from left-to-right.

Path Comprehension Operator	Description	Example	Usage
>, <, <> or their weighted equivalents	Applies the given edge operator in between consecutive nodes on the right.	`[<10> : x, y, z]` is the same as `[x, <10>, y, <10>, z]`	Constructing longer paths quickly.
`#=L`	Creates nodes where each element in the path is an identifier and `L` is the load.	`[#=10 : "a", "b"]` is the same as `["a"::10, >, "b"::10]`	Initialising graphs.
`&=`	Sets the identifier and load of each element in the path to that element.	`[&= : "a", >, "b"]` is the same as `["a"::"a",>,"b"::"b"]`	Constructing graphs when the initial node loads are not important.

TABLE 5.1: Path comprehension operators.

5.2.4 Nested Paths

Nested paths are used to exploit the construction of graphs that appear to *spread out* from, or *converge* to, a particular area on the graph. For example, looking at the graph `dLine2` in FIG 5.2, we see that all but one node have a weightless edge in both directions with Earl's Court. It seems that the graph `[wk, wb, gr, hk]` has edges `<>` to and from `ec`. This is exactly the motivation for the nested paths feature. `dLine2` is expressed with a nested path in LISTING 5.11.

```
dLine2 = [ec, <>, [wk, wb, gr, hk]] + [gr, <>, sk]
```

LISTING 5.11: The graph `dLine2` from FIG 5.2 expressed using a nested path.

In general, the nested path `[f,>,g,>,h]`, where `f`, `g` and `h` and are paths, will apply the edge operator (in this case `>`, but any can be used) firstly to all pairwise combinations of nodes taking

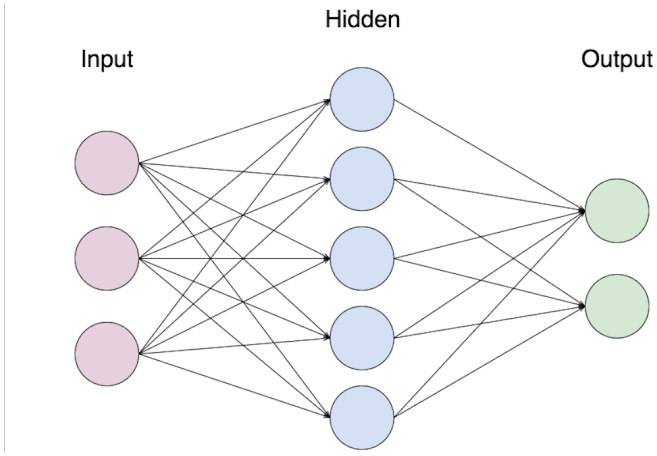


FIG 5.4: A typical representation of a neural network [76].

the first node from `f` and the second from `g`, and then all pairwise combinations of nodes taking the first from `g` and the second from `h`. All original edges are preserved. Nested paths can be freely used with path comprehensions or already constructed graphs to concisely construct larger graphs.

As an indicative example, the neural network shown in FIG 5.4 can be constructed using a nested path by realising that the all nodes in one layer connect to all other nodes in the next layer. Furthermore, typically all nodes in one layer will have the same function, which can be expressed concisely using a path comprehension. The neural network is constructed in LISTING 5.12. For simplicity, nodes have integer identifiers `1` - `10` and the function of each node is given as a string.

5.2.5 Outputting Graphs

Graphs are fully serialisable using `print` or

```
print g to "output.txt"
```

which writes `g` to the file `output.txt`. Graphs are serialised in a canonical form, rather than as longer path expressions. Each edge is a single path and these are summed together. For example

```
["a"::10, >, "b"::20, >, "c"::30, "d"::40]
```

is serialised as

```
["a"::10, >, "b"::20] + ["b"::20, >, "c"::30] + ["d"::40].
```

This choice was made on the basis of four points

1. The output graph is valid Bilbo code and so can be written to a file and read in to another Bilbo program using `import`.

```
type Node = function

in = Node("input")
hid = Node("hidden")
out = Node("output")

// use path comprehensions to set the loads
inputLayer  = [#=in: 1, 2, 3]
hiddenLayer = [#=hid: 4, 5, 6, 7, 8]
outputLayer = [#=out: 9, 10]

// use a nested path to join every node in a layer
//  to every node in the next layer
net = [inputLayer, >, hiddenLayer, >, outputLayer]
```

Listing 5.12: The neural network in Fig 5.4 constructed as the graph `net`. For simplicity, nodes have integer identifiers `1` to `10` and strings are used to describe the function of each node.

2. The output graph is easy to read for a human.
3. The output graph is easy to parse by another program which may convert it to a structured data format, such as JSON.

With regards to the last point, in such formats, for example in [77, 78, 79, 80], edges are usually given one at a time, rather than as a sequence of edges. Bilbo's canonical serialisation means it is straightforward to convert Bilbo graphs into such formats because a sequence of edges are serialised one edge at a time, as shown above. For similar reasons, Bilbo graphs can be easily produced programatically in a general-purpose programming language. The fact that whitespace has no meaning in Bilbo is also helpful for programatic code generation, as mentioned in Section 5.1.1, and was in part the motivation for this feature.

```
def removeEdge(g) =
    match g
    | [x,>,y] => become [x,y]
```

Listing 5.13: A transform that matches on an unweighted edge and removes it.

5.3 Graph Manipulation

The previous section focused on constructing graphs. This section will focus on how graphs can be manipulated using transforms, based on the graph rewriting model of computation discussed in Section 2.2. This will form the basis of the solutions to well-known graph problems in Bilbo, presented Section 7.1.

5.3.1 Transforms

Graph transforms in Bilbo are used to implement graph rewriting and so are based on production rules from Section 2.2. The syntax for transforms is inspired by the F# syntax for pattern matching with influences from the mathematical notation used for graph rewriting.

Basic Transforms

Consider program in Listing 5.13. This defines a the transform `removeEdge` that accepts one argument, a graph which will be bound to `h`, and matches within that graph (making that graph the host graph) any subgraph isomorphic to `[x,>,y]`, the left-hand side of the graph. `[x,>,y]` is also called the *pattern graph*. Once a match is found, the unbound variables `x` and `y` become bound to nodes within the host. The `=>` is used to delimit between the left-hand side graph and the right-hand side of the transform. This is simmilar to the notation used for direct derivations of production rues in Section 2.2, such as $G \Rightarrow H$. The right-hand side uses the keyword `become` which can only appear on the right-hand side of transforms and can only be applied to graphs. `become` returns the original graph, `g`, but with the bound pattern graph replaced with the graph `become` is applied to. That is, in this case, it replaces the subgraph that `[x,>,y]` binds to in `g`, with `[x,y]`.

Transforms are applied to a graph by *enpiping*[1] the graph through the the transform. The enpipe operator is `>>`. In Bilbo, sequences of transforms are called *pipelines* and operators for composing pipelines are covered later in this section and in Section 5.4 where pipeline modifiers are introduced.

[1] The word *enpipe* is used to mean pushing something into a pipe, similar to how *enqueue* means to add something to a queue.

```
g = [a,>,b,c]
g' = g >> removeEdge
```

Listing 5.14: Enpiping `g` into `removeEdge`, defined in Listing 5.13. `g'` is equal to `[a,b,c]`.

Let `a`, `b` and `c` be nodes. The program in Listing 5.14 enpipes `g` into `removeEdge`. The resulting graph is `[a,b,c]`. When the transform is applied, the unbound pattern graph `[x,>,y]` binds to `[a,>,b]`, and then replaces this with `[a,b]`, removing the edge. The node `c` is unaffected during this process because of the semantics of `become`. This is conceptually very similar to a production rule in graph rewriting.

Rewrite Semantics

The tools covered in Section 2.3 each had rewrite semantics derived from the double or single pushout approaches, covered in Section 2.2. Similarly, Bilbo's rewrite semantics is based on the single pushout with isomorphic, rather than homomorphic, matching. In particular, Bilbo's rewrite semantics are:

1. *Single pushout.* Conflicts between preservation and deletion favour deletion. There is no dangling condition.

2. *Isomorphic matching.* Every unbound variable used in the pattern graph matches a node or edge weight in the host graph that no other unbound variable matches. Therefore, if two unbound variables are different (for example `x` and `y` in `removeEdge`), they will not match the same node. In Bilbo this is called the *unbound variable guarantee* and means that the bijective mapping between the left- and right-hand side graphs is obvious and natural.

The second point means that the transform `removeEdge` will not remove loops since two distinct unbound variables `x` and `y` are used and so a loop such as `[a,>,a]`, where `a` is a node, will not match. Furthermore, a weighted edge in the host graph will not match an unweighted edge in the pattern graph, and vice versa.

The single pushout approach was chosen since the double pushout requires the dangling condition to hold. This means transforms would be more verbose to explicitly show deletion.

Isomorphic matching was chosen because it is more intuitive than homomorphic matching. For example, the pattern graph `[x,>,y]` suggests that there are two distinct nodes `x` and `y`. If homomorphic matching was used, this pattern graph could bind `x` and `y` to the same node which is unintuitive and could lead to bugs, decreasing productivity.

```
def removeEdge(g) =
    match g
    | [x,>,y] => become [x,y]
    | [x,>,x] => become [x]
```

Listing 5.15: `removeEdge` matches on an unweighted edge, including a loop, and removes it.

Multiple Match Cases

In the previous section it was noted that the transform `removeEdge` would not match loops because of the unbound variable guarantee. If the intended behaviour was to remove any edges, including loops, then this can be solved by also matching on the pattern graph `[x,>,x]`. Transforms can have multiple match cases, each with a left- and right-hand side. The cases are attempted in order and the first pattern graph that matches is bound and its right-hand side evaluated, without attempting the later match cases. The transform `removeEdge` can be rewritten as shown in Listing 5.15.

`removeEdge` could also be extended, by adding more match cases, to match on weighted edges and weighted loops and remove them as well. Having multiple match cases in a transform is a useful feature (as shown by its use in the solutions to well-known graph problems in Section 7.1) not available in any of the rewrite tools reviewed in Section 2.3.

5.3.2 Multiple Matches, Nondeterminism and Collections

Computation based on graph rewriting has inherent nondeterminism because of the possibility of a production rule matching multiple times within the host, as discussed in Section 2.2.5.

In Bilbo, the default behaviour, if there are multiple matches, is to return all possible graphs as a *collection*. A collection can be seen as an unordered set of graphs that can be operated on as if it were a single graph. Collections can be created explicitly by separating graphs with the operator `|&|`. For example, given two graphs `f` and `g`, they can be placed into a collection `c = g |&| f`.

Let `a`, `b`, and `c` be nodes and recall the transform `removeEdge` in Listing 5.15. In the program in Listing 5.16, multiple subgraphs of `g`, `[a,>,b]` and `[b,>,c]`, will match in `removeEdge` and so `g'` is the collection `[a,b,>,c] |&| [a,>,b,c]`.

The syntax for collections means that a single graph and a collection containing a single graph is the same thing. This is by design. Collections liberate programmers from worrying about the nondeterminism or how many graphs were returned from a pipeline. Operations on collections, such as graph addition and subtraction, or even collection addition and subtraction, are applied

```
g = [a,>,b,>,c]
g' = g >> removeEdge
```

Listing 5.16: Enpiping a graph into `removeEdge` from Listing 5.15.

to all the graphs in the collection or collections. So

`(g1 |&| g2) + (g3 |&| g4)`

will yield a collection

`g1+g3 |&| g1+g4 |&| g2+g3 |&| g2+g4`.

The operator `|&|` has the second lowest precedence in Bilbo, only higher than `>>`. This doesn't include `=` for assignments which is always evaluated last.

In Section 5.4.3 it will be shown that Bilbo allows programmers to also control this nondeterminism and to take advantage of it. In the next section, the composition of transforms into a pipeline is covered.

It should be noted that collections, like graphs, are completely serialisable and are outputted as valid Bilbo code.

Return and Query-Style Transforms

So far, all transforms have used `become` to replace the pattern graph. Instead using `return`, with transforms and collections, it is possible to query the graph for certain patterns. `return` will return whatever it is applied to.

Recall the District line graph `dLine2` defined in Listing 5.8 and shown in Fig 5.2. Imagine one wanted to obtain all stations that allowed travel in more than the standard two directions. This could be expressed as the transform `manyDirections` given in Listing 5.17. Due to the unbound variable guarantee, the variables `a`, `b` and `c` will match distinct nodes and therefore from the node `x` one can go in more than two directions (to `a`, `b`, and `c`), which is the desired property. Enpiping `dLine2` into `manyDirections` yields a graph containing a single node for Earl's Court. If more such stations existed in the graph, these would be returned in a collection of single-node graphs. Note that because `manyDirections` returns a graph (with a single node `x`), rather than just the node `x`, a collection is formed. This is a good pattern for query-style operations on graphs.

In summary, transforms must have one match statement. A match statement can contain multiple match cases and each match case must have right-hand side terminated with either a

```
def manyDirections(g) =
    match g
    | [a,<,x,>,b] + [x,>,c] => return [x]
```

Listing 5.17: The transform `manyDirection` matches on a node `x` and three distinct edges leaving `x` going to distinct nodes. The node `x` is returned in a graph.

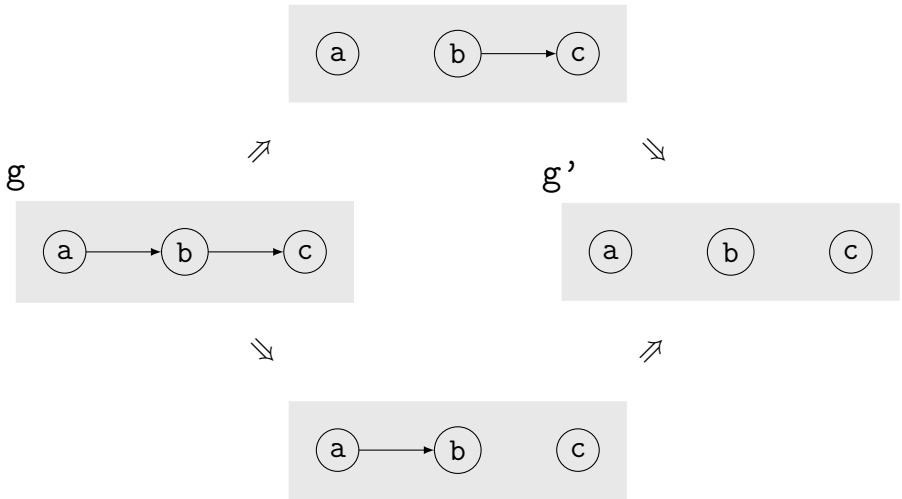


Fig 5.5: The result of enpiping `[a,>,b,>,c]` into a pipeline `removeEdge |> removeEdge` as done in Listing 5.18. The two intermediate graphs (towards the top and bottom) yield the same graph `g'` after the second application of `removeEdge`. For clarity, nodes are labelled here with their variables rather than their identifiers.

`become` or `return` statement. Transforms can have other statements before the match statement and also on the right-hand side of a match case before the `become` or `return`. Functions are similar to transforms but do not have match statements. Functions are covered in Section 5.5.1.

5.3.3 Transform Composition and Pipelines

Transforms are composed into a pipeline with the `|>` operator, called the *then pipe*. The syntax for pipelines, inline with the `|&|` operator for creating collections, means that a single transform can be seen as pipeline of one stage. The transform on the left of the `|>` is applied first and *then* the output of that is enpiped into the transform on the right, hence the name *then pipe*.

Assume again that `a`, `b` and `c` are nodes. The program in Listing 5.18 constructs a pipeline of two stages, both `removeEdge`, and enpipes `[a,>,b,>,c]` into this pipeline. This process is shown diagrammatically in Fig 5.5. After the first application of `removeEdge`, the result is a collection of two graphs (as we saw in Section 5.3.2), shown towards the top and bottom in

```
removeTwoEdges = removeEdge |> removeEdge

g = [a,>,b,>,c]
g' = g >> removeTwoEdges
```

Listing 5.18: The transform `removeEdge` from Listing 5.15 composed twice into a pipeline. The graph `g` has two edges so these will both be removed yielding `g'` equal to `[a,b,c]`. This process is shown diagrammatically in Fig 5.5.

Fig 5.5. This collection is then passed to the second `removeEdge`. The semantics of enpiping a collection of graphs into a transform are also designed to free the programmer from worrying about whether an output of a previous pipeline stage is a single graph (which is just a one-graph collection) or a collection. Each graph in the collection is enpiped through the transform and all the outputs are *collected* together as a single collection. Since the original graph `g` only has two edges, applying `removeEdge` twice will yield a unique graph. To be precise, each of the two intermediate graphs, enpiped through `removeEdge`, will yield one graph (since they each have exactly one edge and so there is only one match) and this graph is the same. Collections have set-based semantics and so the resulting collection contains a single graph `[a,b,c]`.

In graph rewriting systems, such a sequence of rewrites, where the resultant graph is the same irrespective of the order of the matches (as shown in Fig 5.5), is called called a *confluent* sequence of rewrites [81]. These are of great importance in graph rewriting because it means that the rewrite system need not make choices nondeterministically [82]. In Section 5.4.3 we will see that Bilbo allows taking advantage of confluence in a novel way.

5.3.4 Generalised Application Conditions

Negative application conditions (covered in Section 2.2.4) are a common extension to the standard rewrite semantics offered by the double or single pushout approaches. They allow more expressive matching by requiring a particular subgraph not to exist in the host graph.

Bilbo extends the idea of negative application conditions to provide a more general way of expressing pattern graphs in a readable and intuitive way.

Negative Application Conditions

Consider the transform `link` in Listing 5.19. The match uses the `not` operator to achieve negative application conditions. The semantics of this are immediately obvious. The pattern `[a,>,b,>,c]`, called the *positive graph*, must match and the pattern `[a,>,c]`, called the *negative graph*, must not match. Bilbo's unbound variable guarantee means that `a` refers to the same node in both the positive and negative graphs. Notice that the positive graph is replaced with

```
def link(g) =
    match g
    | [a,>,b,>,c] and not [a,>,c] => become [a,>,b,>,c] + [a,>,c]
```

Listing 5.19: The transform `link` adds an edge, if one doesn't already exist, from `a` to `c` if `c` is reachable from `a` using two edges.

```
def link(g) =
    match g
    | [a,>,b,>,c] and not [a,>,c] => become [+] + [-]
```

Listing 5.20: The transform `link` from Listing 5.19 rewritten using `[+]` and `[-]`.

the sum of the positive and negative graphs. This is quite a common idiom because, essentially, the transform is checking that what it is about to do (add an edge from `a` to `c`) hasn't already been done. If it hasn't been done, it does it. In Section 5.4.2 it'll be shown that the ability to do such checks ensures that programs terminate. For productivity, Bilbo allows this idiom to be expressed in shorthand form. The special variables `[+]` and `[-]` are the bound positive and negative graphs, respectively. The transform `link` can be rewritten to use these graphs as shown in Listing 5.20.

The negative graph `[-]` should, however, be used with care. In the `link` example, all the variables that appear in the negative graph also appear in the positive graph, so they all have well-defined bound value. On the other hand, consider the following pattern graph

`[a,>,b] and not [b,>,c]`

This is a valid pattern graph. It matches an edge `[a,>,b]` only if `b` has no outgoing edge to another node `c`, whatever `c` may be. So if this pattern graph matches, `b` must not have an outgoing edge to another node and therefore `c` has no bound value. In this case, using `[-]` would be erroneous. In particular, `[-]` should only be used if the unbound variables used in the negative graph are subset of those used in the positive graph. This isn't a restriction on the usage of `[-]` as such, because using the negative graph as it is given in the pattern (in this case `[b,>,c]`) would also cause the same problem, it is just more obvious that `c` has no value when it is written explicitly.

The use of the operator `and`, in this case, is syntactic sugar for `+`, since pattern graphs can be added in the same way that graphs can be. The prefix operator `not` designates that the pattern it is being applied to is negative. They can be combined freely.

```
def specialDeadEnd(g) =
    match g
    | [a,>,d] and not ([d,>,b] or [e,>,d] or [d,>,a]) =>
        return [d]
```

Listing 5.21: The transform `specialDeadEnd` returns a collection of graphs, each with a single node that has incoming edges from only one node and no outgoing edges.

Disjunctive Application Conditions

On Wikipedia, a page that has no outgoing links to other Wikipedia pages is called a *dead-end* page [83]. Imagine one is searching a graph where nodes represent Wikipedia pages and an edge from `x` to `y` means page `x` has an outgoing link to a page `y`. Imagine further that it is desirable to find a special kind of dead-end page that not only has no outgoing links, but also has incoming links from only one page. The transform `specialDeadEnd` in Listing 5.21 will return such nodes.

The transform `specialDeadEnd` uses the `or` operator for pattern graphs, which represents a disjunctive operation. That is, it requires at least one of the graphs it is applied to to match. In `specialDeadEnd`, `or` is used in the negative graph to make sure none of the edges given would match.

Note, however, that the unbound variable guarantee means that, as well as making sure `[d,>,b]` does not match, `[d,>,a]` must also not much since an edge going back to `a` from `d` would not be matched by `[d,>,b]`.

Due to the semantics of the pattern matching operators (`and`, `or`, `not`), they form a system where De Morgan's laws[2] can be applied. For example, the pattern graph in `specialDeadEnd`, shown in Listing 5.21, could be rewritten as

`[a,>,d] and not [d,>,b] and not [e,>,d] and not [d,>,a]`

The pattern graph system allows for expressive rewriting and querying through transforms and, to the author's, no such system for combining patterns has been incorporated into a practical programming tool before.

[2]De Morgan's laws can be expressed, in propositional logic, as

$$\neg(A \vee B) \Leftrightarrow (\neg A) \wedge (\neg B) \quad \text{and} \quad \neg(A \wedge B) \Leftrightarrow (\neg A) \vee (\neg B)$$

where A and B are propositional variables, $\neg$ is negation, $\vee$ is a disjunctive logical operator, and $\wedge$ is a conjunctive logical operator.

```
def oneAwayFromEarls(g) =
    match g
    | [a,>,e] where &e == "Earl's Court" => return [a]
```

Listing 5.22: The transform `oneAwayFromEarls` finds nodes that are one edge away from Earl's Court.

The Where Clause

The `where` clause in Bilbo patterns is inspired by *guards* from functional programming languages, namely the `when` clause from F#.

The `where` clause allows checking the value of bound variables once a pattern graph has matched. It is an expression and if it evaluates to `True`, the match succeeds and the right-hand side is evaluated.

Recall again the District line graph `dLine2` defined in Listing 5.8 and shown in Fig 5.2. Imagine one wanted to find all the stations next to Earl's Court, that is, one edge away. This can be done by matching on `[a,>,e]` and checking in the `where` clause that `e` is indeed Earl's Court. The transform `oneAwayFromEarls` in Listing 5.22 does this (recall that `&` returns the identifier of the node it is applied to). Note that it `return`s a graph with a single node in it. Enpiping `dLine2` into `oneAwayFromEarls` indeed yields a collection of four graphs, each with a single node in, the four nodes being Gloucester Road, High Street Kensington, West Brompton, and West Kensington.

Since graphs are heterogeneously typed, the type introspection operators `is` and `has` (discussed in Section 5.1.3) can be used effectively in the `where` clause to ensure that a node or edge weight is of a certain type or has a certain attribute.

```
removeTwoEdges = removeEdge |> removeEdge
removeTwoEdgesAgain = removeEdge ** 2

removeThreeEdges = removeEdge ** 2 |> removeEdge
removeEightEdges = (removeTwoEdges |> removeTwoEdges) ** 2
```

Listing 5.23: Multiplicative pipelines defined in terms of `removeEdge`, from Listing 5.15.

5.4 Pipeline Modifiers and Reductions

This section introduces pipeline modifiers which are operators for constructing and modifying pipelines.

5.4.1 Multiplicative Application

Pipelines can be constructed by separating transforms with the operator `|>`. Repeated application of the same transform can be expressed using the `**` operator which acts like a multiplier for pipelines. It is a binary operator that requires a pipeline on the left and an integer on the right. For example, recall the transform `removeEdge`, defined in Listing 5.15, that removes an unweighted edge from a graph. Using the `**` operator, pipelines made out of `removeEdge` are defined in Listing 5.23. The transforms `removeTwoEdges` and `removeTwoEdgesAgain` are exactly the same. The operator `**` is useful when wanting to apply a pipeline a known number of times, and bears resemblance to the concept of Church Numerals for functions [84].

Note, however, that if one wanted to remove *all* edges from a graph, the number of edges must be known a priori in order to know how many times `removeEdge` is to be applied. The pipeline modifier introduced in the next section solves this issues.

5.4.2 As-Long-As-Possible Application

The postfix operator `!` is the *as-long-as-possible* application pipeline modifier. The pipeline `p!` will apply `p` as many as times as possible, until any stage in `p` fails to match. It then returns the latest successful output of `p`. The means that if, in the pipeline

`(T1 |> T2 |> T3)!`

`T2` fails to match, the latest output of `T3` will be returned.

This property of the operator `!` was designed to be analogous to the atomicity[3] property of databases, that is, it applies the *whole* pipeline as many times possible. The output will never

[3] *Atomicity* is a database property where a transaction on the database either occurs successfully in its entirety, or not at all.

```
g = [a,>,b,>,c,>,d]
g' = g >> removeEdge!
```

Listing 5.24: Applying `removeEdge`, defined in Listing 5.15, as many times as possible to `g`. `g'` is equal to `[a,b,c,d]`.

be a result of applying the pipeline only part of the pipeline. If the pipeline fails on an internal stage (such `T2`, above) it will return the output of the latest successful, complete, application of the pipeline. If, alternatively, the pipeline fails within its first application, the original input is returned. This element of the semantics, regarding failure within the first application, may seem peculiar, however, it is justified below.

The desired operation of removing all edges from a graph without knowing how many there are can now be addressed. Let `a`, `b`, `c`, and `d` be nodes. The program in Listing 5.24 applies `removeEdge` as many times as possible to the graph `g`. The value of `g'` is single graph containing the nodes of `g`, with no edges.

There is an interesting paradigm shift here. Where recursion would have been used in classical graph algorithms, as-long-as-possible application is used in Bilbo. Transforms, applied as-long-as-possible make the *base case*, to use the terminology of recursion, very explicit. Simply, the base case is when the left-hand side no longer matches. This shift in paradigm prompts a good pattern for solving graph problems in Bilbo: if the defining characteristic of a solution to a graph problem can be expressed as a graph that *does not contain* a particular subgraph, then such a problem is likely solvable with as-long-as-possible application. Furthermore, the subgraph that the solution *does not contain* is likely to be the left-hand side of a transform that is (potentially as part of a pipeline) applied as-long-as-possible. In the indicative problem of removing all the edges from a graph, the defining characteristic is that the final graph *does not contain* any edges. Therefore, the subgraph that should not be found in the solution is simply an edge (with two endpoints) and indeed we see that this manifests itself as the left-hand side of a transform applied as-long-as-possible, in `removeEdge!`. This pattern for solving graph problems in Bilbo is in fact used in the solutions to the shortest path problem, the transitive closure problem, and the vertex colouring problem in Section 7.1.

The operator `!` was chosen to be a postfix operator because one may imagine the output of the pipeline being enpiped back round, from the end of the pipeline to the beginning. Furthermore, the semantics of returning the original input if the pipeline fails within its first application are now justified: consider enpiping a graph with no edges into `removeEdge!`. Intuitively, this should return the original graph. The semantics of the `!` operator were influenced by the same operator in GP 2 [47].

It is worth noting that the features of graph collections, transform composition using `|>`, and as-long-as-possible application with `!` make Bilbo a computationally complete language under the graph rewriting model of computation (which was discussed in Section 2.2.7). That is to say, any function (in the mathematical sense) on graphs can be computed using Bilbo.

Considering again the application of `removeEdge!`, there is a large amount of nondeterminism in the intermediate stages, before termination. This is shown in Fig 5.6. The intermediate collections have been grouped as i_1 and i_2. The collection i_1 is the result of removing one edge from `g`. Similarly, i_2 is the result of removing one edge from each of the graphs in i_1. The collection i_2 would in fact contain six graphs (since i_1 has three graphs each with two edges and therefore there are six matches) however only three of these are unique. Maintaining the intermediate collections turns out to be wasteful because all the resultant graphs are the same, regardless of the order the matches. As seen in Section 5.3.3, such an operation is said to be *confluent*. Confluent operations such as this can be handled without any mechanism for nondeterminism [82]. Indeed, at the two intermediate stages i_1 and i_2 in Fig 5.6, maintaining any one of the graphs would have yielded the same graph. This motivates the pipeline modifier introduced next.

5.4.3 One-Match Application

The prefix operator `$` is the *one-match* pipeline modifier. The modified pipeline `$p` applies the pipeline `p` and maintains only one output graph for every input graph. That is, if a collection of c graphs were enpiped into `$p`, the output collection would have at most c graphs.

The one-match operator is suited to confluent operations. For example, consider again the task of removing all edges from a graph without knowing how many edges it has. The particular graph maintained at every stage is not of importance, because the order of matches used does not affect the result, so we need only keep one graph at each. This is the exact semantics of `$removeEdge`. So the task is best done as

```
($removeEdge)!
```

Using `$` with `!` in this way is idiomatic for confluent operations (as we will see in Section 7.1) and so the precedence of the operators was chosen to increase readability and not require brackets. Therefore, an equivalent, more readable, solution is

```
$removeEdge!
```

The operator `$` was chosen to be a prefix operator since it appears as a prefix operator in a number of existing languages, such as Bash, Tcl and PHP.

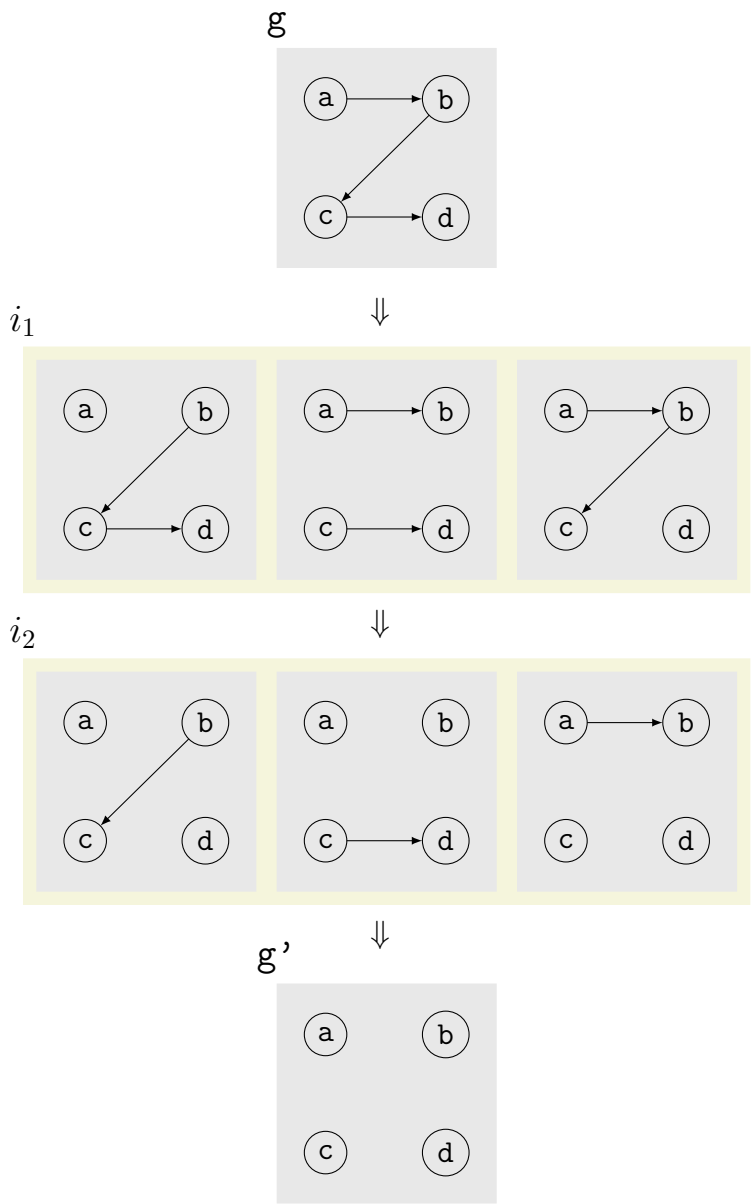


FIG 5.6: The graphs maintained at every stage of `g >> removeEdge!` from LISTING 5.24. i_1 and i_2 represent the intermediate collections. This graph shows node variables rather than identifiers.

5.4.4 Optional Application

Up until now, failing to match, unless within a pipeline applied as-long-as-possible, would have resulted in a program failure. This means that there is no way to apply a pipeline (and therefore a transform) a set number of times (not as long as possible) without knowing in advance that it will match. This motivates the postfix operator `?` which provides optional application.

The pipeline `p?`, pronounced *maybe p*, will attempt to apply `p` once. If any stage in `p` fails to match, the original input to the pipeline is returned.

5.4.5 The Or Pipe

Consider again the problem of removing all edges from a graph. This can be done by enpiping the graph into `$removeEdge!`. Knowing that the aim is to remove all edges from the graph, the number of applications required can potentially be reduced if a pair of edges in both directions between a given pair of nodes (or the same node) were matched and removed in one go. We'll call such a pair of edges, for brevity, an *edge pair*. An edge pair between different nodes can be removed using the following match case

```
| [x,<>,y] => become [x,y]
```

and similarly for an edge pair with one node using

```
| [x,<>,x] => become [x]
```

To achieve this, the above match cases could be added to the existing match cases in the transform `removeEdge`, however that would require rewriting it. The pipeline modifier `<|>`, called the *or pipe*, provides the ability to do this without rewriting the transform. The pipeline

```
p <|> q
```

will first attempt to apply the pipeline `p`. If `p` succeeds then its output is returned. If `p` fails to match then the pipeline `q` will be attempted. If `q` also fails to match then this results in a program failure (which can be avoided using optional application `?`). In the case that `p` and `q` are single transforms (one-stage pipelines), then `p <|> q` is the equivalent of a transform that first has the match cases of `p` and the then the match cases of `q`. However, if `p` and `q` are multi-stage pipelines, this is not the case. In any case, using `<|>` would mean less code repetition and allows keeping `p` and `q` separate transforms, and thus usable individually.

The transform `removeAllEdges` in Listing 5.25 makes use of the `<|>` operator and a transform `removeEdgePair` that contains the match cases discussed above. `removeAllEdges` will

1. first attempt to remove an edge pair of the form `[x,<>,y]` or `[x,<>,x]`,
2. and if that fails (because no edge pairs exist) it will use `removeEdge` to remove an edge of form `[x,>,y]` or `[x,>,x]`.

```
def removeEdgePair(g) =
    match g
    | [x,<>,y] => become [x,y]
    | [x,<>,y] => become [x]

removeAllEdges = ($removeEdgePair <|> $removeEdge)!
```

Listing 5.25: The transform `removeEdgePair` attempts to remove an edge of the form `[x,<>,y]` or `[x,<>,y]`. The transform `removeAllEdges` uses `removeEdge` as well as `removeEdgePair` to remove all edges. If edges of the form that are removed by `removeEdgePair` exist, this will require fewer applications than `$removeEdge!`.

This is then repeated as long as possible until no edges remain.

However, in this case, the pipeline `removeAllEdges` can be further improved by noting that after all of the edge pairs have been removed, `removeEdgePair` will fail to match every time and so `removeEdge` will be applied. `removeAllEdges` can be rewritten to avoid this, as

`removeAllEdges = $removeEdgePair! |> $removeEdge!`

It is worth noting that the flexibility provided by the pipeline modifiers in Bilbo allows complex compositions of transforms to be easily expressed and reasoned about.

The symbol[4] for the or pipe, `<|>`, was inspired by the operator of the same symbol in FParsec [74], the parser combinator library used to write the Bilbo interpreter.

5.4.6 Reductions

Reduction allow selecting a particular graph or subset of graphs from a collection. Reductions go beyond computational completeness of graph rewriting and are a completely novel feature.

Consider a collection of graphs `c`, where each graph in the collection represent a candidate solution to a problem. Using a reduction, from these candidate solutions, a subset, or a unique global solution can be selected. In particular, reductions allow

1. selecting a single graph from a collection based on some criteria, or

2. or filtering the graphs in the collection, selecting a subset of them, based on some criteria.

[4] Till now, it may have seemed that familiar Bilbo simply swapped F#'s *pipe forward* operator `|>` and *function composition* operator `>>`. The choice for the operators was done to allow the *or pipe* to resemble the *then pipe* but pointing to either side, suggesting both pipelines will be used.

```
def distanceLessThan100(g) =
    match g
    | [f] where &f == "finish" => return f..distance < 100

c' = c >> filter by distanceLessThan100
```

Listing 5.26: `c'` is a collection of all the graphs in `c` for which the transform `distanceLessThan100` returns `True`.

```
def getDistance(g) =
    match g
    | [f] where &f == "finish" => return f..distance

bestG = c >> min by getDistance
```

Listing 5.27: The graph `bestG` is the graph which has the lowest distance between the start and finish nodes.

Imagine that the graphs in the collection `c` are each a potential layout for a transport network and the desired solution to the problem is the one that reduces the shortest distance between the node with identifier `"start"` and the node with identifier `"finish"`. Assume that a shortest path algorithm (the implementation of a shortest path algorithm in Bilbo is given in Section 7.1.2) has already been done on all of the graphs in the collection and that the shortest distance between the desired nodes is the attribute `distance` on the finish node.

Let the criteria be that all solutions where this distance is less than 100, arbitrarily, will be considered further. Therefore, the collection `c` must be *filtered* accordingly. This can be done as shown in Listing 5.26. The collection `c'` contains only the graphs from `c` that satisfy this criteria. `by` is a binary operator that requires a reduction specifier, such as `filter`, on the left and a transform on the right. This creates a pipeline stage that can be enpiped into or composed. The `filter` reduction specifier applies the associated transform (in this case `distanceLessThan100`) to every graph in the collection. All graphs for which at least one of the outputs of the transform (since there could be multiple matches, and thus multiple outputs) is `True` are returned as a collection. Therefore this operation yields the desired graphs.

The other available reduction specifiers are given in Table 5.2.

The `min` reduction specifier can similarly be used to find the graph with the shortest distance. The program for doing this is shown in Listing 5.27.

Reduction Specifier	Description
`filter`	`filter by p` applies `p` to every graph enpiped in. Any graph for which cdp returns at least one `True`, is returned in the resulting collection. A graph that does not match is not returned.
`min`	`min by p` applies `p` to every graph enpiped in. The graph with the lowest value (returned from `p`) is returned. If `p` matches multiple times in a given graph, the lowest value is chosen for comparison.
`max`	`max by p` applies `p` to every graph enpiped in. The graph with the highest value (returned from `p`) is returned. If `p` matches multiple times in a given graph, the highest value is chosen for comparison.

Table 5.2: Reduction specifiers.

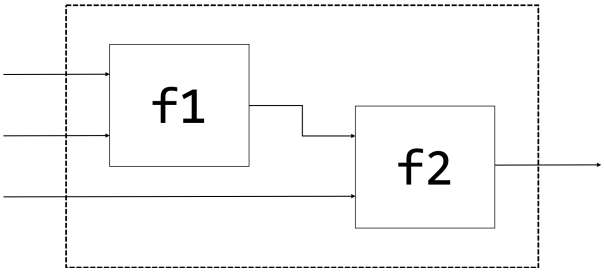


FIG 5.7: The pipeline `f1 |> f2` where `f1` and `f2` both require two arguments can be seen as a single function, shown by the dashed box, that requires three arguments.

```
def avg(x,y) = return (x+y)/2.0

mid = (100,99) >> avg // 99.5
```

LISTING 5.28: Functions are used by enpiping comma-separated arguments. Multiple arguments can be enpiped by enclosing them in brackets.

5.5 Other Bilbo Features

This section introduces other Bilbo features that do not fit under graph construction or manipulation. They have been included here because these appear in programs in SECTION 7.1, as part of the evaluation, which looks at solving well-known graph problems in Bilbo.

5.5.1 Functions

Functions in Bilbo are similar to functions in functional programming languages. They can have multiple arguments and can be applied partially. They are defined similarly to transforms, using `def`, and must finish with a `return` statement. In the style of functional languages, all arguments are passed by value, making functions referentially transparent.

The function `avg` in LISTING 5.28 calculates the mean of the two numbers enpiped in. Functions also use `>>` for enpiping, and multiple arguments should be comma-separated and enclosed in brackets. Functions can be joined together in pipelines using `|>` and `**`.

Functions requiring multiple arguments in a pipeline, however, are not composed as in standard functional composition. Each stage in the pipeline will not proceed until it has received all of its arguments. At which point, the function body is evaluated and the output enpiped to the next stage. This means that two function `f1` and `f2`, that each require two arguments, composed in a pipeline

`f1 |> f2`

```
def sum(x,y) = return x+y

sum3 = sum |> sum
sum5 = sum ** 4

val = (1,2,3,4,5) >> sum5 // 15
```

Listing 5.29: The function `sum` sums its two arguments. This means that `sum3` will sum its three arguments and `sum5` its five arguments

can be seen as a single function that requires three arguments. This is shown diagrammatically in Fig 5.7. While this isn't inline with function composition in typical functional programming languages (such as F# and Haskell), it allows pipelines made up of multiple stages requiring multiple arguments to have all their arguments enpiped in at once, which is more useful in his case. It is therefore a worthwhile tradeoff since Bilbo is designed to be productive rather than theoretically pure.

As an indicative example, the function `sum` in Listing 5.29 returns the sum of its two arguments. This means that `sum |> sum` is a pipeline that sums three arguments, and similarly `sum ** 4` sums five arguments. Transforms with multiple parameters have similar semantics.

5.5.2 Manipulating Nodes In-Place

The node construction operator `::` can be used to manipulate nodes in place. The normal semantics for `::` is that it constructs a node made up of the identifier on its left and the load on its right. If, on the other hand a node is on the left of `::`, for example

`n::l`

where `n` is a node and `l` is not, then this constructs a new node with identifier equal to the identifier of `n` and a load `l`. Similarly,

`i::n`

where `n` is a node and `l` is not, constructs a node with identifier `i` and a load equal to the load of `n`. Both rules apply if there are nodes on both sides of `::`.

While the semantics of `::` with nodes can be achieved using the operators `&` and `#` (discussed in Section 5.2.1), `::` allows the node construction to be done more concisely in-place, without requiring an assignment. For example, the two implementations of `load10` in Listing 5.30 are equivalent.

```
def load10(g) =
    match g
    | [x] =>
        #x = 10
        become [x]
```

(a)

```
def load10(g) = match g | [x] => become [x::10]
```

(b)

Listing 5.30: The use of `::` with nodes allows node manipulation in-place, without needing an assignment. The two programs in (a) and (b) are equivalent however (b) is much more concise and arguably more readable.

5.6 Summary

As seen from this chapter, Bilbo has been designed in a principled way based on the objectives of expressivity and productivity in graph programming, as well good practices regarding language design.

The features covered in this chapter appear extensively in Section 7.1 which presents solutions to well-known graph problems using Bilbo.

Chapter 6

The Bilbo Interpreter

This chapter concentrates on the implementation of the semantics presented in the previous chapter. In particular, it presents the Bilbo interpreter and how it was tested.

F# was a well-justified implementation language for the interpreter, as discussed in Section 4.3.2, and as a result it was possible to switch, seamlessly, between concentrating on development work and concentrating on language design.

Going forward, Bilbo is being continued as an open source project, under the MIT licence [85], at

`https://github.com/maccth/bilbo`.

Instructions on how to use the interpreter (and the accompanying REPL) or run the tests, can be found in the README.

The interpreter currently supports all of the features presented in Chapter 5 except the variables `[+]` and `[-]`. Since these features provide an elegant shorthand for accessing the pattern graph, but not extra functionality, the development effort concentrated on other features.

6.1 Implementation of the Interpreter

This section first provides an overview of the interpreter, and maps constructs explained in Chapter 5 to where they are implemented in the interpreter.

After this, the section concentrates on two particularly interesting parts of the interpreter. Firstly, the process by which a pattern graph within a match case becomes bound, allowing evaluation of the right-hand side (covered in Section 5.3.1). Secondly, the process which allows pattern graphs to be combined using `and`, `or`, and `not` (covered in Section 5.3.4).

The second feature is completely novel and has not been implemented in any other programming tool or language that the author could find.

6.1.1 Overview of The Interpreter

An overview of the modules in the interpreter, and how they interact, is shown in Fig 6.1. The top-level of the interpreter first initiates parsing by calling the parser. If this is successful, the AST is passed to the evaluator. If it is not, the error state is passed to the the output module which is used to ensure all printing and serialisation (for values and errors alike) is done consistently. The top-level interpreter can be found under `src/Bilbo`.

The modules in *Common* represent the common functionality used by both the parser and the evaluator. Of these, *Value* contains the top-level discriminated union for Bilbo values. *Result* contains the main result-style monad used, as well as functions for quickly generating precise and useful error messages. Finally, *Symbols* defines the type for the programs symbols and useful functions for binding values to identifiers. This module is used to maintain program state and therefore the program symbols are the variable bindings. The code for these modules can be found in `src/Common`.

The evaluator receives the AST and traverses it, threading through a result monad of the program symbols. The statement and expression handler are responsible for evaluating all the statements in the AST. Transforms, reductions and functions are all handled through a series of (F#) functions that are responsible for enpiping and applying arguments. The transform handler is responsible for evaluating the match statement but calls out to the graph library to solve the subgraph isomorphism problem. This makes use of the functionality for unbound graphs, found in the graph library. The statement and expression handler also interacts with the graph library directly for graph constructions. The code for these modules can be found in `src/Evaluator` and `src/GraphLib`.

The next two sections now discuss in detail two particularly interesting parts of the interpreter that are both within the *Transform handler* module in Fig 6.1.

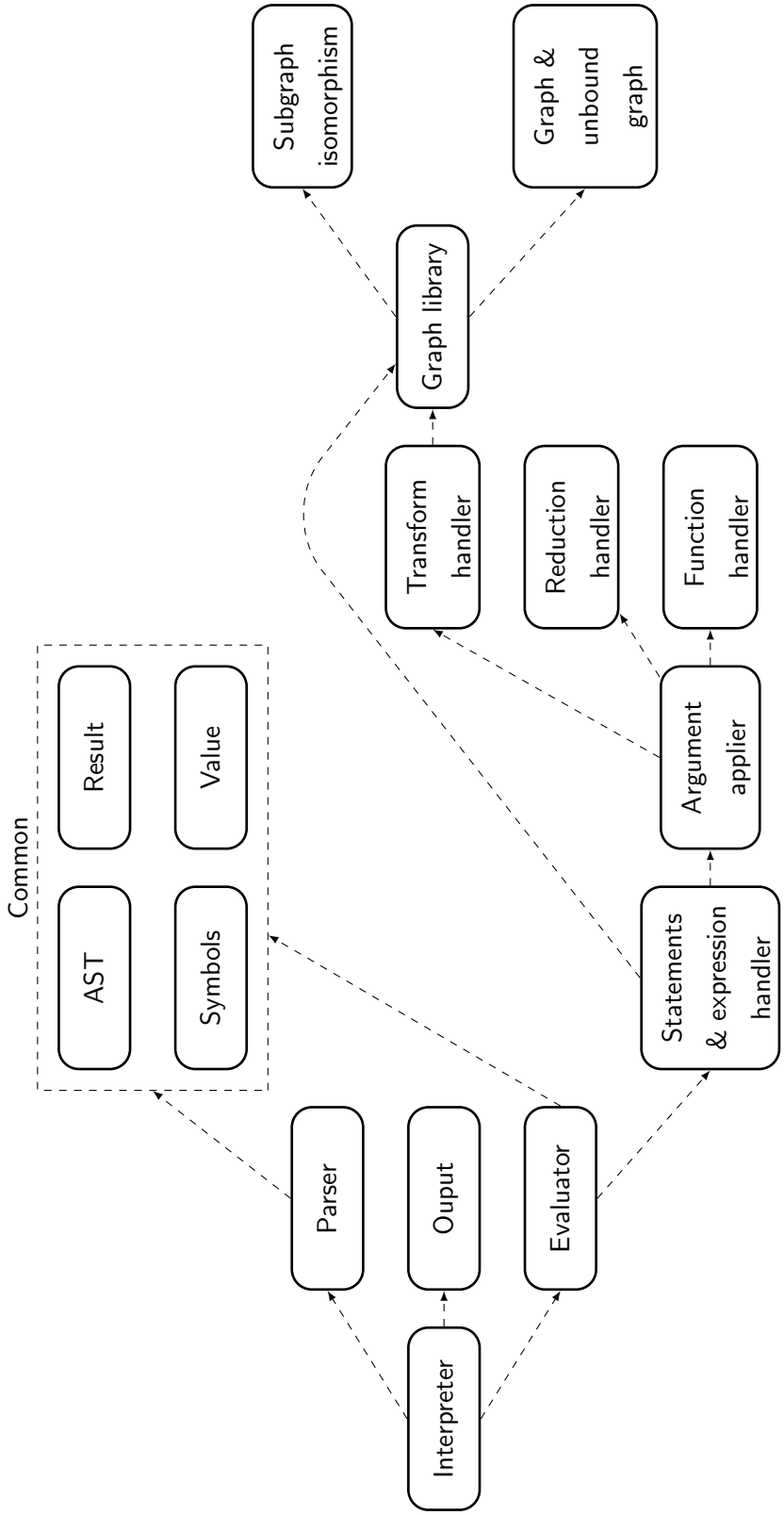


Fig 6.1: This shows module interactions in the Bilbo interpreter. An arrow from module X to module Y means that module X calls out to module Y.

6.1.2 Binding Pattern Graphs

Given a match case of the form

`match g |` P `=>` R

where P represents a pattern graph, R represents the right-hand side, and `g` is the host graph. This section describes the process used in the Bilbo interpreter for binding the variable identifiers in P to nodes and edge weights in `g`. By variable identifiers we mean, if the pattern graph is `[a,w>,b]`, the symbols `a` and `b` which will be bound to nodes, and the symbol `w` which will be bound to an edge weight. This is shown diagrammatically in Fig 6.2 where the blue boxes represent functions and the white boxes represent inputs and outputs. This entire computation takes place in the *Transform handler* module in Fig 6.1.

The first step, shown as *Construct unbound graph* in Fig 6.2, is to parse the pattern graph and turn it into an object of type `UnboundGraph` (this type can be seen in `src/Common/Value.fs`). This process differs from that of parsing a normal path expression (outside of a pattern graph) because the variable identifiers given in a normal path expression must already be bound to a value. Whereas, in the case of a pattern graph, the variables identifiers represent nodes or edges to be bound once an isomorphic subgraph in the host is found. The `UnboundGraph` type stores the pattern as a graph containing `UnboundNodes` and `UnboundEdges` which, instead of having identifiers, loads, and weights, store their corresponding variable identifier used in the pattern graph. In this case, we will assume that the pattern graph does not contain `and`, `or`, or `not`. The process used for pattern graphs that contain these is covered in the next section.

Once an `UnboundGraph` has been created, a subgraph isomorphism problem is solved between the unbound graph and the host, shown as *Subgraph isomorphism* in the diagram. The subgraph isomorphism finds all subgraphs in the host that are isomorphic to the unbound graph. This is done using `Graph.unboundSgiAll` (in `src/Graph/Isomorphism`) which uses a simplified version of Ullman's algorithm [86]. Since there may be more than one isomorphic subgraph, this yields a set of map pairs (each pair containing a node map and an edge map) that map node and edge variable identifiers (originally given in the pattern graph) to nodes and edges in the host.

If this set is empty it means that no subgraphs in the host were isomorphic to the unbound graph. This means the match case has failed. If there are other match cases then they are attempted under the same procedure. If there are not, the result monad `BilboResult` (in `src/Common/Result.fs`) is used to propagate a `MatchError`. The `MatchError` is also used to implement the pipeline modifiers `!` and `?` described in Section 5.4 so no extra infrastructure is requires to manage the way these modifiers deal with failed matches.

The maps in the set are then used to generate graph symbols, shown as the *Bind graph* stage in the diagram. For example, if the symbol `x` in the pattern graph is mapped to a node `"foo"::"bar"` in the host, then the corresponding variable identifier and value pair must be generated so that the variable `x` is bound to the value `"foo"::"bar"` when evaluating the right-hand side. This is done using `UnboundGraph.bind` (in `src/Graph/Graph.fs`). These symbols are then added to the current symbol table, shown as *Add to symbol table*, which uses a stack to push on symbols when a transform is entered, and pop them off after it has been evaluated.

The updated symbol table is then used to evaluate the right-hand side of the match case, which is shown as *Evaluate right-hand side* in the diagram. Unless `$` is used, the process of generating graph symbols, adding these symbols to the symbol table, and evaluating the right-hand side must be done for every pair of maps returned. The output of all the right-hand side evaluations is then returned as a collection. If `$` is used, it must only be done for one pair of maps, the first is chosen to avoid wasted computation.

For simplicity this explanation has left out the `where` clause. This is easily handled. Once the graph symbols have been added to the symbol table, the `where` clause is evaluated. If the evaluation yields `True`, the right-hand side is evaluated as normal. Otherwise the match case fails in the same way that it would if the pattern graph did not match in the first place.

6.1.3 Combing Pattern Graphs

Bilbo allows combing pattern graphs freely with `and`, `or`, and `not` to create expressive pattern searching within the graph. This section describes how this is done in the interpreter. With regards to the diagram in Fig 6.2, this section describes an extra stage between *Subgraph isomorphism* and *Bind graph*.

The `and` operator for combining pattern graphs works exactly the same as graph addition if both graphs are positive (not prefixed with `not`). So, the pattern graph

```
[a,w>,b] and [c,>,d]
```

is equivalent to

```
[a,w>,b] + [c,>,d].
```

The process for generating the mappings in this case is the same as given in the previous section since the pattern graph can be rewritten as a single path

```
[a,w>,b,c,>,d]
```

which can be directly converted into an unbound graph.

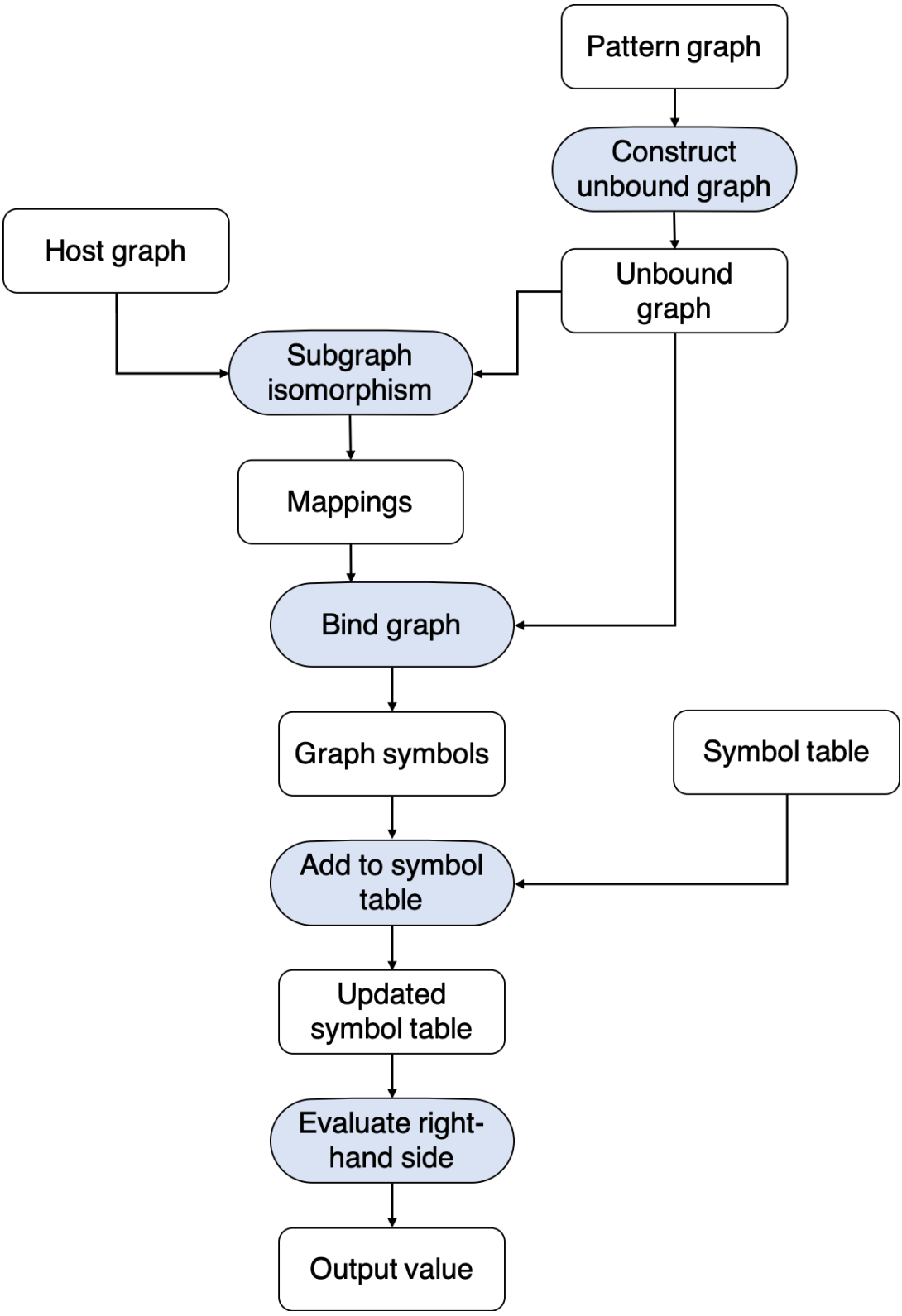


Fig 6.2: The process by which variable identifiers in the patter graph are bound to nodes and edge weights in the host graph. The blue boxes represent function and the white boxes represent inputs and outputs.

If one of the graphs is negative the process is different. Consider the pattern graph

$$P \texttt{ and not } N$$

where, within P and N, `not` is not used. This makes P the positive graph and N the negative graph. The subgraph isomorphism problem is solved independently for the unbound graphs corresponding to P and N. This yields two sets of mappings, a positive set for P and a negative set for N. If the negative set is empty then the whole match is successful, all positive mappings are valid, and the right-hand side can be evaluated for all of the mappings (as detailed in the previous section). If it isn't, the mapping sets must be checked against each other. In particular: for all positive mappings, if there exits a negative mapping that does not contain an inconsistent mapping of the variable identifiers, then such a positive mapping should be disregarded since the negative graph has successfully matched. If any positive graph mappings remain then they are valid, the match is successful, and the right-hand side can be evaluated using those valid matches. An *inconsistent mapping* is one that breaks the unbound variable guarantee explained in Section 5.3.1.

As an example, consider the host graph `g` shown in Fig 6.3 and the pattern graph

`[x,>,y] and not [y,>,z]`.

For convenience the nodes in `g` have been labelled so they can be easily referred to. Furthermore, since there are no unbound variables for edge weights in the pattern graph, the mapping pairs will only contain node mappings. The four positive mappings are

$$\texttt{x} \to \texttt{a} \quad \text{and} \quad \texttt{y} \to \texttt{b} \tag{6.1}$$

$$\texttt{x} \to \texttt{b} \quad \text{and} \quad \texttt{y} \to \texttt{c} \tag{6.2}$$

$$\texttt{x} \to \texttt{d} \quad \text{and} \quad \texttt{y} \to \texttt{e} \tag{6.3}$$

$$\texttt{x} \to \texttt{e} \quad \text{and} \quad \texttt{y} \to \texttt{c} \tag{6.4}$$

and the four negative mappings are

$$\texttt{y} \to \texttt{a} \quad \text{and} \quad \texttt{z} \to \texttt{b} \tag{6.5}$$

$$\texttt{y} \to \texttt{b} \quad \text{and} \quad \texttt{z} \to \texttt{c} \tag{6.6}$$

$$\texttt{y} \to \texttt{d} \quad \text{and} \quad \texttt{z} \to \texttt{e} \tag{6.7}$$

$$\texttt{y} \to \texttt{e} \quad \text{and} \quad \texttt{z} \to \texttt{c} \tag{6.8}$$

where $\alpha \to \beta$ means the variable identifier α in the pattern graph maps to the node β in the host graph.

Since there are more than zero negative mappings, the mappings must be compared. Each positive mapping must be compared against all negative mappings. If, between the positive mapping and negative mapping, the unbound variable guarantee holds, then the negative graph

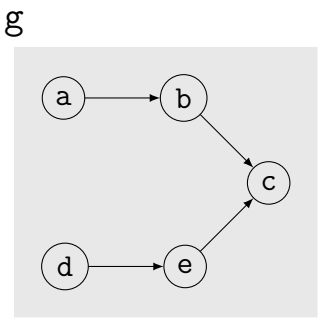


FIG 6.3: The graph `g`.

has successfully matched and so that positive mapping must be disregarded. For example, between the positive mapping (6.1) and the negative mapping (6.6), the unbound variable guarantee holds because

1. `x` is mapped to `a` in (6.1) and nothing else maps to `a`,
2. `y` is mapped to `b` in both (6.1) and (6.6) and nothing else maps to `b`,
3. `z` is mapped to `c` in (6.6) and nothing else maps to `c`.

Therefore the positive mapping (6.1) must be disregarded. Similarly, (6.3) must be disregarded because of (6.8). This leaves two valid positive graph mappings (6.2) and (6.4) for which the graph symbols should be generated, added to the symbol table, and the right-hand side evaluated. Intuitively, this is the right result since `c` is the only node that has no outgoing edges and therefore the only node that `y` can validly map to.

The only operator left is `or`, which is simply handled by taking union of the corresponding mapping sets. For example, in

$$(P_1 \texttt{ or } P_2) \texttt{ and not } (N_1 \texttt{ or } N_2)$$

the overall positive mappings can be found by finding the mappings for P_1 and P_2 and taking the union of these. Similarly for the negative mappings and N_1 and N_2.

6.2 Testing the Interpreter

The Bilbo interpreter and REPL comprises of around 3,500 lines of F# code (within `src/`). The testing suite comprises of a further 2,500 lines (within `test/`) that build on the Expecto library and make heavy use of helper functions (which can found within a `Helpers.fs` file in each of the testing subdirectories). The testing directory `test/` contains three subdirectories:

1. `EvaluatorTests` contains tests related to the evaluator,
2. `GraphTests` contains test for the subgraph isomorphism algorithm, and
3. `ParserTests` contains test regarding AST generation.

As it currently stands there are around 400 unit tests, all of which are passing. The coverage of the evaluator tests is around 65%, as found by the *AltCover* tool [87]. While this number is comparatively low by software engineering standards, this is due to the large amount of code that handles error monads (mostly in `src/Common/Result.fs`) allowing precise and useful error messages, including the line and column number of the erroneous code. This code adds no functionality in terms of language features (it only improves the error messages), and therefore, due to time constraints was not the primary focus of the testing effort, resulting in a smaller coverage. As such, the tests focus on *positive* testing rather than *negative* testing [88].

6.2.1 Testing in the Early Stages of Development

In the early stages of the project, when parsing was the main focus, testing was done to ensure the correct AST was being constructed by the parser.

When development shifted to the evaluator, parser tests were no longer the main focus. This is because the evaluator was tested using whole (but small) Bilbo programs that would first need to be parsed, and then evaluated. Thus an evaluator test is very unlikely to pass if the parser is not working as expected. If an evaluator test was not passing, this would be investigated and a corresponding parser unit test would be added if necessary, to isolate the problem. Due to the good initial testing for the parser, this was rarely needed.

As mentioned previously, program state in the Bilbo interpreter is maintained using a symbol table approach, the code for which can be seen in `src/Common/SymbolTable.fs`. A program can therefore be tested by checking the program state after each statement, which is the smallest program unit in Bilbo (as discussed in Section 5.1.1), has been evaluated. This approach was taken initially to test the evaluator. A one-line program of the form

$$\texttt{a} = E$$

where E is an expression, would be run. To make it as easy as possible to add tests, `a` was always the variable begin bound meaning only the expression E would need to specified. This

was done as an F# string containing the Bilbo expression. The expected value of `a`, as an F# value, was also given alongside the expression string. This meant that tests could be specified easily as a list of pairs, each pair containing the Bilbo program string and the expected value. The program string would be run through the Bilbo parser and evaluator, resulting in a final symbol table. Since the program only contained one-line, the symbol table would only contain one entry, the value of the variable `a`. The testing infrastructure would check that these two values were indeed the same. F# was a great choice in this respect because all types, including user-defined types, have structural equality built in.

This approach was productive in the early stages of development when the main focus was that program state was being correctly maintained. This approach, however, was not scalable since

1. the programs were only one line, and
2. specifying the expected value of `a` became a task in and of itself, as it required specifying an internal Bilbo value as an F# value.

The second point was the inspiration for the testing framework used in the later stages of development. In particular, testing would become much easier if, going forward, the expected value of a particular variable could also be expressed in Bilbo, rather than F#.

6.2.2 Testing in the Later Stages of Development

The focus in the later stages of development was to allow the testing of more substantial features, such as pipelines, pipeline modifiers and reductions but do so in a way that makes it quick and simple to add tests. In particular, expressing Bilbo values as F# values was to be avoided since this is lengthy and error-prone task.

The testing done in the early stages of development provided confidence that program state was being maintained correctly, that is, a value expressed in Bilbo was being stored as the correct F# value in the symbol table. This meant that correctness could be established, with confidence, by specifying the expected value of particular variable as a Bilbo value, rather than an F# value.

Two special variables were used in the Bilbo test programs:

1. `got` would hold a value representative of the whole program. For example, if the program was testing transforms the output of the transform would be assigned to `got`.
2. `exp` would hold the expected value of `got`.

If `got` and `exp` were equal then the test was successful. This allowed more complex tests to be written productively in native Bilbo code.

```
def delNode(g) = match g | [x] => become []

// construct the input graph and find the returned value
a = "a"::10
b = "b"::20
g = [a,b]
got = g >> delNode

// the expected value of got
exp = [a] |&| [b]
```

Listing 6.1: An early test for collections adapted from `test/EvaluatorTetsts/Transform.fs`. The transform `delNode` matches on a single node and removes it.

An example of a test that used this method is shown in Listing 6.1. The testing infrastructure would run the program and obtain the F# values of `got` and `exp` from the final symbol table, upon which the equality comparison would be done. There was further functionality in place, when structural comparison was not enough, that would canonicalise the data structure concerned (such as sorting the list of edges in a graph whose order does not matter for equality purposes), thus allowing structural comparison to be used.

Lastly, when testing nondeterministic functionality, such as the one-match application modifier `$` which chooses a match arbitrarily, it was possible to provide a list of expected value variables (in the same style as `exp`), one of which must match `got`.

6.2.3 Randomised and Property-Based Testing

Randomly generating tests using the FsCheck library was also investigated in two capacities:

1. *Randomly generated transforms.* Transforms would be randomly generated and then properties such as transform associativity (`T1 |> (T2 |> T3)` and `(T1 |> T2) |> T3` are equivalent) would be tested.

2. *Randomly generated graphs.* A graph would be randomly generated and enpiped, for example, through a transform that removes a single node (such as `delNode` in Listing 6.1). A number of properties can be asserted on the output. First, the number of graphs in the output collection is equal to the number of nodes in the original graph. Second, the number of nodes in each of the graphs in the collection have one less node than the original graph.

Unfortunately, due to time constraints, this work is still in development. Going forward, as an open source project, this will be the focus of the testing effort.

Chapter 7

Evaluation

This chapter seeks to evaluate the success of the project. This is done in three ways. Firstly, Bilbo was designed for productivity and expressivity when solving graph problems. In the first section of this chapter, solutions to well known graph problems are presented and discussed. Secondly, the design and results of a usability testing study comparing Bilbo to a Python graph library are presented. DSLs are often said to increase productivity in their intended domains, this study takes a quantitative approach to evaluate if this is the case for Bilbo. Finally, the original requirements set out in Chapter 3 are revisited.

7.1 Solving Graph Problems In Bilbo

The aims for Bilbo were expressivity and productivity in the domain of graph programming. This part of the evaluation shows that Bilbo can be used to solve well-known graph problems intuitively and at a high level of abstraction. In particular, this section presents Bilbo solutions to constructing the transitive closure of a graph, the shortest path problem (with an interesting extension), and the vertex colouring problem. This selection of problems and their solutions are based on [58] and [89].

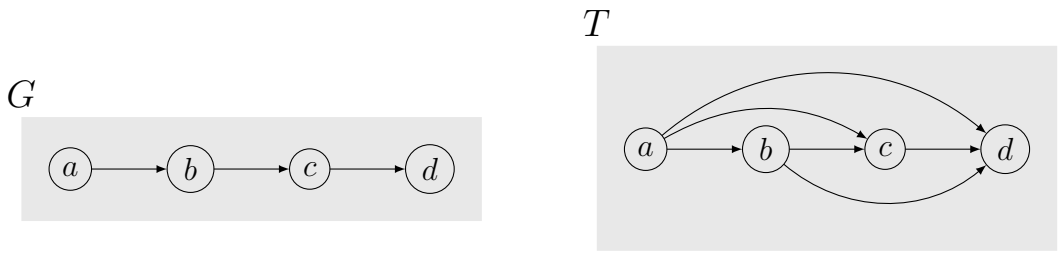
All the Bilbo solutions presented in this section are available in the `examples` folder in the repository.

7.1.1 Constructing the Transitive Closure of a Graph

The transitive closure of a graph is a structure that allows one to answer reachability questions of the form

Is it possible to get from node x to node y?

The transitive closure of a graph G, can itself be seen as a graph T that has an edge from one node to a second if it is possible to get from the first node to the second in G. An example of this is shown in Fig 7.1.



(a) The graph G.

(b) The graph T is the transitive closure of G.

FIG 7.1: The transitive closure of the graph G is the graph T, where T has an edge from one node to a second if it is possible in G to get from the first node to the second.

```
def link(g) =
    match g
    | [a,>,b,>,c] and not [a,>,c] => become [a,>,b,>,c] + [a,>,c]

closure = link!
```

LISTING 7.1: The pipeline `closure` constructs the transitive closure of the enpiped graph.

The operation of finding the transitive closure can be seen intuitively as repeatedly adding an edge between two nodes that are two edges away from each other, as long as the edge being added doesn't already exist. This can be readily expressed in Bilbo as shown in LISTING 7.1. The transform `link` finds nodes `a` and `c` such that two edges must be taken to get from `a` to `c`, and adds an edge directly from `a` to `c`. Since this single operation must be done repeatedly while nodes such as `a` and `c` exist, this operation is applied as long as possible using `!`, creating the pipeline `closure`. Whenever using the `!` operator, a programmer should be sure that the pipeline to which it is applied will terminate. In this case, termination is obvious because, eventually, all the edges between reachable nodes will have been added. When this is the case, the pipeline will terminate and the transitive closure is, almost by definition, remaining. The pattern for solving problems with as-long-as-possible application, discussed in SECTION 5.4.2, is used here. The defining characteristic of the transitive closure graph is that it *does not contain* the subgraph

`[a, >, b, >, c] and not [a, >, c]`.

Indeed this graph, expressing the defining characteristic, is the left-hand side graph of `link` and is applied as-long-as-possible, as described in SECTION 5.4.2 .

Two comments can be made about the program in LISTING 7.1.

```
def link(g) =
    match g
    | [a,>,b,>,c] and not [a,>,c] => become [+] + [-]

closure = $link!
```

Listing 7.2: An improved version of the transitive closure program shown in Listing 7.1. This version uses one-match and as-long-as-possible application.

Firstly, the transitive closure of a graph is unique and, moreover, the operation of applying `link` as long as possible is confluent. This means that the collections maintained in the intermediate stages, before `link!` terminates, are not needed because upon termination they will all yield the same graph. To avoid the wasted computation, `$` should be used. This is, in fact, the motivation for the `$` operator.

Secondly, the right-hand side of the match case in `link` is the sum of the positive graph `[a,>,b,>,c]` and the negative graph `[a,>,c]`. Furthermore, all the unbound variables that appear in the negative graph also appear in the positive graph, meaning that, when the right-hand side is evaluated, they will have a bound value. Therefore, the special variables `[+]` and `[-]` can be used. The rewritten version of `closure`, with these two improvements, is shown in Listing 7.2.

For completeness, this is now taken further and we use the transitive closure to answer the reachability question mentioned above. Given a graph `g`, the reachability question can be answered for any two nodes `x` and `y` using the pipeline `canReach` shown in Listing 7.3. This program also shows how partial application of pipelines can be used. `canReach` is created by enpiping only one argument, the transitive closure graph `t`, into `reachable`. This makes `canReach` a pipeline that accepts two arguments and returns `True` if there is a path, in the original graph `g`, from the node with an identifier equal to the first argument to the node with an identifier equal to the second argument. An example application of `canReach` is shown on line 11. We also note that since graphs are completely serialisable, the transitive closure could be printed to a file and imported from another Bilbo program.

```
def reachable(t,x,y) =
      match t
      | [a,>,b] where (&a==x) and (&b==y) => return True
      | _ => return False

t = g >> closure

canReach = t >> reachable

// example usage
print ("m", "n") >> canReach
```

Listing 7.3: The reachability question for any two nodes can be answered by enpiping their identifiers into `canReach`. This program will print `True` if the node with identifier `"n"` is reachable from the node with identifier `"m"` in the graph `g`. This program uses the `closure` pipeline defined in Listing 7.2.

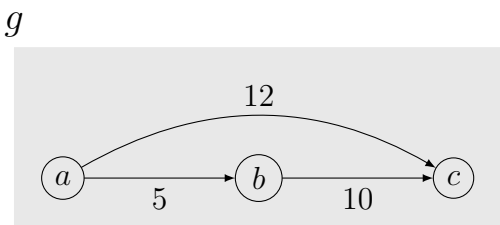


Fig 7.2: The sum of the edge weights from a to c is greater if the direct edge is taken, rather than going via b.

7.1.2 The Shortest Path Problem

The shortest path problem for graphs is to find the shortest distance from one node to another in a given graph, where the distance is the sum of the weights for the edges traversed between the two nodes. This problem is of great importance, both in graph theory and in its application, which is diverse. For example, it is used to design transportation networks, guide autonomous vehicles, and even in VLSI design for wire layout [90].

Using Bilbo, an extension of the problem will be solved. Which requires finding the shortest distance between a designated start node and all other nodes, individually.

When a shortest path problem is solved visually by a person, given a pictured graph, one heuristic they may use is to search for patterns of three nodes a, b, and c where the distance from a to c directly is larger than the distance of a to c via b. Such a situation is shown in the graph g in Fig 7.2. Solving the problem in Bilbo uses a similar intuition.

```
type Place = set, dist

def add(g) =
    match g
    | [x,w>,y] where x..set and not y..set =>
        y..set = True
        y..dist = x..dist + w
        become [x,w>,y]

def reduce(g) =
    match g
    | [x,w>,y] where y..dist > x..dist+w =>
        y..dist = x..dist + w
        become [x,w>,y]

shortest = $add! |> $reduce!
```

Listing 7.4: The pipeline `shortest` sets `dist` on every node to the shortest distance from the start node to that node.

The program in Listing 7.4 defines a pipeline `shortest` that, given a graph with node loads of type `Place` and positive (`int` or `float`) edge weights, sets the attribute `dist` on every node to the shortest distance required to reach it from the start node.

The designated start node is the node with `set` equal to `True` before enpiping into `shortest`. It should also have `dist` equal to zero. All other nodes should have `set` equal to `False`. The pipeline `$add!` simply sets the initial value of `dist` on every node to a valid distance from the start node. Since the node `x` (in the pattern graph of `add`) requires `set` to be `True`, this will *spread out* from the start node, setting `dist` to the sum of the edge weight and the valid `dist` value of the node on the other (source) side of the edge. The pipeline `$add!` terminates when every node has `set` equal to `True`, and therefore every node has valid `dist` value.

The transform `reduce` captures the intuition mentioned above. It finds instances where a node `y` has a `dist` value larger than the sum of the `dist` value of a node `x` and the weight `w` of an edge from `x` to `y`. This means that the `dist` value of `y` can be lowered, and remain valid, by setting it equal to the the sum of `x..dist` and `w`. The pipeline `$reduce!` does the repeatedly and so it terminates when no `dist` values can be lowered. This means that, upon termination, every node has a `dist` value equal to the shortest distance from the start node to it. This is the solution to the shortest path problem for all nodes in the graph.

The operator `$` is also used since the solution is unique and at no point will will the algorithm make a *guess* which may be incorrect, whenever a `dist` value is reduced, it is reduced to a valid value. Therefore, there is no need to generate all graphs at any stage in the algorithm.

For completeness the solution is generalised further. Using `shortest` requires the graph to have loads of type `Place` and the start node to have `set` equal to `True` and `dist` equal to 0. Furthermore, the resulting graph keeps the original edges, which are not needed since only the shortest distances are sought after and the original graph (with the edges) will still be available.

The program in Listing 7.5 uses a transform `init` to initialise the nodes. All nodes, apart from the designated start node, will have a load of type `Place` with `dist` equal to `0` and `set` equal to `False`. The start node is chosen by passing in `startId` which should be equal to the identifier of the desired start node. The start node has `set` equal to `True`. When a transform with multiple parameters is applied as long as possible, for example `init`, the first argument is updated to the previous output, while the other arguments remain the same. So, in every iteration of `init`, the `startId` will be the same. This behaviour is intuitive and means the program works as expected. The transform `removeEdge` is then used to remove all edges so only the nodes, with their shortest distances, are returned. On line 16 of Listing 7.5 there is an example usage, where `g` is a graph.

```
def init(g,startId) =
    match g
    | [n]
        where (&n == startId)
        and not (#n is Place) => become [n::Place(True, 0)]
    | [n] where not (#n is Place) => become [n::Place(False, 0)]


def removeEdge(g) =
    match g
    | [x,<w>,y] => become [x,y]
    | [x,w>,y] => become [x,y]

// example usage for a graph g
// node with identifier "a" is the start node
g' = (g, "a") >> $init!
dists = g' >> shortest |> $removeEdge!
```

Listing 7.5: This program is generalised version of Listing 7.4 and uses the pipeline `shortest` defined there. It returns a graph containing the nodes of `g` where each graph has a load with `dist` set to the shortest distance from the start node to it. In this case, the start nodes is chosen as the node in `g` with identifier `"a"`.

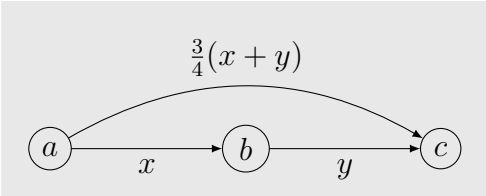


Fig 7.3: The bridge is an edge from a to c with distance equal to three-quarters the sum of distances from a to b and b to c.

7.1.3 Where to Build a Bridge?

Using the solution to the shortest path problem, a more difficult question is now answered. Consider the scenario of building a bridge from one location a, over a second location b, to a third location c. The bridge will make the distance from a to c three-quarters the current distance required between a and c via b. This is shown in Fig 7.3.

Consider further the question of where to build this bridge, given that it can be built for any three nodes a, b and c but only one bridge can be built. The aim is to minimise the total distance from a designated start node to a designated finish node, so the bridge should be built such that this distance is minimised.

The solution to this problem in Bilbo is shown in Listing 7.6. Firstly, the addition of the bridge is naturally represented as a transform that matches on three nodes and the edges between them and constructs the bridge according to the weights. This is done in the transform `addBridge`. Without loss of generality, let the input graph be `g` and the desired start and finish locations be nodes with identifiers `"a"` and `"z"` respectively. For ease, these two nodes will now be referred to as `"a"` and `"z"`. Furthermore, recall the type `Place` and the transforms `shortest` and `init` from the shortest path programs in Listing 7.4 and Listing 7.5.

On line 8, the transform `findDistToNode` simply returns the `dist` value of the node with identifier equal to the parameter `fin`. So the transform `distToZ`, on line 18, returns the `dist` value of the node `"z"`, the designated finish node.

The transform `init` is used on line 15 to initialise all nodes so the loads are of type `Place`, similar to what was done in Listing 7.5. `"a"` is also specified as the start location. This is assigned to the graph `g'`.

We now focus on lines 21 to 25. `g'` is first enpiped into `addBridge`. Notice that no `$` is used, this will therefore create a collection of graphs where each graph will have one bridge added in a particular location, and the collection will contain such a graph for all locations that a bridge

```
def addBridge(g) =
    match g
    | [a,x>,b,y>,c] =>
        w = 0.75*(x+y)
        bridge = [a,w>,c]
        become [+] + bridge

def findDistToNode(fin, g) =
    match g
    | [x] where &x == fin => return x..dist


// w.l.o.g let g be the input graph
//  and the node with identifier "a" be the start
g' = (g, "a") >> $init!

// let the node with identifier "z" be the finish
distToZ = "z" >> findDistToNode


bestG =
    g'
    >> addBridge
    |> shortest
    |> min by distToZ
```

Listing 7.6: The graph `bestG` is the graph with a bridge added in the location that minimises the distance from the node with identifier `"a"` to the node with identifier `"z"`.

can be added. This collection is then passed to `shortest` from Listing 7.5. For all graphs, this will set the `dist` value for all nodes to the shortest distance from the start location, `"a"`, to that node.

The solution to the problem is the graph in this collection that minimises the distance from the node with identifier `"a"` to the node with identifier `"z"`. This distance, per graph, is the `dist` value on `"z"`. Recall that `distToZ` returns the `dist` value of `"z"`. So this collection is then passed to a `min`-reduction that returns the graph which has the lowest `dist` value on `"z"`. This mean that the graph `bestG` is the graph that contains the bridge added in the location that minimises the distance from `"a"` to `"z"`. This is the desired solution.

The bridge itself could easily be returned by marking it when it was added and enpiping `bestG` into a transform that returns only the bridge. Furthermore, the fact that three-quarters was used as the scaling factor for the bridge distance is arbitrary, and this could be parameterised for any factor and even added as an argument to `addBridge`.

It is important to notice that this solution utilises many of Bilbo's features

1. The operators `!` and `$` are used in `shortest`.
2. The default behaviour, to return a collections where all matches are evaluated, is used to get all possible places a bridge can be added.
3. The ability to enpipe a collection of graphs into a transform, outputting a collection is used with `shortest`.
4. Partial application of transforms is used to create the transform `distToZ`.
5. A `min`-reduction is used to find the best graph in the collection according to the `dist` value of the finish node.

Reductions and one-match application using `$` are completely novel features, not available in any rewriting tool, without which this question could not be answered. The operator `$` allows controlling the inherent nondeterminism in graph rewriting, while reductions allow taking advantage of that nondeterminism, as was done by adding a bridge in all locations and then finding the best location.

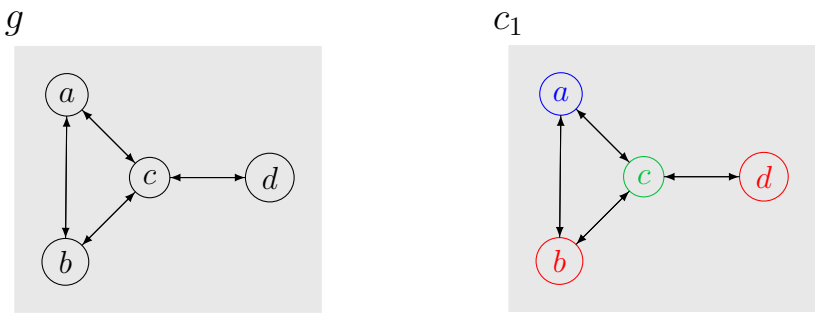


FIG 7.4: The graph g and one its graph colourings c_1.

7.1.4 The Vertex Colouring Problem

The vertex colouring problem is, given a graph g, assign a colour to each node such that no two adjacent nodes (nodes that have an edge between them in either direction), are assigned the same colour. For example, a graph g and one of its colourings c_1 is shown in FIG 7.4. For a given graph, there may exist multiple colourings, for example the graph g in FIG 7.4 has twelve distinct colourings. Furthermore, computing the minimum number of colours required to colour a given graph is a known NP-complete problem [91]. Graph colouring has applications in register allocation for optimising compilers [92] and even the game Sudoku can be solved as an extension of a graph colouring problem [93]. The graph colouring problem is now solved using Bilbo.

The problem is again approached using some intuition. Simply, if two adjacent graphs have the same colour, one of their colours must be updated. This is what the transform `changeCol` does in LISTING 7.7. The pipeline `vcolour` first initialises the graph, using `$init!`, so every node has a load of type `Vertex` and given a colour `0` (integers are used to represent colours). `changeCol` is then used to keep changing the colours of adjacent nodes that have the same colour. Once this no longer matches, it means that no two adjacent nodes have the same colour, which is exactly the solution of the vertex colouring problem. We see again (as was seen in the transitive closure problem) that the solution to graph problems in Bilbo is based precisely on the definition of the problem. This is a result of working at the level of abstraction suited to graphs and leveraging a model of computation designed for graphs. Furthermore, the pattern for solving problems with as-long-as-possible application, discussed in SECTION 5.4.2, is again utilised. The defining characteristic of a valid colouring is that no two adjacent nodes have the same colour. In other words, the solution *does not contain* two such nodes with an edge between them, which is exactly what the left-hand side graph of `changeCol` is.

Finally, enpiping a graph into `vcolour` will yield a valid vertex colouring. While this solution is for graphs with unweighted edges, it can very easily be extended for both weighted and unweighted edges by adding a second match case to `changeCol`.

```
type Vertex = colour

def init(g) = match g | [a] where not (#a is Vertex) =>
    become [a::Vertex(0)]

def changeCol(g) =
    match g
    | [a,>,b] where (a..colour == b..colour) =>
        b..colour = b..colour + 1
        become [a,>,b]

vcolour = $init! |> $changeCol!
```

Listing 7.7: The pipeline `colour` will give each node a colour, solving the vertex colouring problem.

7.1.5 Summary

This chapter has shown that Bilbo makes it possible to solve graph problems at a high level of abstraction. Constructing the transitive closure and solving the vertex colouring problem followed from the very definition of each problem. The shortest path problem was solved using obvious intuition. Problems can be solved in this way, without requiring knowledge of particular algorithms or data structures, as a result of Bilbo's model of computation which allows graph operations to be expressed precisely and easily. The problem of where to build a bridge showed how Bilbo's completely novel constructs for first-match application (`$`) and reductions can be used to control and take advantage of nondeterminism to answer a difficult question in a simple way. As a result, we conclude that Bilbo is a productive and expressive tool for solving graph problems.

7.2 Usability Testing

In order to evaluate Bilbo's expressivity and productivity, usability testing was done through a worksheet-based empirical study. Evaluating the productivity of a DSL was done in [57] and [94] by comparing, through worksheets, the DSL with a software library that provides similar functionality. Since Bilbo is a DSL with a design objective of productivity, this was seen as an appropriate method for evaluation.

The study compares Bilbo against the Python library ReGraph [48], involved ten participants and took place between March and May 2019. The aim of the study is to evaluate the extent to which Bilbo makes it easier to do graph-related tasks, as a measure of productivity.

7.2.1 Tool for Comparison

The decision to use ReGraph for comparison was based on a number of reasons. Firstly, ReGraph provides library functions for graph rewriting. This makes the languages more comparable since Bilbo's model of computation is based on graph rewriting. Secondly, ReGraph is a Python library and so it benefits from Python's readability [95], setting the bar high. While readability isn't an explicit design aim of Bilbo, there is of course an interplay between readability, expressivity, and productivity. Lastly, Python's popularity as a scripting language, as well as the fact that it is often taught to science and engineering students, meant that all the participants were already confident Python users. This means that the results would not be skewed in favour of Bilbo, due to lack of Python knowledge. The validity of results is discussed in more detail in Section 7.2.6.

7.2.2 Participants

Ten students were approached regarding the study. In [96] it is recommended to recruit widely and not have a participant archetype in mind. All participants were students at Imperial College London but came from a diverse range of technical backgrounds. Participants were a mixture of undergraduate and postgraduate students and were from the mathematics, mechanical engineering, and electronic engineering departments. Furthermore, some students had over a year of industry experience while others had done research placements and were starting PhD programmes.

The use of students in software-related studies is widely recognised [97]. The framework laid out in [98] for having student participants was followed where possible. In particular, it was decided that the worksheet should be take-home rather than at a prescribed time and date. This provided participants with necessary flexibility, and meant that all the participants that were approached took part in the study.

Please indicate the time that you are starting this part of the worksheet.

(a) The reminder to start timing.

Please indicate the time that you finished this part of the worksheet.

(b) The reminder to stop timing.

Fig 7.5: The reminders to start and stop timing which appeared at the start and end of both the Bilbo questions and the ReGraph questions.

7.2.3 Experimental Design

The study was delivered as three PDF documents:

1. The worksheet, which can be seen in Appendix A
2. A Bilbo tutorial, Appendix D
3. A ReGraph tutorial, Appendix E

All documents were written by the author.

The worksheet was split into three parts. The first part contained eight questions on Bilbo, similarly for the second part on ReGraph, and the third part was a feedback questionnaire.

The tutorials contained everything required to answer the corresponding questions in the worksheet. This was checked independently by an eleventh participant as a pilot, in the style of an external examiner, who did the worksheet and was asked for detailed feedback. No major changes were made to the tutorials or worksheet as a result of this. The only changes were regarding the phrasing of some questions. The results of the eleventh participant do not contribute to the final results.

While the Bilbo tutorial is longer, because the language had to be explained assuming no prior knowledge and the ReGraph tutorial concentrated only on ReGraph features and not Python itself, they contained equivalent information.

As can be seen in the instructions in Appendix A, participants were asked to time themselves while doing the first two parts, timing each part separately. There were reminders throughout the worksheet that are shown in Fig 7.5. The instructions also remind participants that they should have read through both tutorials and settled any queries they had with the author before

starting the questions.

Participants were asked to answer the questions on paper, rather than on a computer. This was done to remove the potential effect on productivity from development environments and other tooling, which, due to the infancy of Bilbo, would favour ReGraph and Python.

7.2.4 The Questions

The parts of the worksheet on Bilbo and ReGraph each contained eight questions based on the style given in [94]. The questions concentrate on graph construction and manipulation which would be fundamental to any graph problem. There were no questions on graph rewriting (and therefore Bilbo transforms and the corresponding feature in ReGraph) as this has a large amount of prerequisite material relating to pushouts and nondeterminism. This would have required a significantly larger time investment from participants and so may have reduced the number of participants that took part in the study.

To ensure consistency between the Bilbo and ReGraph questions, the template, shown in Appendix C, was created before the questions themselves were written. Questions 1 to 5 were multiple choice, with three choices and only one correct answer, and questions 6 to 8 required written code as answers.

The feedback questionnaire, the third part of the worksheet, contained thirteen questions. The first two questions were used to ensure that the participants did not feel disadvantaged by any lack of Python knowledge, or having not been exposed to graphs before. All remaining questions were answered on an interval scale between 1 to 5 inclusive. The following nine questions asked about the perceived difficulty of the questions, as well as how intuitive, productive and readable each of the tools were. Participants were also asked about their previous experience with ReGraph. The final two questions asked how well each tool *captured the domain* of graph programming. An explanation of what it meant to capture a domain well was also given. Participants were invited to leave extended feedback for any question alongside their 1 to 5 answer.

In line with GDPR, all results in raw form will be deleted in August 2019 [99]. Participants were also asked to consent to their data being used as part of this project. All participants were happy to do this.

7.2.5 Results and Statistical Analysis

Participants returned their answers, either as a physical copy or electronically. They were marked by the author. With regards to the Bilbo and ReGraph questions, all participants answered correctly to all questions. It is expected that this is a result of not capping the time, allowing each participant to take as long as they need to complete the questions. Furthermore,

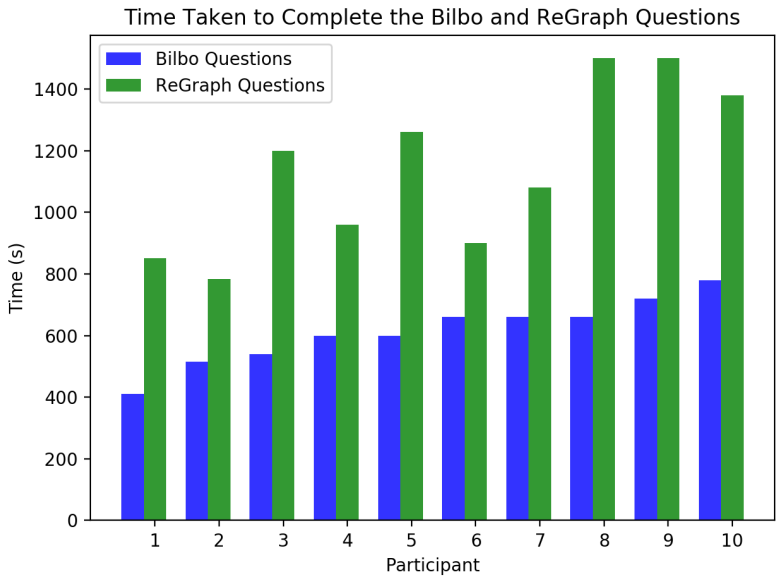


FIG 7.6: The time taken by each participant to complete the Bilbo and ReGraph questions.

all participants, through their answers to the first two feedback questions, indicated that they did not feel disadvantaged because of their level of familiarity with Python, or lack of exposure to graph concepts. This means that the background of the participants does not pose a threat to the validity of conclusions drawn from the results. Since all answers were correct, the analysis focuses on the time taken to complete the Bilbo and ReGraph questions as well as the answers to the feedback questionnaire.

The table in APPENDIX B contains the worksheet results. Participants have been anonymised but, for identification purposes, have been given a number between 1 and 10 according to how long it took them to answer the Bilbo part of the worksheet. Participant number 1, therefore, completed the Bilbo questions in the shortest amount of time.

Time Taken

The times taken by each participant to complete the Bilbo and ReGraph questions are shown in FIG 7.6. The (arithmetic) mean and standard deviation of the time taken is also shown in TABLE 7.1. The mean time for ReGraph is almost double (1.86×) the mean Bilbo time, while the standard deviation of the ReGraph times is more than double (2.50×) that of the Bilbo times. It should also be noted that the slowest Bilbo time (participant 10) is faster than the fastest ReGraph time (participant 2). From this, it is clear that answering the questions using Bilbo was faster, and the variability of the Bilbo times was much smaller. The large variance in ReGraph times could be related to some participants having more experience with Python (though no participants indicated that they felt disadvantaged due their own Python experience). If this was the case, it seemed to have much less of an effect when it came to the Bilbo times.

Questions	Mean Time (s)	Standard Deviation (s)
Bilbo	614.7	106.4
ReGraph	1141.4	266.5

TABLE 7.1: The mean and standard deviation of the time taken for the Bilbo and ReGraph questions.

Feedback Questionnaire

Other than the background questions (regarding Python and exposure to graphs), the feedback questionnaire asked each participant how intuitive, productive and readable Bilbo and ReGraph were. Participants were also asked how well each of these tools captured the domain of graph programming. The difficulty of the questions, which participants were also asked to rate, is discussed in SECTION 7.2.6.

Definitions were not given for *intuitive*, *productive*, and *readable* as the aim for these questions was to get natural and instinctive responses. Since all of the participants were confident programmers (as evidenced by the Python background question), it was assumed that they all had a perception of these properties. Capturing the domain was described as follows:

A tool is said to capture a domain well if, when using that tool, it is straightforward to express the important ideas in that domain. It is a measure of how effective a tool is in a given problem domain. For example, one may say that MATLAB captures the domain of linear algebra well since it is easy to express vector and matrix operations.

Since the values are given on a 1 to 5 interval scale, the arithmetic mean and standard deviation are both valid operations. The summary of the answers to these question can be seen in TABLE 7.2. In all four properties, Bilbo has a higher mean. With regards to the standard deviation, Bilbo has smaller standard deviation in all properties except readability and capturing the domain. For readability Bilbo received four 5s and six 4s while ReGraph received eight 2s and two 1s. In other words, there was very low variation in ReGraphs's readability, which was deemed to be low, while the higher standard deviation for Bilbo was between the two highest scores. The higher standard deviation regarding capturing the domain is also due to this. From these results, it is clear that participants felt Bilbo was more intuitive, productive and readable than ReGraph. Participants also felt that Bilbo captured the domain of graph programming better than ReGraph did.

Property	Bilbo		ReGraph	
	Mean	Standard Deviation	Mean	Standard Deviation
Intuitive	4.3	0.48	2.4	0.84
Productive	4.4	0.52	2.2	0.79
Readable	4.4	0.52	1.8	0.42
Captures the domain	4.5	0.53	2.3	0.48

Table 7.2: The mean and standard deviation of the 1 to 5 scores rating Bilbo and ReGraph on how intuitive, productive, and readable they are, as well as how well they capture the domain of graph programming.

Questions	Mean	Standard Deviation
Bilbo	1.6	0.70
ReGraph	2.5	0.71

Table 7.3: The mean and standard deviation of the perceived difficulty of the Bilbo and ReGraph questions.

7.2.6 Threats to Validity

This section addresses potential threats to the validity of conclusions drawn based on the results. In particular it addresses the issue of whether the Bilbo and ReGraph questions were of equal difficulty. If the questions for one of the tools were indeed easier, this may have meant participants completed those questions faster and felt that the corresponding tool was more productive.

To gauge this, within the feedback questionnaire, participants were asked to rate the difficulty of each part of the worksheet on the 1 to 5 scale. The mean and standard deviation of the answers are given in Table 7.3. These suggest that participants did feel that the Bilbo questions were easier to answer. No definition of *difficulty* was given in the feedback questionnaire. This may have allowed two potential interpretations:

1. *Intrinsic difficulty.* The difficulty of a question, regardless of the tool being used to answer it.

2. *Answering difficulty.* The difficulty in answering a question with a given tool.

It was chosen not to define difficulty (which may have avoided multiple interpretations) since the intrinsic difficulty and answering difficulty can be hard to separate. In hind sight this was a bad choice and a definition should have been given.

Question	Bilbo	ReGraph
1	5	5
2	10	10
3	15	14
4	1	1
5	8	9
6	16	15
7	16	15
8	28	25

TABLE 7.4: The number of components (total number of nodes and edged) in the correct answer for each of the Bilbo and ReGraph question.

While a definition of difficulty was not given, the issue of question difficulty was actually taken into account when writing the questions themselves. In [94], a question's difficulty is related to the number of components involved its answer. A component should be something at the level of abstraction of the problem (something related to graphs) rather than something found in a particular solution (such as the number of lines of code). A a component was defined as a node or edge. A question's difficulty was therefore seen as the number of nodes or edges involved in that question's correct answer. To ensure that the difficulty would not skew the results in favour of Bilbo, in all but one, a ReGraph question had equal or fewer components than the corresponding Bilbo question. The number of components in each question is shown in TABLE 7.4. Only question 5 has more components for ReGraph, while four Bilbo questions have more components than the corresponding ReGraph question.

As a result of this, it was concluded that the lower difficulty perceived in the Bilbo questions was in fact due to the ease in answering those questions using Bilbo and the relative difficulty in answering similar questions using ReGraph. Therefore the apparent discrepancy in difficulty does not threaten the validity of the results.

7.2.7 Participant Feedback

A number of participant did choose to leave feedback regarding Bilbo which was almost all positive. The notation for edges and addition between graphs was said to be intuitive. One participant commented that the clarity of Bilbo's syntax and semantics meant that, while doing the worksheet, the Bilbo tutorial was seldom referenced while the ReGraph tutorial was referenced multiple times, despite being a confident Python programmer. Another user said that the syntax is *fantastically concise* and *completely clear*.

The two pieces of negative feedback received were both regarding commas in path expressions.

In particular, it was proposed that they could be removed. For example,

`[a > b > c]`

instead of

`[a, >, b, >,c]`.

The commas were originally intended to provide clarity, especially in the case of objects being used as loads and weights which would make the path expression more crowded. They also provide the familiarity of comma-separated data structures, rather than space-separated data structures, which are rarer. Going forward, the use of commas in path expressions, and whether they do provide clarity, will be evaluated.

7.2.8 Summary

The usability testing concentrated on comparing Bilbo to ReGraph. Through a worksheet-based study the two tools were compared with regards to how intuitive, productive, and readable each they are, as well as how quickly simple questions can be answered using them. Bilbo beats ReGraph in all criteria and often by some margin. Participants gave mostly positive feedback regarding Bilbo and changes in the language as a result of the constructive feedback are being considered. Confidence can be drawn from the care and rigour taken to make the study fair. As a result, we can conclude with confidence that Bilbo succeeds in being an expressive and productive tool for graph programming.

7.3 Revisiting the Requirements

In this section, as a final method of evaluation the, requirements set out in Chapter 3 are revised. The current state of the project is compared against those requirements in Table 7.5.

7.3.1 Summary

From Table 7.5, it can be seen that the original requirements set out have mostly been achieved. The shorthand variables for the positive and negative graphs, `[+]` and `[-]` respectively, are the only feature that was designed but not implemented in the interpreter. Overall this shows that the project has succeeded in fulfilling the original requirements. Nevertheless, there remains areas for improvement within the Bilbo language that are detailed in the conclusion.

Requirement	Outcome
System-Level Requirements	
Ability to input and output graphs	Graphs are fully serialisable and Bilbo files can be imported
Ability to import functionality	Bilbo files can be imported in a module-like way
One language	Bilbo is one language with consistent syntax for graph construction and manipulation
Type-System and Program State Requirements	
Mutable variable bindings	A symbol table is used for mutable variable bindings
User-defined object types	User-defined types can be created easily and concisely
Graph-Specific Requirements	
Declarative syntax for graphs	Paths are completely declarative, the underlying representation of the graph is not exposed
Graph rewriting model of computation	The single pushout with isomorphic matching is used
Programatic control of nondeterminism	Collections, as-long-as-possible application and one-match application allow controlling nondeterminism
Computational completeness	Bilbo is computationally complete
Reference Implementation Requirements	
Interpreter provides good portion of designed functionality	The interpreter implements all the designed functionality except the shorthand variables for the positive and negative graph
Well-tested codebase	The codebase has reasonable test coverage

Table 7.5: A comparison of the original requirements set out for the project and the corresponding outcome within the project in its final state.

Chapter 8

Conclusion

Graphs are ubiquitous in science and engineering. Yet, few tools exist that concentrate on programming with graphs in a productive way. Existing tools either concentrate solely on performance and thus requiring the use of low-level constructs, or require maintaining program state as a single graph, which is uncommon outside of graph rewriting communities.

Many languages contain constructs for productivity in their intended domains. F# has easy to use monads to write safe code that handles exceptional behaviour in a typed way, Haskell has powerful list processing capabilities due to their popularity in functional programming, and C++ has templates for statically typed metaprogramming. This work presented Bilbo. Bilbo follows this trend by providing a host of constructs aimed at increasing expressivity and productivity when programming with graphs.

8.1 Achievements

Bilbo achieves the two main design objectives of expressivity and productivity. The evaluation showed that well-known graph problems can be solved intuitively using Bilbo. The usability testing showed that the syntax and semantics of Bilbo are expressive and capture the domain well. Finally, the initial requirements set out have all been achieved.

Paths are an intuitive way to express chains of nodes and edges, and features such as path comprehensions and nested paths allow exploiting regularity when constructing graphs. Bilbo's graph addition allow building graphs incrementally while the novel definition of graph subtraction allows precisely removing nodes or edges with a single operator. In Bilbo, graph interactions are at a high level of abstraction.

The ability to read other Bilbo files allows creating a library-like collection of functionality that can be saved in a file and used in a modular way by other programs. It also means input graphs can be defined once and imported by each program. Furthermore, Bilbo graphs and collections

are fully serialisable as canonical Bilbo code and so the solution to a graph problem can be written to file and used by other Bilbo programs or easily converted to another structured data format.

Transforms allow operations on graphs to be expressed at the domain level of abstraction, as simply finding a subgraph and transforming it. In the evaluation it was shown that solutions to graph problems can be readily expressed using transforms. In the case of the transitive closure and vertex colouring, the Bilbo solution followed almost immediately from the problem statements. Similarly, solving the shortest path problem required only some simple intuition. At no point was knowledge of algorithms or data structures required, showing that Bilbo is productive tool for graph programming at the correct level of abstraction.

Bilbo also contains a number of novel features such as the ability to combine pattern graphs, collections, the one-match operator, and reductions. These make Bilbo more productive in graph programming. Pattern graphs can be combined using `and`, `or`, and `not` to express complex pattern searches using familiar the logical operators. This extends the negative application conditions offered by existing graph rewriting tools. Bilbo's concept of collections make the non-deterministic nature of graph rewriting straightforward. Collections are well integrated across all language features, so working with a collection is just like working with a single graph. The one-match operator `$` allows controlling the nondeterminism and is especially useful for confluent operations. No existing graph rewriting tools allow taking advantage of confluence in programmatic way. Reductions, also novel in graph languages, go beyond computational completeness of graph rewriting tools. They allow aggregating or filtering over a collection, meaning that when all matches are used, the best match can be found. This feature can be used to solve non-trivial problems such as how best to change a graph according to some desired goal, as was done when solving the bridge problem in Section 7.1.3.

In the usability testing, it was found that solving simple graph problems took almost twice as long in a Python library, when compared to Bilbo. Bilbo's syntax and semantics were also seen as highly intuitive, productive, and readable. In all metrics, Bilbo beat the competitor.

Alongside the design and evaluation of Bilbo, this project saw the implementation of an interpreter and REPL. These are both in a very usable state with good test coverage. The interpreter was used to run and check all programs in this report, including those from the evaluation in Section 7.1.

8.2 Future Work

The work completed within the time frame of the project is very promising. As such, the design and development of Bilbo as well as the interpreter is being continued as an open source project,

on the MIT license. The further work planned thus far falls into two categories: the open source development of the project, and planned language features. These are discussed below.

The author is also in the process of preparing a paper for submission to GCM 2020[1], which welcomes papers on *graph-based programming models*.

8.2.1 The Open Source Development of Bilbo

Property-Based testing

The property-based testing discussed in Section 6.2.3 will be the concentration of the development effort in the short-term. This will not only increase the coverage of the tests, it will also streamline the process of testing new features as the language grows.

Tooling

While Bilbo has found success as a fully textual language, being able to visualise output graphs would be helpful for large graphs with objects as weights and loads. This would likely take shape as an F# tool that converts Bilbo's graph output to JSON, making it readily usable in a graph visualisation framework such as CytoScape [80].

A syntax highlighting package for the prominent text editors, such as Visual Studio Code, Atom, and Sublime, would also help with writing Bilbo programs. Since these text editors all use TextMate grammars [100] for syntax highlighting, a single package could be created for use in all three.

Language Reference

Due to the time constraints of the project, no formal language reference was created. This is a large piece of work but essential to allowing the adoption of the language amongst relevant communities. Going forward a language reference for Bilbo will be developed. This will likely be done using the popular structural operational semantics [101]. It was shown in [58] that this method can be used to describe graph rewriting.

The testing, tooling, and language reference will all be done within the public domain to generate interest in the project.

8.2.2 Future Language Features

A number of future language features are currently under consideration, as well as the feedback from the usability testing study. Two are discussed below, and a third is mentioned in the limitations.

[1] The call for papers for the 2019 conference was on the 7th of May 2019. The webpage is `http://gcm2019.imag.fr`.

Value-Based Pattern Graphs

Pattern graphs currently only allow variable identifiers in the pattern. Any value-based requirement on the nodes or edge weights that are matched must be expressed in the `where` clause, for example

```
match g | [x] where (&x == "Earl's Court") => ....
```

This match case may be more readable as

```
match g | ["Earl's Court"::y] => ...
```

where `y` would bind to the load of the node. This would also mean that the check on the identifier (or load) of the node could take place as part of the subgraph isomorphism algorithm, rather than after the match has been generated, when the `where` clause is evaluated. A similar syntax could be used for specifying the value of edge weights. The precise semantics of this and how it would affect the unbound variable guarantee are currently under consideration.

Pipeline Comprehensions

Pipeline comprehensions, like path comprehensions, will aim to allow taking advantage of regularity in pipelines. A number of the programs in Chapter 5 and Chapter 7 contained a sequence of transforms in a pipeline, all with the same modifiers, for example

```
$t1! |> $t2! |> $t3!.
```

Pipeline comprehensions will allow expressing this, more concisely, as

```
{$,!: t1, t2, t3}.
```

8.3 Limitations

While this project has not concentrated on performance, one of the main limitations of Bilbo, and graph transformation as whole, is the inherent complexity in the operations required. Graph rewriting depends on solutions to the subgraph isomorphism problem which is NP-complete, and so has exponential time complexity. Bilbo currently uses a simplified version of Ullman's algorithm. VF3 [102] is the state of the art algorithm for subgraph isomorphism which uses heuristics to find matches quicker. If the project gains popularity, it is likely that better performance will become desirable, in this instance the VF3 algorithm will be explored.

Another limitation of Bilbo is that nodes require an identifier and a load. This was originally added to overcome the common language design problem of equality and identity [103]. On one hand, identifiers and loads separates the uniquely identifying characteristic of an entity (for example the name of city) from its transient properties (such as the city's population). On the other hand, not all entities can be split up in this way. A graph problem may require adding a

node to a graph where only its identifier, only its load, or only the nodes it is connects to are meaningful. In this case the programmer has to use a non-meaningful, arbitrary, identifier or load. In the pathological case, though very unlikely, the chosen identifier already exists in the graph and the (meaningful) load is overwritten with a potentially arbitrary load.

A feature which is currently under consideration, and a way to avoid this, is a construct for a *don't care* identifier. For example, consider the addition

```
g' = g + [_::100],
```

which adds to the graph `g` a node with a load equal to `100` and with an identifier that is chosen arbitrarily (by the interpreter) but is guaranteed not to clash with any existing identifiers in `g`. Some care is needed with the semantics regarding what happens if a node with the same identifier, as the don't care identifier, is added to the graph. The don't care identifier would need to be changed, making sure the new value does not clash with other existing identifiers.

8.4 Summary

Taking into account these achievements and limitations, as well as the evaluation, this work, in large, has been successful and fulfilled the original aims of the project.

The project is available at

`https://github.com/maccth/bilbo`

with instructions on the README.

Appendix A

Worksheet for Usability Testing Study

Thank for agreeing to do this worksheet!

- The questions are divided into two parts, the first part contains questions on Bilbo and the second on ReGraph.
- Please ensure you've read the accompanying cheat sheets for Bilbo and ReGraph before starting the questions. If anything is unclear on either sheet don't hesitate to drop me a message.ReGraph is a Python library so I've assumed some basic Python knowledge. Please do let me know if you are not comfortable reading and writing Python.
- You can use the cheat sheets at any time while answering the questions but please don't use the internet.
- Please write your answers on paper rather than on a computer.
- Since this worksheet is part of a project on programming language design, please do answer with syntactically correct code rather than pseudocode. Sorry to be pain!
- When starting and finishing the questions on Bilbo and ReGraph please write down the time with your answers. Anywhere it is necessary to do so, there will be an ominous box that looks like this:

Please indicate the time that you are starting this part of the worksheet.

- If possible, please answer the questions in one sitting. If that isn't possible, that's fine, just let me know and indicate with your answers the times you stopped and restarted.
- Lastly, there is a simple feedback questionnaire after the questions. This shouldn't take longer than a couple of minutes to do.

Part 1: Bilbo Questions

Information A number of questions in this part state *Given node* `a`, ... This means that you do not have to construct the node yourself, instead the node is already bound to the variable `a`. In this case, on a graph diagram the node would be labeled as `a`.

Please indicate the time that you are starting this part of the worksheet.

Question 1 Given nodes `a`, `b` and `c`, which program is syntactically correct?

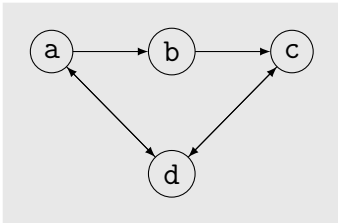
(a) `g = [(a > b), (b > c)]`

(b) `g = [a,>,b,>,c]`

(c) `g = [a > b > c]`

Question 2 Given nodes `a`, `b`, `c` and `d`, which program correctly constructs the graph `g` shown below?

g



(a) `g = [a,b,c] + [c,d] + [d,c] + [a,d] + [d,a]`

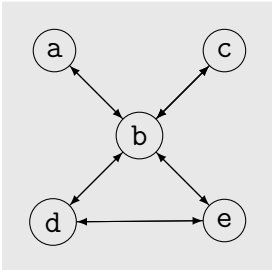
(b) `g = [a,>,b,>,c,>,d]`

(c) `g = [a,>,b,>,c] + [c,<>,d] + [d,<>,a]`

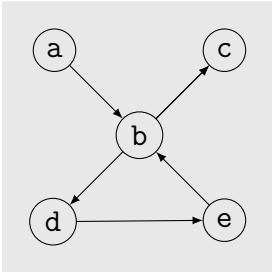
Question 3 Given nodes `a`, `b`, `c`, `d`, `e` and the graph `g` defined as below. Which graph is a valid representation of `g`?

```
g = [a,<>,b,<>,c] + [b,<>,d,<>,e,<>,b]
```

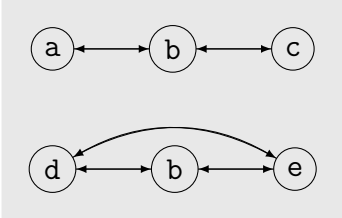
gA



gB



gC



(a) `gA`

(b) `gB`

(c) `gC`

Question 4 Which program constructs a node `n` with identifier `"nodeId"` and a load `Value(5)` where `Value` is a type defined as below.

```
type Value = val
```

(a)
```
n = Node("nodeId", Value(5))
```

(b)
```
n = "nodeId"::Value(5)
```

(c)
```
n.id = "nodeId"
n.load = Value(5)
```

Question 5 Given nodes `a`, `b`, `c`, `d`, `e` and the graph `g` defined as below. Which program constructs a graph that is identical to `g`?

```
g = [a, >, b, c, d, e, >, b, >, c]
```

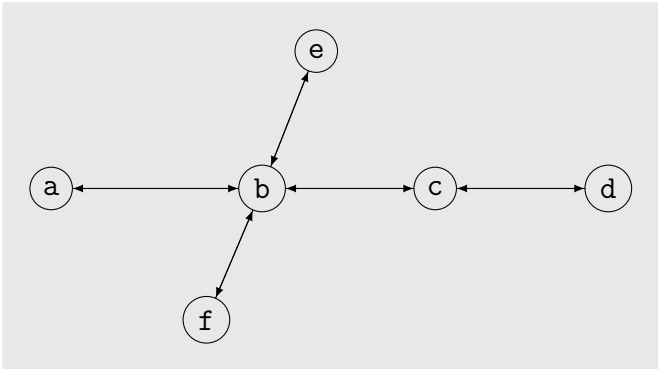
(a)
```
gA = [a,b] + [e,b] + [b,c] + [d]
```

(b)
```
gB = [a,>,b,>,e,>,c] + [d]
```

(c)
```
gC = [a,>,b,>,c] + [e,>,b] + [d]
```

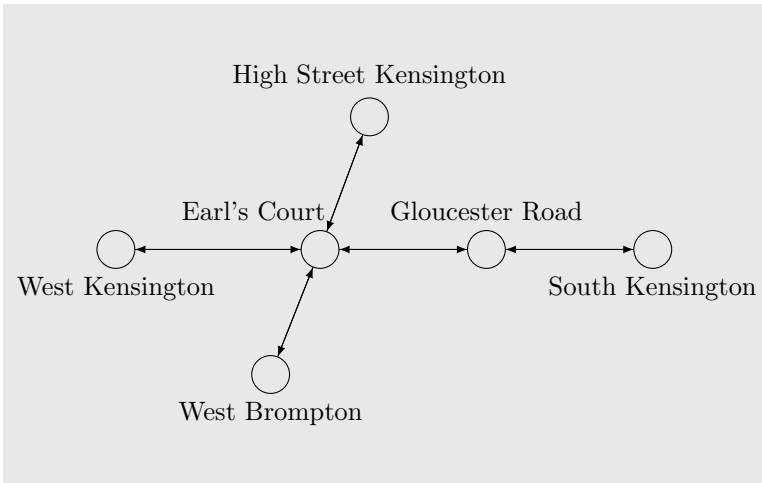
Question 6 Given nodes `a`, `b`, `c`, `d`, `e` and `f`. Construct the graph `g` shown below.

g



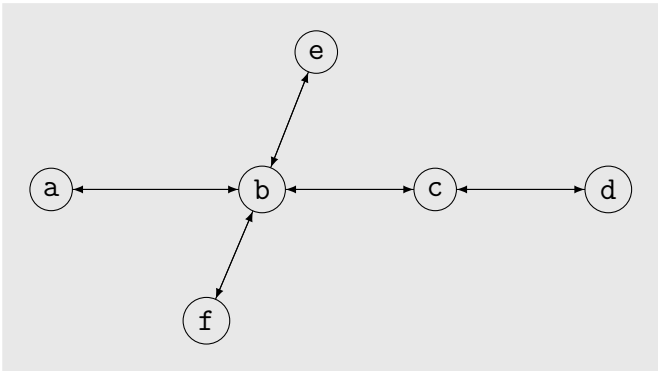
Question 7 The graph `g` shown below represents a (familiar) network of underground stations. Define a type `Station` with a single attribute `interchange` and construct the graph `g`. Each node should have a string identifier set to the name of the station and a load of type `Station`. The attribute `interchange` should be set to `True` if that station is an interchange station and `False` otherwise. Earl's Court, South Kensington, and West Brompton are the only interchange stations.

g

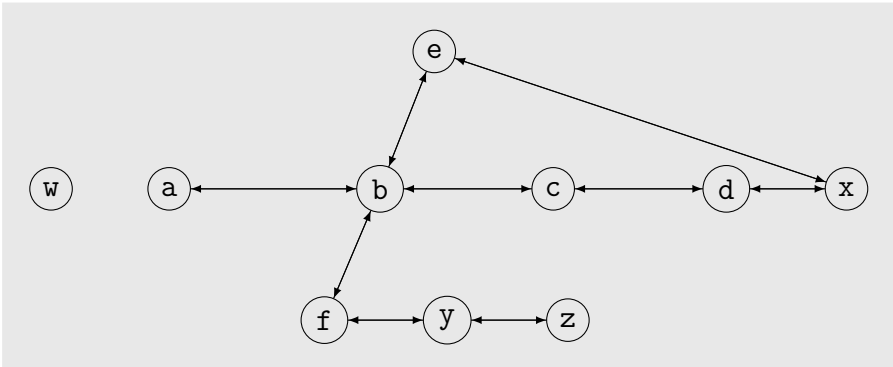


Question 8 Using the graph `g` shown below, construct the graph `h` also shown below. You are also given the nodes `a`, ..., `f` and `w`, `x`, `y`, `z`.

`g`



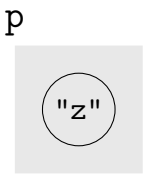
`h`



Please indicate the time that you finished this part of the worksheet.

Part 2: ReGraph Questions

Information The graph diagrams in this part label nodes with their identifier. For example, the graph diagram below shows a graph `p` that contains a single node with identifier `"z"`. Where attributes are required, it will be stated.



Furthermore, assume that the following two import statements always exist.

```
import networkx as nx
import regraph as re
```

Please indicate the time that you are starting this part of the worksheet.

Question 1 Assume the graph `g` exists and the nodes `"a"`, `"b"`, and `"c"` exist in the graph `g`. Assume further that `g` contains no edges. Which program is syntactically correct?

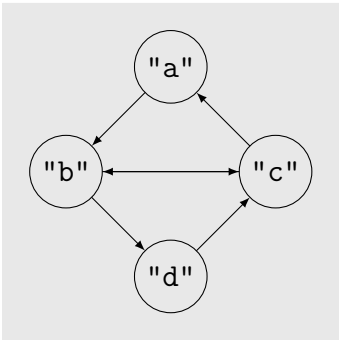
(a)
```
edges = [
    ("a", "b"),
    ("b", "c")
]
re.add_edges_from(g, edges)
```

(b)
```
edges = [
    ("a", "b")
    ("b", "c")
]
re.add_edges_from(g, edges)
```

(c)
```
edges = [
    ("a", "b"),
    ("b", "c")
]
re.add_edges_from(g edges)
```

Question 2 Which program correctly constructs the graph `g` shown below?

g



(a)
```
g = nx.DiGraph()

nodes = ["a", "b", "c", "d"]
edges = [
    ("a", "b"),
    ("b", "d"),
    ("d", "c"),
    ("c", "a"),
    ("b", "c"),
    ("c", "b")
]

re.add_nodes_from(g, nodes)
for edge in edges:
    re.add_node(g, edge)
```

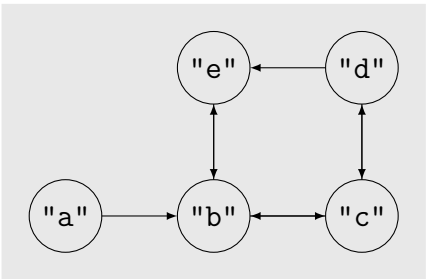
(b)
```
g = nx.DiGraph()

nodes = ["a", "b", "c", "d"]
edges = [
    ("a", "b"),
    ("b", "d"),
    ("d", "c"),
    ("c", "a"),
    ("b", "c"),
    ("c", "b")
]

re.add_nodes_from(g, nodes)
re.add_edges_from(g, edges)
```

(c)
```
g = nx.DiGraph()

re.add_edge(g, "a", "b")
re.add_edge(g, "b", "d")
re.add_edge(g, "d", "c")
re.add_edge(g, "c", "a")
re.add_edge(g, "b", "c")
re.add_edge(g, "c", "b")
```

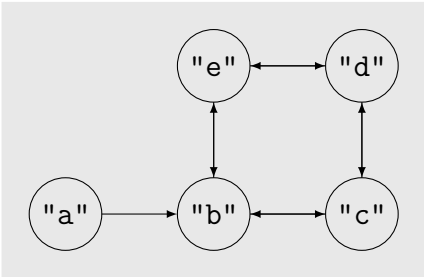
Question 3 Given the graph `g` defined as below. Which graph is a valid representation of `g`?

```
g = nx.DiGraph()

nodes = ["a", "b", "c", "d", "e"]
edges = [
    ("a", "b"),
    ("b", "c"),
    ("c", "b"),
    ("c", "d"),
    ("d", "c"),
    ("d", "e"),
    ("e", "d"),
    ("e", "b"),
    ("b", "e")
]

re.add_nodes_from(g, nodes)
re.add_edges_from(g, edges)
```

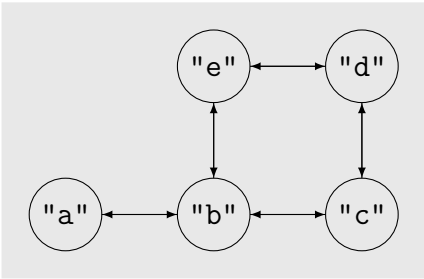
gA



gB



gC



(a) `gA`

(b) `gB`

(c) `gC`

Question 4 Assume the graph `g` exists and contains no nodes and no edges. Which program adds to the graph `g` a node with identifier `"nodeId"` and with a single attribute `value` equal to 10.

(a)
```
class Value(object):
    def __init__(self, value):
        self.value = value

attr = Value(10)
re.add_node(g, "nodeId", attr)
```

(b)
```
re.add_node(g, "nodeId", {"value": 10})
```

(c)
```
re.add_node(g, Node("nodeId", {"value": 10}))
```

Question 5 Given the graph `g` defined as below. Which program constructs a graph that is identical to `g`?

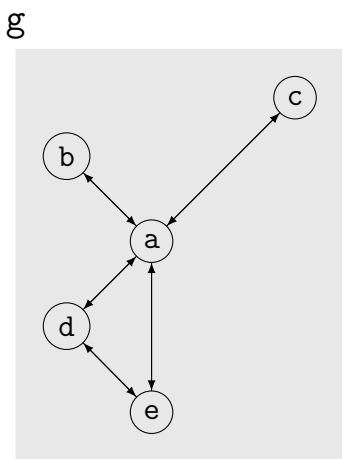
```
g = nx.DiGraph()

nodes = ["a", "b", "c", "d", "e", "f"]
edges = [("a", "b"), ("e", "d"), ("c", "b")]

re.add_nodes_from(g, nodes)
re.add_edges_from(g, edges)
```

(a)
```
gA = nx.DiGraph()

nodes = ["a", "b", "c", "d", "e", "f"]
edges = [("a", "b", "c"), ("e", "d")]

re.add_nodes_from(gA, nodes)
re.add_edges_from(gA, edges)
```

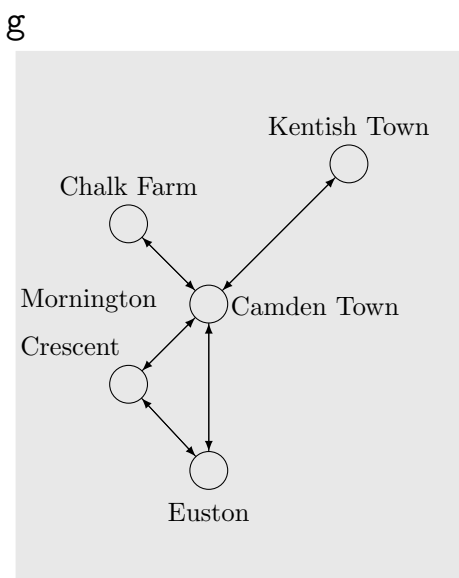
(b)
```
gB = nx.DiGraph()

nodes = ["a", "b", "c", "d", "e"]
edges = [("a", "b"), ("e", "d"), ("c", "b")]

re.add_nodes_from(gB, nodes)
re.add_edges_from(gB, edges)
```

(c)
```
gC = nx.DiGraph()

nodes = ["a", "b", "c", "d", "e", "f"]
edges = [("a", "b"), ("c", "b"), ("e", "d")]

re.add_nodes_from(gC, nodes)
re.add_edges_from(gC, edges)
```

Question 6 Construct the graph `g` shown below. Nodes should have string identifiers and no attributes.

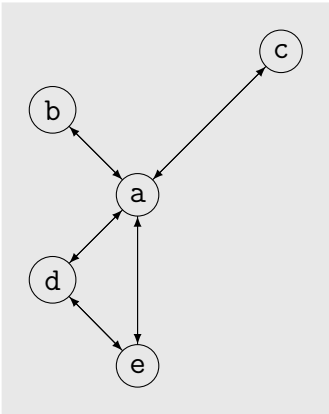


Question 7 The graph `g` shown below represents a network of underground stations. Construct the graph `g`. Each node should have a string identifier set to the name of the station and an attribute dictionary with a single attribute `interchange`. The attribute `interchange` should be set to `True` if that station is an interchange station and `False` otherwise. Camden Town, Kentish Town and Euston are the only interchange stations.

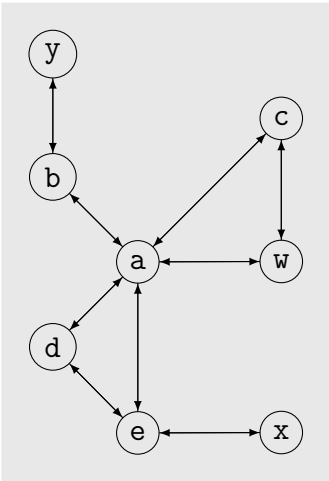


Question 8 Using the graph `g` shown below, construct the graph `h` also shown below. Nodes should have string identifiers and no attributes.

`g`



`h`



Please indicate the time that you finished this part of the worksheet.

Part 3: Feedback Questionnaire

(1) Do you feel that the level of familiarity with Python required to understand and answer the ReGraph questions was beyond your familiarity with Python?

(2) Did the cheat sheets or worksheet introduce any concepts (other than Bilbo and ReGraph) you were not familiar with? If so, do you feel that this made the worksheets more difficult?

Please answer the following questions on a scale of 1 to 5 inclusive, with 1 indicating *very low* and 5 indicating *very high*. Other comments are welcome for any question, but do also include a 1 to 5 answer.

(3) How would you rate the difficulty of the questions in part 1 (Bilbo questions)?

(4) How intuitive is Bilbo?

(5) How productive would you say Bilbo is?

(6) How would you rate the readability of Bilbo code?

(7) How would you rate your previous familiarity with ReGraph?

(8) How would you rate the difficulty of the questions in part 2 (ReGraph questions)?

(9) How intuitive is ReGraph?

(10) How productive would you say ReGraph is?

(11) How would you rate the readability of Python code that uses ReGraph?

The last two questions are concerned with the idea of *capturing a domain*. A tool is said to capture a domain well if, when using that tool, it is straightforward to express the important ideas in that domain. It is a measure of how effective a tool is in a given problem domain. For example, one may say that MATLAB captures the domain of linear algebra well since it is easy to express vector and matrix operations.

Please answer these questions using the same 1 to 5 scale as above.

(12) How well did Bilbo capture the domain of graphs?

(13) How well did ReGraph capture the domain of graphs?

Once again, thank you! Any further comments about Bilbo, ReGraph or the worksheet are more than welcome.

Appendix B

Worksheet Results

The table below, Table B.1, contains the results of the worksheet shown in Appendix A. Participants have been anonymised but given a number for identification purposes. Numbers were assigned to a candidates based on how long it took them to complete the Bilbo part of the worksheet. Therefore, participant number 1 completed this part of the worksheet in the shortest amount of time.

The Bilbo and ReGraph questions were answered correctly by all participants. As a result, their marks have been left out of the table below.

The column *Part 1 Time* refers to the time taken, in seconds to complete the first part of the worksheet, the question on Bilbo. Similarly with *Part 2 Time* and ReGraph.

The columns marked *F3* to *F13* refers to the answers of the feedback questionnaire, which had thirteen questions. The first two questions, regarding the participant's previous experience with Python and previous exposure to graph concepts were answered to the effect of *no*, by all participants. That is to say that no participant felt disadvantaged by their lack of Python knowledge or exposure to graph concepts. The answers for these two questions have thus been left out, as they are all the same.

Participant	Part 1 Time (s)	Part 2 Time (s)	F3	F4	F5	F6	F7	F8	F9	F10	F11	F12	F13
1	411	851	1	4	4	5	1	2	3	2	2	4	2
2	516	783	1	4	4	4	1	2	1	3	1	5	3
3	540	1200	2	4	4	4	1	3	2	3	2	4	2
4	600	960	2	4	4	4	1	3	2	1	1	4	2
5	600	1260	1	5	5	5	1	4	3	1	2	5	2
6	660	900	1	4	4	4	1	2	3	2	2	4	2
7	660	1080	1	5	4	5	1	2	2	2	2	5	2
8	660	1500	3	5	5	5	3	3	4	3	2	5	3
9	720	1500	2	4	5	4	1	2	2	2	2	4	2
10	780	1380	2	4	5	4	1	2	2	3	2	5	3

Table B.1: The answers to the feedback questionnaire and the times taken to answer the first two parts of the worksheet.

Appendix C

Worksheet Question Template

The template shown below in Fig C.1 was created before the Bilbo and ReGraph questions were written, to ensure consistency.

Bilbo and ReGraph Questions Template

(Q1) Which program is syntactically correct?

(Q2) Which program correctly constructs the graph G_1?

(Q3) Which pictured graph correctly represents the graph constructed in the program P_1?

(Q4) Which program correctly constructs a node holding the desired data?

(Q5) Which program constructs a graph equal to the graph constructed in program P_2?

(Q6) Construct the graph G_2 pictured.

(Q7) Construct the graph G_3 pictured, with nodes holding the desired data.

(Q8) Given the graph G_4 construct the graph G_5.

Fig C.1: The question template used for the Bilbo and ReGraph parts of the worksheet. G_1, G_2, G_3, G_4 and G_5 are used as placeholder graphs, P_1 and P_2 as placeholder programs. The full worksheet can be seen in Appendix A.

Appendix D

Bilbo Tutorial for Usability Testing Study

(1) Bilbo is a domain-specific language for programming with graphs.

(2) Nodes have an identifier and a load. The load is data relevant to that node. So if nodes were cities, the load could be the population. Identifiers are used to identify the node within a graph. More on that later. For now, we'll use strings as identifiers and integers as loads.

(3) Nodes are constructed using the `::` operator, with the identifier in front and the load behind.

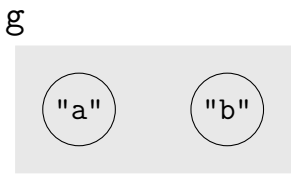
```
n1 = "a"::10
```

This constructs a node `n1` with a string identifier `"a"` and an integer load `10`.

(4) Graphs are constructed using path expressions (or just paths for short). Square brackets are used for paths and nodes are comma-separated within them.

```
n2 = "b"::20
g = [n1,n2]
```

This constructs the graph `g` shown below.



We'll label nodes with their identifiers and leave out the loads to keep the diagrams simple.

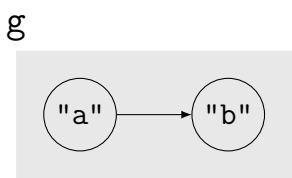
(5) The order of the nodes within a path does not matter.

```
g1 = [n1,n2]
g2 = [n2,n1]
```

Here, `g1` and `g2` are identical graphs. By identical we mean that they have the same nodes with the same identifiers and loads, and the same edges (none, edges are coming next).

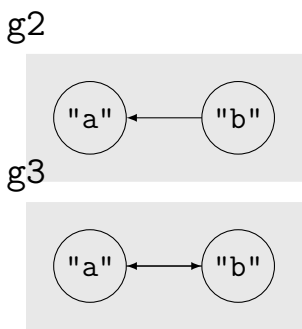
(6) Edges are constructed using the `>`, `<` and `<>` operators within a path. Edge operators must have a node either side and are comma-separated between the nodes.

```
g = [n1,>,n2]
```



Similarly, for `<` and `<>`.

```
g2 = [n1,<,n2]
g3 = [n1,<>,n2]
```



(7) Edges are unidirectional, so the `<>` operator is a short cut for creating the two edges separately.

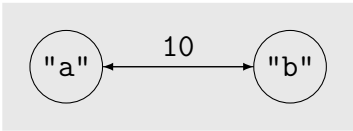
```
g1 = [n1,<>,n2]
g2 = [n1,>,n2,>,n1]
```

Here, `g1` and `g2` are identical but the construction of `g1` is easier to follow.

(8) Edges can also be weighted. So let's say it takes 10 minutes to get from `a` to `b` in either direction.

```
g = [n1,<10>,n2]
```

g

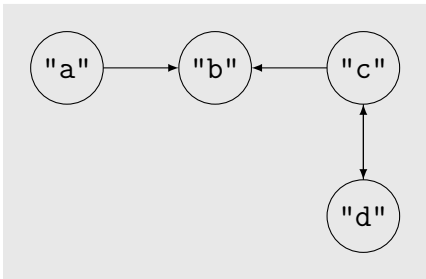


The other edge operators are weighted similarly, for example `10>` and `<10`.

(9) Longer paths can be constructed by chaining together multiple edges.

```
a = "a"::1
b = "b"::2
c = "c"::3
d = "d"::4
g = [a,>,b,<,c,<>,d]
```

g

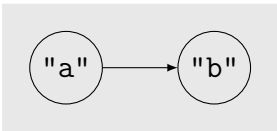


(10) Nodes can appear multiple times within a path. As long as those nodes have the same identifier, they will become one node in the graph.

```
g = [n1,>,n2,n1,n1,n1,n1]
```

This constructs a graph `g` with two nodes and one edge.

g

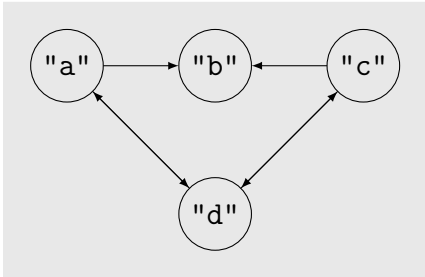


This is because all of the nodes bound to `n1` have the same identifier `"a"`. In general, nodes with the same identifier are the same node. This allows larger graphs and longer paths to be constructed easily. It also allows for intuitive graph addition (which we will meet in a moment).

(11) Since a node can appear multiple times in a path, cycles can be easily constructed.

```
g = [a,>,b,<,c,<>,d,<>,a]
```

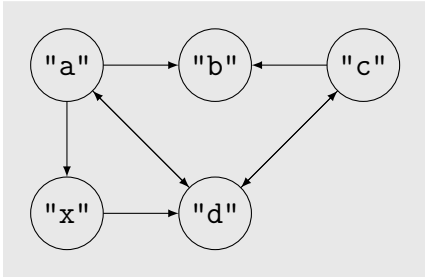
g



Here we see the node with identifier `"a"` appear twice in the path which forms the cycle.

(12) Graph addition allows larger graphs to be built up incrementally. Suppose, given the graph `g` above we wanted to construct the graph `h` shown below. Notice that `h` builds on `g`. This can be achieved using graph addition.

h



When two graphs are added together, nodes with the same identifier become the same node (like in paths). So given the graph `g`, we can construct `h` as shown below.

```
x = "x"::10
h = g + [a,>,x,>,d]
```

This works because `g` has nodes with identifiers `"a"` and `"d"`.

(13) For now we won't worry about graph subtraction or graph addition in the case where two nodes have the same identifier but different loads.

(14) In general, nodes can have objects of any type as their load, and objects of any type other than nodes and graphs as their identifier.

(15) Bilbo has integer, float, string, and boolean primitive types. Boolean values are `True` and `False`. Strings use double quotes (as we have seen).

(16) User defined types are created with the `type` keyword.

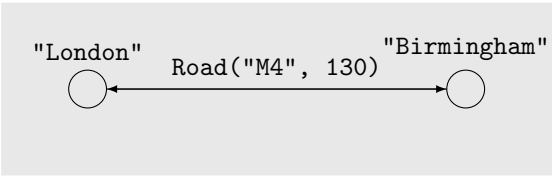
```
type City = population, capital
london = City(100, True)
```

Here, `City` is a type with two attributes and `london` is an object of type `City` with `london.population` equal to `100` and `london.capital` equal to `True`. Objects can be used for identifiers, loads and weights.

(17) We now use the type `City` together with another type `Road` to construct a graph.

```
type Road = name, distance

// Using the london object constructed above
ldn = "London"::london
bham = "Birmingham"::City(13, False)
m4 = Road("M4", 130)

g = [ldn, <m4>, bham]
```

g



(18) Edges can not be bound to a variable. The following construction is not valid.

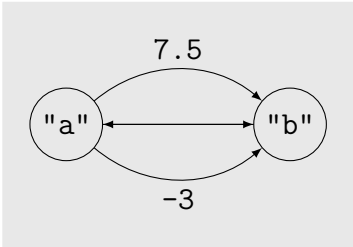
```
m4 = <Road("M4", 130)>
g = [ldn, m4, bham]
```

If this was valid it would decrease readability. It would be unclear whether `g` is a graph with three nodes (`ldn`, `m4`, `bham`) or a graph with two nodes (`ldn`, `bham`) and an edge (`m4`).

(19) Graphs are multigraphs, so a graph can have multiple edges in the same direction between a pair of nodes. The following construction is valid.

```
a = "a"::1
b = "b"::2
g = [a,7.5>,b] + [a,(-3)>,b] + [a,<>,b]
```

g



Note the use of brackets in the edge weight `(-3)>`. Weights can be freely bracketed.

Appendix E

ReGraph Tutorial for Usability Testing Study

(1) ReGraph[1] is a Python library for programming with graphs.

(2) ReGraph uses NetworkX[2] graphs under the hood so we'll assume that the following two import statements always exist.

```
import networkx as nx
import regraph as re
```

(3) We'll be using directed graphs, so we'll instantiate graphs as follows.

```
g = nx.DiGraph()
```

(4) Nodes have an identifier which can be any hashable Python type and an optional attribute dictionary (which is a standard Python dictionary) for data related to the node.

```
g = nx.DiGraph()
re.add_node(g, "a")
```

This adds the node with a string identifier `"a"` to the graph `g`. So `g` has a single node.

(5) The function `add_node` has an optional third argument for the node's attribute dictionary.

```
g = nx.DiGraph()
re.add_node(g, "a", {"value": 20})
```

This adds a node `"a"` with a single attribute `"value"` set to `20`.

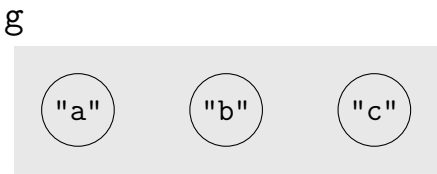
[1] `http://dev.executableknowledge.org/ReGraph/`. Some examples found in this document have been adapted from the official documentation.

[2] `https://networkx.github.io/documentation/stable/index.html`

(6) Multiple nodes can be added at the same time.

```
g = nx.DiGraph()
nodes = [
    ("a", {"name": "alice", "color": "blue"}),
    ("b", {"name": "bob", "color": "blue"}),
    ("c", {"name": "john", "color": "red"})
]
re.add_nodes_from(g, nodes)
```

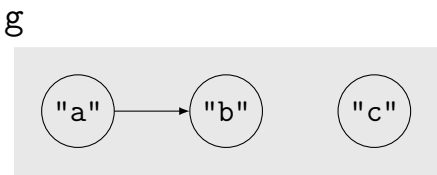
Note that `nodes` is a list of 2-tuples with the first element being the identifier and the second being the attribute dictionary. The graph `g` is shown below.



We'll label nodes with their identifiers and leave out the attribute dictionaries to keep the diagrams simple.

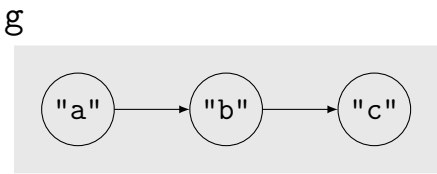
(7) Edges can be added in a similar manner. First, singly.

```
g = nx.DiGraph()
nodes = ["a", "b", "c"]
re.add_nodes_from(g, nodes)
re.add_edge(g, "a", "b")
```



Alternatively, from a list.

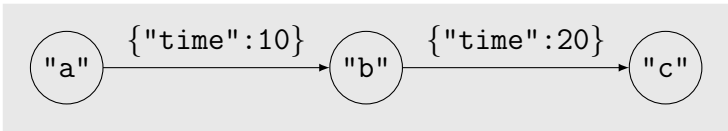
```
g = nx.DiGraph()
nodes = ["a", "b", "c"]
edges = [("a","b"), ("b","c")]
re.add_nodes_from(g, nodes)
re.add_edges_from(g, edges)
```



(8) Edges can have a weight in the form of an attribute dictionary, similar to that of nodes. So let's say it takes 10 minutes to get from `"a"` to `"b"` and 20 minutes to get from `"b"` to `"c"`.

```
g = nx.DiGraph()
nodes = ["a", "b", "c"]
edges = [
    ("a","b", {"time": 10}),
    ("b","c", {"time": 20})
]
re.add_nodes_from(g, nodes)
re.add_edges_from(g, edges)
```

g

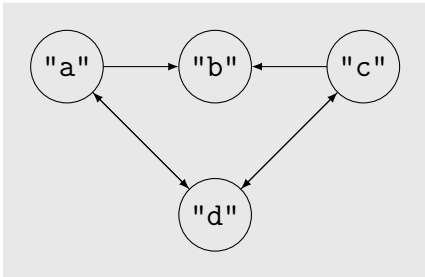


(9) Edges are unidirectional and graphs are simple. This means ReGraph does not allow multiple edges in the same direction between a pair of nodes.

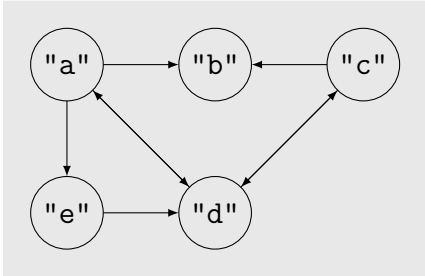
(10) Graphs can be merged by adding to one graph the nodes and edges of the other. However, because graphs are simple, before an edge can be added to a graph, it is necessary to check if that edge already exists in the graph. If it does, attempting to add it again will cause an error. A similar restriction applies to nodes. A node that already exists in a graph cannot be added again to that graph. Consider the graphs `g` and `h` below.

```
g = nx.DiGraph()
nodes = ["a", "b", "c", "d"]
edges = [
    ("a","b"),
    ("c","b"),
    ("c","d"), ("d","c"),
    ("a","d"), ("d","a")
]
re.add_nodes_from(g, nodes)
re.add_edges_from(g, edges)
```

g



h



The graph `h` can be constructed from `g` as shown below.

```
h = nx.DiGraph()
nodesH = ["a", "d", "e"]
edgesH = [("a","e"), ("e","d")]
re.add_nodes_from(h, nodesH)
re.add_edges_from(h, edgesH)

extraNodes = [n for n in g.nodes if n not in h.nodes]
extraEdges = [e for e in g.edges if e not in h.edges]

re.add_nodes_from(h, extraNodes)
re.add_edges_from(h, extraEdges)
```

Note the list comprehensions for `extraNodes` and `extraEdges` that check if the node or edge in question already exists in `h`, if it does, it will not be added again.

(11) The restriction that a node already in a graph cannot again be added to that graph applies based on the identifier. So if an identifier already exists in a graph, another node with that identifier cannot be added, even if they have different attributes. The construction below is not valid because a node with identifier `"a"` already exists in `g`.

```
g = nx.DiGraph()
nodes = ["a","b","c"]
re.add_nodes_from(g, nodes)
re.add_node(g, "a", {"name" : "node1"})
```

Appendix F

Bibliography

[1] D. Blostein, H. Fahmy, and A. Grbavec, "Issues in the practical use of graph rewriting," in *Graph Grammars and Their Application to Computer Science* (J. Cuny, H. Ehrig, G. Engels, and G. Rozenberg, eds.), (Berlin, Heidelberg), pp. 38–55, Springer Berlin Heidelberg, 1996.

[2] Y. Xu, "Graph databases burst into the mainstream," 2018.

[3] Transport for London, "Standard London Underground map - version A1, with larger type.." `https://tfl.gov.uk/info-for/suppliers-and-contractors/map-sizes-and-formats`.

[4] B.-H. Yoon, S.-K. Kim, and S.-Y. Kim, "Use of graph database for the integration of heterogeneous biological data," *Genomics & Informatics*, vol. 15, pp. 19–27, 03 2017.

[5] J. Daltio and C. B. Medeiros, "Hydrograph: Exploring geographic data in graph databases," in *GeoInfo*, 2015.

[6] D. Flora, "Graph databases – a natural way to represent data," 2016.

[7] A. Czerepicki, "Application of graph databases for transport purposes," *Bulletin of the Polish Academy of Sciences Technical Sciences*, vol. 64, no. 3, 2016.

[8] G. Sadowksi and P. Rathle, "Fraud detection: Discovering connections with graph databases," tech. rep., Neo4j, 2017.

[9] V. Manjalapur, *Some Advances in Graph Theory.* PhD thesis, 05 2018.

[10] N. Yuhanna, G. Leganza, and J. Lee, "Vendor landscape: Graph databases," 2017.

[11] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring Network Structure, Dynamics, and Function using NetworkX," in *Proceedings of the 7th Python in Science Conference* (G. Varoquaux, T. Vaught, and J. Millman, eds.), (Pasadena, CA USA), pp. 11 – 15, 2008.

[12] Y. Zhang, M. Yang, R. Baghdadi, S. Kamil, J. Shun, and S. Amarasinghe, "GraphIt: A High-performance Graph DSL," *Proc. ACM Program. Lang.*, vol. 2, pp. 121:1–121:30, Oct. 2018.

[13] R. Machado, L. Ribeiro, and R. Heckel, "Rule-based transformation of graph rewriting rules: Towards higher-order graph grammars," *Theoretical Computer Science*, vol. 594, 02 2015.

[14] A. M. Hein, "Identification and Bridging of Semantic Gaps in the Context of Multi-Domain Engineering," *Forum on Philosophy, Engineering & Technology*, Jan 2010.

[15] Neo4j, "What is a Graph Database?." `https://neo4j.com/developer/graph-database/`.

[16] Neo4j, "The Cypher Query Language." `https://neo4j.com/developer/cypher-query-language/`.

[17] Apache TinkerPop, "The Gremlin Graph Traversal Machine and Language." `https://tinkerpop.apache.org/gremlin.html`.

[18] W3C, "SPARQL Query Language for RDF." `https://www.w3.org/TR/rdf-sparql-query/`.

[19] T. Jelvis, "What is an embedded domain-specific language?." `https://www.quora.com/What-is-an-embedded-domain-specific-language`.

[20] J. Siek, L.-Q. Lee, and A. Lumsdaine, "Boost Graph Library (BGL)." `http://www.boost.org/libs/graph/`, June 2018.

[21] S. Hong, H. Chafi, E. Sedlar, and K. Olukotun, "Green-Marl: A DSL for Easy and Efficient Graph Analysis," *SIGARCH Comput. Archit. News*, vol. 40, pp. 349–362, Mar. 2012.

[22] J. Shun and G. E. Blelloch, "Ligra: A Lightweight Graph Processing Framework for Shared Memory," *SIGPLAN Not.*, vol. 48, pp. 135–146, Feb. 2013.

[23] A. Sch Urr, A. J. Winter, A. Zündorf, and A. Softwaretechnik, "Handbook of graph grammars and computing by graph transformation," ch. The PROGRES Approach: Language And Environment, pp. 551–603, River Edge, NJ, USA: World Scientific Publishing Co., Inc., 1999.

[24] C. Ermel, M. Rudolf, and G. Taentzer, "Handbook of graph grammars and computing by graph transformation," ch. The AGG Approach: Language and Environment, pp. 551–603, River Edge, NJ, USA: World Scientific Publishing Co., Inc., 1999.

[25] M. Bauderon and H. Jacquet, "Pullback as a generic graph rewriting mechanism," *Applied Categorical Structures*, vol. 9, pp. 65–82, 01 2001.

[26] G. T. Leavens, "Major Programming Paradigms." http://www.eecs.ucf.edu/~leavens/ComS541Fall97/hw-pages/paradigms/major.html.

[27] S. Mennicke, J.-C. Kalo, and W.-T. Balke, "Querying graph databases: What do graph patterns mean?," in *Conceptual Modeling* (H. C. Mayr, G. Guizzardi, H. Ma, and O. Pastor, eds.), (Cham), pp. 134–148, Springer International Publishing, 2017.

[28] C. H. Papadimitriou, "Computational complexity," in *Encyclopedia of Computer Science*, pp. 260–265, Chichester, UK: John Wiley and Sons Ltd.

[29] F. Parisi-Presicce and B. Westfechtel, eds., *Graph Transformation - 8th International Conference, ICGT 2015, Held as Part of STAF 2015, L'Aquila, Italy, July 21-23, 2015. Proceedings*, vol. 9151 of *Lecture Notes in Computer Science*, Springer, 2015.

[30] H. Ehrig, C. Ermel, U. Golas, and F. Hermann, *Graph and Model Transformation: General Framework and Applications.* Monographs in Theoretical Computer Science. An EATCS Series, Berlin, Heidelberg: Springer Berlin Heidelberg, 1st ed. 2015 ed., 2015.

[31] A. Corradini, U. Montanari, F. Rossi, H. Ehrig, R. Heckel, and M. Löwe, "Handbook of graph grammars and computing by graph transformation," ch. Algebraic Approaches to Graph Transformation. Part I: Basic Concepts and Double Pushout Approach, pp. 163–245, River Edge, NJ, USA: World Scientific Publishing Co., Inc., 1997.

[32] H. Ehrig, M. Korff, and M. Löwe, "Tutorial introduction to the algebraic approach of graph grammars based on double and single pushouts," in *Graph Grammars and Their Application to Computer Science* (H. Ehrig, H.-J. Kreowski, and G. Rozenberg, eds.), (Berlin, Heidelberg), pp. 24–37, Springer Berlin Heidelberg, 1991.

[33] G. for Geeks, "Backtracking Algorithms." https://www.geeksforgeeks.org/backtracking-algorithms/.

[34] M. Burgin, "The Rise and Fall of the Church-Turing Thesis," *CoRR*, vol. cs.CC/0207055, 2002.

[35] C. Böhm and G. Jacopini, "Flow Diagrams, Turing Machines and Languages with Only Two Formation Rules," *Commun. ACM*, vol. 9, pp. 366–371, May 1966.

[36] S. Zeil, "Turing Completeness." https://www.cs.odu.edu/~zeil/cs390/latest/Public/turing-complete/index.html#3.

[37] A. Habel and D. Plump, "Computational completeness of programming languages based on graph transformation," in *Foundations of Software Science and Computation Structures* (F. Honsell and M. Miculan, eds.), (Berlin, Heidelberg), pp. 230–245, Springer Berlin Heidelberg, 2001.

[38] A. Fuggetta, "A Classification of CASE Technology," vol. 26, (Los Alamitos, CA, USA), pp. 25–38, IEEE Computer Society Press, Dec. 1993.

[39] A. Schürr, "Programmed Graph Replacement Systems," in *In Rozenberg, G. (Ed.), Handbook on Graph Grammars: Foundations*, pp. 479–546, World Scientific, 1997.

[40] Oracle, "Java." `https://www.oracle.com/java/`.

[41] G. Taentzer, "Agg: A graph transformation environment for modeling and validation of software," in *Applications of Graph Transformations with Industrial Relevance* (J. L. Pfaltz, M. Nagl, and B. Böhlen, eds.), (Berlin, Heidelberg), pp. 446–453, Springer Berlin Heidelberg, 2004.

[42] U. Nickel, J. Niere, and A. Zündorf, "The FUJABA environment," *Proceedings - International Conference on Software Engineering*, pp. 742–745, Feb. 2000.

[43] T. Hettel, M. Lawley, and K. Raymond, "Model synchronisation: Definitions for round-trip engineering," in *Theory and Practice of Model Transformations* (A. Vallecillo, J. Gray, and A. Pierantonio, eds.), (Berlin, Heidelberg), pp. 31–45, Springer Berlin Heidelberg, 2008.

[44] T. Fischer, J. Niere, L. Torunski, and A. Zündorf, "Story Diagrams: A New Graph Rewrite Language Based on the Unified Modeling Language and Java," in *Theory and Application of Graph Transformations* (H. Ehrig, G. Engels, H.-J. Kreowski, and G. Rozenberg, eds.), (Berlin, Heidelberg), pp. 296–309, Springer Berlin Heidelberg, 2000.

[45] R. Geiß, G. V. Batz, D. Grund, S. Hack, and A. Szalkowski, "GrGen: A Fast SPO-Based Graph Rewriting Tool," in *Graph Transformations* (A. Corradini, H. Ehrig, U. Montanari, L. Ribeiro, and G. Rozenberg, eds.), (Berlin, Heidelberg), pp. 383–397, Springer Berlin Heidelberg, 2006.

[46] J. Blomer and R. Geiß, "The GrGen.NET User Manual." `http://www.info.uni-karlsruhe.de/software/grgen/GrGenNET-Manual.pdf`, July 2018.

[47] D. Plump, "The Design of GP 2," in *Proceedings 10th International Workshop on Reduction Strategies in Rewriting and Programming, WRS 2011, Novi Sad, Serbia, 29 May 2011.* (S. Escobar, ed.), vol. 82 of *EPTCS*, pp. 1–16, 2011.

[48] Kappa-Dev GitHub Organisation, "ReGraph." `http://dev.executableknowledge.org/ReGraph/`.

[49] A. Corradini, T. Heindel, F. Hermann, and B. König, "Sesqui-pushout rewriting," in *Graph Transformations* (A. Corradini, H. Ehrig, U. Montanari, L. Ribeiro, and G. Rozenberg, eds.), (Berlin, Heidelberg), pp. 30–45, Springer Berlin Heidelberg, 2006.

[50] M. Mann, H. Ekker, and C. Flamm, "The Graph Grammar Library - A Generic Framework for Chemical Graph Rewrite Systems," in *Theory and Practice of Model Transformations* (K. Duddy and G. Kappel, eds.), (Berlin, Heidelberg), pp. 52–53, Springer Berlin Heidelberg, 2013.

[51] F. Lanotte, "How to reduce the gap between OLTP and OLAP graph solutions with Spark-Tinkerpop." `http://cgnal.com/blog/how-to-reduce-the-gap-between-oltp-and-olap-graph-solutions-with-spark-tinkerpop/`.

[52] F. Vázquez, "Graph Databases. What's the Big Deal?." `https://towardsdatascience.com/graph-databases-whats-the-big-deal-ec310b1bc0ed`.

[53] D. Spinellis, "Notable Design Patterns for Domain Specific Languages," *Journal of Systems and Software*, vol. 56, pp. 91–99, Feb. 2001.

[54] G. Karsai, H. Krahn, C. Pinkernell, B. Rumpe, M. Schindler, and S. Völkel, "Design guidelines for domain specific languages," Sept. 2014.

[55] D. L. Parnas, "On the Criteria to Be Used in Decomposing Systems into Modules," *Commun. ACM*, vol. 15, pp. 1053–1058, Dec. 1972.

[56] Joy Chao, "Imperative vs. Declarative Query Languages: What's the Difference?." `https://neo4j.com/blog/imperative-vs-declarative-query-languages/`.

[57] T. Kosar, M. Mernik, M. Črepinšek, P. R. Henriques, D. da Cruz, M. J. V. Pereira, and N. Oliveira, "Influence of domain-specific notation to program understanding," in *2009 International Multiconference on Computer Science and Information Technology*, pp. 675–682, Oct 2009.

[58] C. Bak, G. Faulkner, D. Plump, and C. Runciman, "A Reference Interpreter for the Graph Programming Language GP 2," *Electronic Proceedings in Theoretical Computer Science*, vol. 181, 04 2015.

[59] R. Heckel, J. M. Küster, and G. Taentzer, "Confluence of Typed Attributed Graph Transformation Systems," in *Graph Transformation* (A. Corradini, H. Ehrig, H. J. Kreowski, and G. Rozenberg, eds.), (Berlin, Heidelberg), pp. 161–176, Springer Berlin Heidelberg, 2002.

[60] G. Bergami, M. Magnani, and D. Montesi, "On Joining Graphs," 08 2016.

[61] D. P. Friedman and M. Wand, *Essentials of Programming Languages, 3rd Edition*. The MIT Press, 3 ed., 2008.

[62] Comp.Lang.Visual, "What is the Deutsch Limit? - comp.lang.visual Frequently-Asked Questions (FAQ)." `http://www.faqs.org/faqs/visual-lang/faq/`.

[63] A. Maruch, "Typing: Strong vs. Weak, Static vs. Dynamic." https://www.artima.com/weblogs/viewpost.jsp?thread=7590.

[64] E. Meijer and P. Drayton, "Static typing where possible, dynamic typing when needed: The end of the cold war between programming languages,"

[65] Amazon, "Amazon Neptune." https://aws.amazon.com/neptune/.

[66] User: Skrew Everything, "OOP: Everything you need to know about Object Oriented Programming." https://medium.com/from-the-scratch/oop-everything-you-need-to-know-about-object-oriented-programming-aee3c18e281b.

[67] O. Kiselyov, R. Lämmel, and K. Schupke, "Strongly Typed Heterogeneous Collections," in *Proceedings of the 2004 ACM SIGPLAN Workshop on Haskell*, Haskell '04, (New York, NY, USA), pp. 96–107, ACM, 2004.

[68] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools (2Nd Edition)*. Boston, MA, USA: Addison-Wesley Longman Publishing Co., Inc., 2006.

[69] B. Brock, "reple: Replay-Based REPLs for Compiled Languages." https://people.eecs.berkeley.edu/~brock/blog/reple.php.

[70] Visual FSharp Team, "Tail calls in F#." https://blogs.msdn.microsoft.com/fsharpteam/2011/07/08/tail-calls-in-f/.

[71] Wikibooks, "F Sharp Programming/Recursion." https://en.wikibooks.org/w/index.php?title=F_Sharp_Programming/Recursion&oldid=3263225, 2017.

[72] H. Feldt, "Expecto." https://github.com/haf/expecto.

[73] FsCheck Organisation, "FsCheck." https://fscheck.github.io/FsCheck/.

[74] S. Tolksdorf, "FParsec." http://www.quanttec.com/fparsec/.

[75] Python Software Foundation, "Why does Python allow commas at the end of lists and tuples?." https://docs.python.org/3/faq/design.html.

[76] Tine, "Neural Networks in JavaScript." https://blog.webkid.io/neural-networks-in-javascript/.

[77] B. Ruth, "JSON Graph Spec." https://github.com/bruth/json-graph-spec.

[78] Real-Time and Embedded Systems, Kiel University, "JSON Graph Format." https://rtsys.informatik.uni-kiel.de/confluence/display/KIELER/JSON+Graph+Format.

[79] Anthony Bargnesi, "JSON Graph Format (JGF). Detailing the specification of the standard JSON Graph Format for Network data interchange.." http://jsongraphformat.info.

[80] M. Franz, C. T. Lopes, G. Huck, Y. Dong, O. Sumer, and G. D. Bader, "Cytoscape.js: a graph theory library for visualisation and analysis," *Bioinformatics*, vol. 32, pp. 309–311, 09 2015.

[81] D. Plump, "Checking Graph-Transformation Systems for Confluence," *ECEASST*, vol. 26, Jan 2010.

[82] D. Plump, "Processes, terms and cycles," ch. Confluence of Graph Transformation Revisited, pp. 280–308, Berlin, Heidelberg: Springer-Verlag, 2005.

[83] Wikipedia contributors, "Dead-end pages — Wikipedia, the free encyclopedia." `"https://en.wikipedia.org/wiki/Wikipedia:Dead-end_pages"`.

[84] M. Oldfield, "Church Numerals." `https://mjoldfield.com/atelier/2011/01/church-numerals.html`.

[85] Open Source Initiative, "The MIT License." `https://opensource.org/licenses/MIT`.

[86] J. R. Ullmann, "An Algorithm for Subgraph Isomorphism," *J. ACM*, vol. 23, pp. 31–42, Jan. 1976.

[87] S. Gilham, "AltCover." `https://github.com/SteveGilham/altcover`.

[88] Smart Bear, "Negative Testing." `https://smartbear.com/learn/automated-testing/negative-testing/`.

[89] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms, Third Edition*. The MIT Press, 3rd ed., 2009.

[90] D. Z. Chen, "Developing algorithms and software for geometric path planning problems," *ACM Comput. Surv.*, vol. 28, Dec. 1996.

[91] R. Karp, "Reducibility Among Combinatorial Problems," vol. 40, pp. 85–103, 01 1972.

[92] G. J. Chaitin, "Register allocation & spilling via graph coloring," *SIGPLAN Not.*, vol. 17, pp. 98–101, June 1982.

[93] F. J. Aragón Artacho and R. Campoy, "Solving Graph Coloring Problems with the Douglas-Rachford Algorithm," *Set-Valued and Variational Analysis*, vol. 26, p. 277–304, 06 2018.

[94] T. Kosar, M. Mernik, and J. C. Carver, "Program comprehension of domain-specific and general-purpose languages: comparison using a family of experiments," *Empirical Software Engineering*, vol. 17, pp. 276–304, Jun 2012.

[95] D. Kuhlman, *A python book: Beginning python, advanced python, and python exercises*. Platypus Global Media, 2009.

[96] S. Krug, *Don'T Make Me Think: A Common Sense Approach to the Web (2Nd Edition)*. Thousand Oaks, CA, USA: New Riders Publishing, 2005.

[97] D. I. K. Sjoberg, J. E. Hannay, O. Hansen, V. By Kampenes, A. Karahasanovic, N.-K. Liborg, and A. C. Rekdal, "A Survey of Controlled Experiments in Software Engineering," *IEEE Trans. Softw. Eng.*, vol. 31, pp. 733–753, Sept. 2005.

[98] J. Carver, M. L. Jaccheri, S. Morasca, and F. Shull, "Issues in using students in empirical studies in software engineering education," pp. 239– 249, 10 2003.

[99] Official Journal of the European Union, "On the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation)," April 2016.

[100] TextMate, "TextMate Language Grammars." `https://macromates.com/manual/en/language_grammars`.

[101] G. D. Plotkin, "A structural approach to operational semantics," *J. Log. Algebr. Program.*, vol. 60-61, pp. 17–139, 07 2004.

[102] V. Carletti, P. Foggia, A. Saggese, and M. Vento, "Introducing VF3: A new algorithm for subgraph isomorphism," in *Graph-Based Representations in Pattern Recognition - 11th IAPR-TC-15 International Workshop, GbRPR 2017, Anacapri, Italy, May 16-18, 2017, Proceedings*, pp. 128–139, 2017.

[103] S. Ochsenreither, "Language Design: Equality & Identity." `https://soc.github.io/languages/equality-and-identity-part1.html`.